Deep Learning-Based Segmentation for Neuronal Cell Detection in Fluorescent Microscopy Images

Gabriele Pascali, Gonçalo Araújo, João Galvão

A. Introduction

1. What is Image Segmentation?

Image segmentation is a fundamental technique in computer vision and digital image processing that partitions an image into distinct regions based on pixel characteristics such as color, texture, or intensity. Unlike image classification, which assigns a single label to an entire image, segmentation combines classification and localization by identifying not only what is present but also where it appears, outlining object boundaries at the pixel level.

This process is essential for tasks that require detailed understanding of visual content, including:

- Object detection: Identifying and classifying objects within specific regions.
- Region identification: Differentiating areas based on visual features.
- Advanced image processing: Supporting applications such as medical imaging, autonomous vehicles, and scene understanding.

Segmentation typically begins by converting an image into a set of labeled regions or a segmentation mask. Techniques often rely on detecting abrupt changes in pixel values (edges), which mark boundaries between different regions. By isolating relevant segments, image segmentation enables more efficient and targeted processing, making it a cornerstone of modern visual data interpretation.

2. Traditional vs. Deep Learning-Based Segmentation

🔹 Traditional Methods

Classical segmentation methods rely on basic pixel-level features such as:

- Color
- Brightness
- Contrast
- Intensity

These approaches are **computationally efficient**, require minimal training, and are well-suited for simpler tasks like basic semantic classification.

Common classical techniques:

- Thresholding: One of the simplest techniques, where pixels are divided into regions based on a predefined intensity threshold.
- Histograms: Analyze the distribution of pixel values to identify clusters (e.g., peaks and valleys), using features like color or intensity. Efficient due to requiring only a single pass through the image.
- Edge detection: Uses filters (e.g., Sobel, Canny) to detect sharp changes in intensity, highlighting object boundaries by computing image gradients along the x and y axes.
- Watershed algorithms: Treats the image as a topographic surface, "flooding" it from minima to form catchment basins, effectively segmenting based on gradient and topology.
- Region-based segmentation: Starts with seed pixels and grows regions by merging adjacent pixels with similar properties.
- Clustering methods: Group pixels into clusters based on similarity in features such as color, intensity, or texture (e.g., k-means, mean-shift).

🔹 Deep Learning-Based Methods

Deep learning–based segmentation leverages neural networks, particularly convolutional neural networks (CNNs), to automatically identify and delineate objects or regions within images. These models learn complex, hierarchical features from large annotated datasets, allowing for highly precise and context-aware segmentation. Applications span diverse domains, including medical imaging, autonomous driving, and remote sensing. Most segmentation networks output a multi-channel (n-channel) binary format, also referred to as a 2D one-hot encoded mask, where each channel corresponds to a specific class.

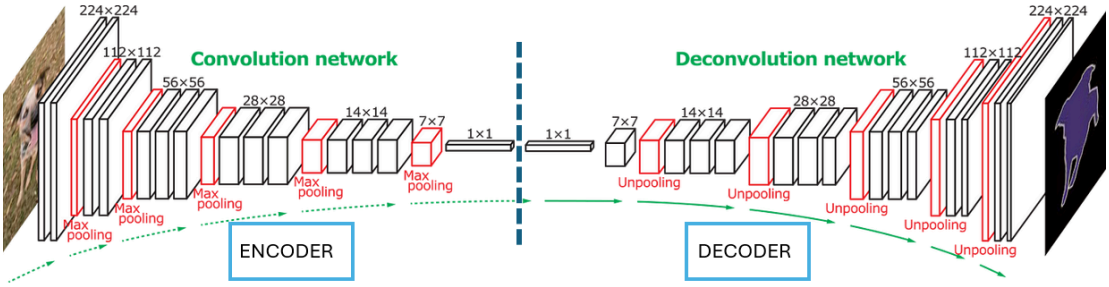
Segmentation networks typically follow an encoder-decoder architecture:

- The encoder extracts feature representations through convolution and downsampling.
- A bottleneck compresses this representation.
- The decoder reconstructs the spatial structure via upsampling, producing the segmentation mask.

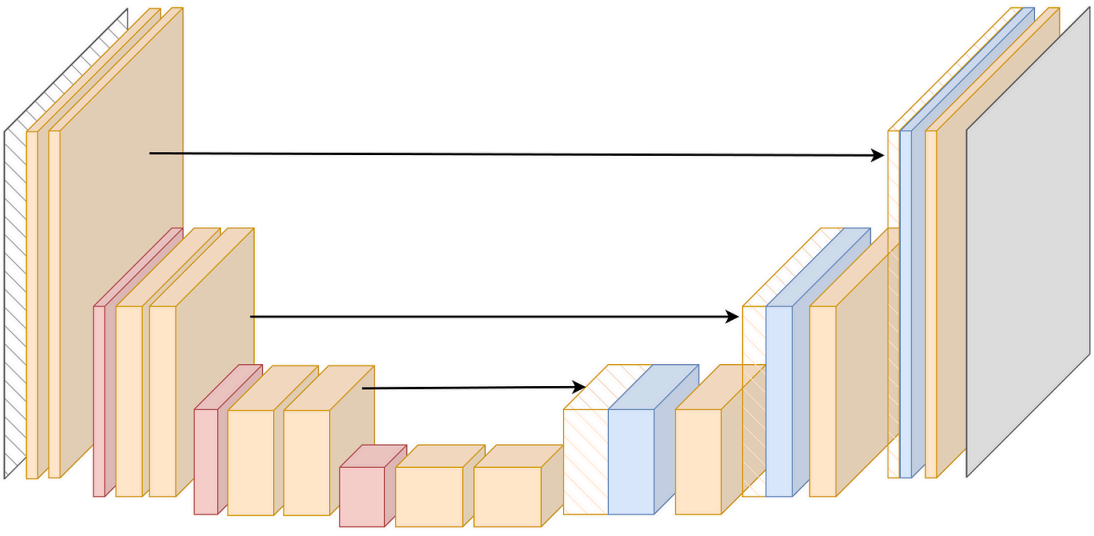
Most segmentation networks output a multi-channel (n-channel) binary format, also referred to as a 2D one-hot encoded mask, where each channel corresponds to a specific class.

Popular deep learning segmentation models:

- **Fully Convolutional Networks (FCNs)**
  Pioneered the use of convolutional layers for segmentation without fully connected layers. The decoder upsamples the encoded features to generate pixel-wise predictions.



- **U-Net**
  Designed for biomedical image segmentation, U-Net added skip connections, which pass feature maps directly from encoder layers to corresponding decoder layers. This mitigates information loss during downsampling and enables more accurate boundary localization. This is the key advantage over FCN, which lacks such detailed skip connections and often produces coarser segmentations.



- **SegNet** Introduced a structured encoder-decoder network using pooling indices to guide upsampling, helping retain spatial precision in segmentation masks.
- **Mask R-CNN**
  Extends Faster R-CNN for instance segmentation by adding a parallel branch to predict segmentation masks alongside object detection.
- **CenterMask2** Builds on Mask R-CNN and combines instance segmentation with real-time object detection, introducing a spatial attention-guided mask head for better mask quality with efficient inference
- **Detectron2** A modular, flexible PyTorch-based framework developed by Facebook AI Research. It supports various state-of-the-art segmentation models (including Mask R-CNN and CenterMask2) and offers easy extensibility for custom segmentation pipelines.

Common Loss Functions in Image Segmentation

Loss functions play a crucial role in training segmentation networks by quantifying the difference between predicted masks and ground truth labels. The choice of loss function affects convergence, class balance, and the final segmentation quality. Below are some of the most widely used loss functions in image segmentation tasks:

1. Binary Cross-Entropy (BCE) Loss

Used in binary (foreground/background) segmentation tasks.

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)\right]$$

Where:

- y_i: Ground truth label (0 or 1)
- $\hat{y}_i$: Predicted probability
- N: Number of pixels

2. Categorical Cross-Entropy (CCE) Loss

Used in multi-class segmentation, where each pixel belongs to one of multiple classes.

$$\mathcal{L}_{\text{CCE}} = -\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c} \cdot \log(\hat{y}_{i,c})$$

Where:

- $y_{i,c}$: One-hot encoded true label for class c
- $\hat{y}_{i,c}$: Predicted probability for class c
- C: Number of classes

3. Dice Loss

Measures overlap between predicted and ground truth masks, often used in medical imaging.

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i y_i \hat{y}_i + \epsilon}{\sum_i y_i + \sum_i \hat{y}_i + \epsilon}$$

Where:

- ϵ: Smoothing constant (to avoid division by zero)

4. Jaccard Loss (IoU Loss)

Related to Dice Loss, but penalizes false positives more heavily.

$$\mathcal{L}_{\text{IoU}} = 1 - \frac{\sum_i y_i \hat{y}_i}{\sum_i y_i + \sum_i \hat{y}_i - \sum_i y_i \hat{y}_i}$$

5. Focal Loss

Designed to handle class imbalance by focusing training on hard-to-classify examples.

$$\mathcal{L}_{\text{Focal}} = -\alpha(1 - \hat{y}_i)^\gamma y_i \log(\hat{y}_i)$$

Where:

- α: Balancing factor
- γ: Focusing parameter (e.g., 2.0)

6. Combo Loss

Combines multiple losses (e.g., BCE + Dice) to leverage their strengths.

$$\mathcal{L}_{\text{Combo}} = \lambda \cdot \mathcal{L}_{\text{BCE}} + (1 - \lambda) \cdot \mathcal{L}_{\text{Dice}}$$

Where:

- λ: Weight factor to balance both losses

---

3. Types of Image Segmentation

I. Semantic Segmentation

Semantic segmentation involves classifying every pixel in an image into a predefined class label. All pixels belonging to a specific class—such as "car," "road," or "sky"—are grouped together, without distinguishing between different instances of the same class.

- Assigns a class label to each pixel.
- Does not differentiate between separate instances of the same object.
- Suitable for labeling both "things" (countable objects) and "stuff" (uncountable regions like grass or sky), but only categorically.

📌 *Example*: All cars in a street scene are labeled as “car,” regardless of how many or where they are.

⚠️ *Limitation*: In scenes with multiple objects of the same class close together (e.g., a crowd), the output lacks detail about individual instances. Cannot separate overlapping objects of same class.

II. Instance Segmentation

Instance segmentation takes pixel-level classification further by identifying individual objects within the same class. While semantic segmentation merges all objects of the same class, instance segmentation separates them into unique instances—even if they overlap.

- Distinguishes between individual objects of the same class.
- Prioritizes separating object boundaries, even without always identifying class labels (in certain formulations).
- Focuses on segmenting "things" (countable objects) precisely.
- Provides pixel-perfect masks rather than bounding boxes (as in object detection).

📌 *Example*: Each pedestrian in a crowd or each parked car is segmented as a distinct object, even when overlapping.

⚠️ *Limitation*: Struggles with occlusion, touching objects, and ignores background

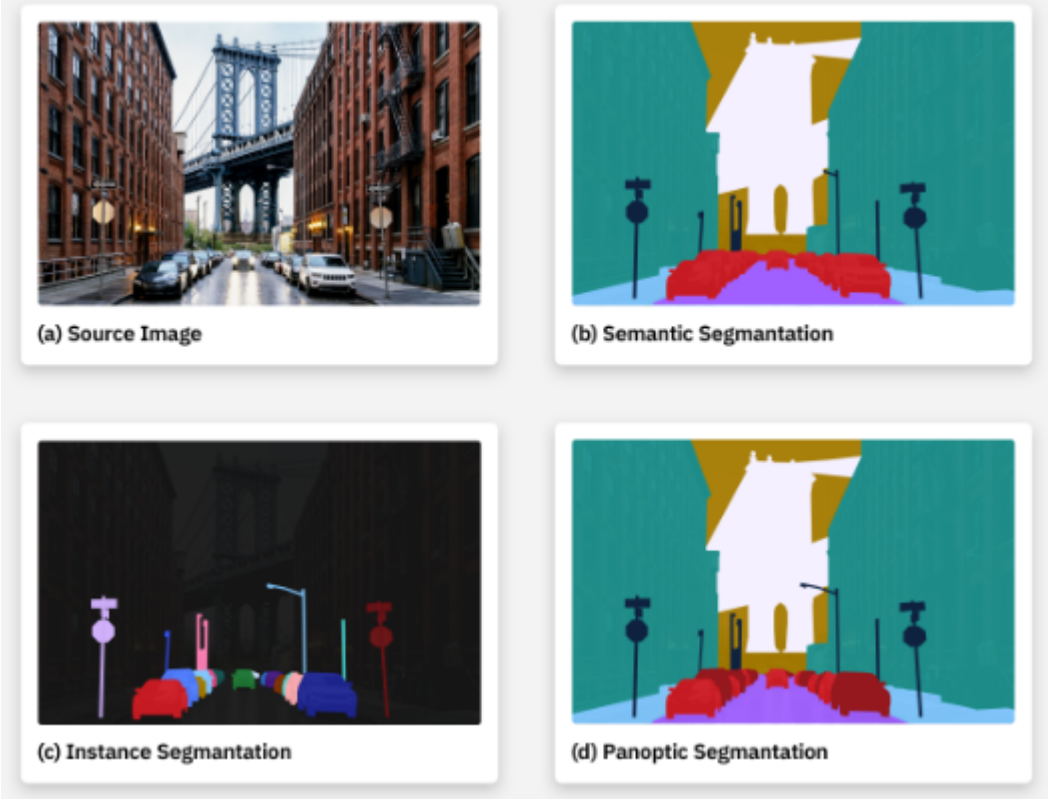
III. Panoptic Segmentation

Panoptic segmentation unifies the goals of both semantic and instance segmentation. It provides a comprehensive view of the scene by labeling each pixel with both a semantic class and, if applicable, an instance ID.

- Assigns: a semantic class to every pixel.
- Assigns an instance ID to each pixel that belongs to a thing (e.g., people, cars).
- Stuff (e.g., road, sky, grass) is labeled by class but not separated into instances.
- Provides a **comprehensive understanding** of the scene.

📌 *Example*: In a street scene, each person and car is individually segmented and labeled, while the sky and road are categorized as “stuff.”

⚠️ *Limitation*: Combines the computational demands of both semantic and instance segmentation. Overlapping or inconsistent predictions



---

4. Types of Semantic Classes

Type	Description	Examples
Things	Countable objects with defined boundaries and shapes	Car, person, tree
Stuff	Uncountable, amorphous regions without clear part-based structure	Sky, water, grass

Reminder: Panoptic segmentation isolates both "things" and "stuff" effectively—something neither semantic nor instance segmentation achieves alone.

Summary

Segmentation Type	Focus	Handles "Things"	Handles "Stuff"	Instance-aware	Use Case Example
Semantic	Classifying every pixel	✅	✅	❌	Aerial land classification
Instance	Separating object instances	✅	❌	✅	Counting parked cars
Panoptic	Unifying both types	✅	✅	✅	Urban street analysis

---

5. Applications of Image Segmentation

📌 Medical Imaging

- Tumor detection, organ delineation, MRI and CT interpretation
- Used in diagnostics and surgical planning
- Clinical diagnosis

📌 Biomedical Research

- Cell and tissue analysis
- Longitudinal and high-throughput studies

📌 Autonomous Vehicles

- Detect pedestrians, cars, lanes, and traffic signs
- Crucial for obstacle avoidance and navigation

📌 Satellite Imaging

- Segment landscapes: forests, urban areas, water bodies
- Land use and environmental monitoring

📌 Smart Cities

- Real-time traffic monitoring
- Surveillance and infrastructure planning

📌 Manufacturing

- Quality control, defect detection, product sorting

📌 Agriculture

- Crop health estimation
- Weed detection and yield forecasting

Aims

Based on those applications, this work aims to develop and evaluate deep learning-based **semantic image segmentation** models to identify and delineate neurons in fluorescent microscopy images accurately. Precise segmentation of neurons is crucial in neuroscience and biomedical research, as it enables quantification and spatial analysis of neuronal populations involved in various biological processes.

By leveraging the Fluorescent Neuronal Cells dataset, which provides high-resolution images and pixel-level ground-truth masks, we aim to address these challenges and contribute to developing robust, generalizable computational tools for neuron detection and counting.

B. Dataset Description

For this study, we used the Fluorescent Neuronal Cells dataset (https://www.kaggle.com/datasets/nbroad/fluorescent-neuronal-cells), which consists of 283 high-resolution (1600x1200 pixels) fluorescence microscopy images of mice brain slices. Each image highlights neurons labeled with a fluorescent marker, appearing as yellow-tinted regions of varying intensity against darker, composite backgrounds.

The dataset includes two main components:

- Original images stored in the all_images/images directory, capturing neuronal structures during a biological experiment.
- Corresponding binary ground-truth masks (in black-and-white) representing pixel-level annotations of stained neurons.

The dataset is well-suited for tasks such as semantic segmentation, object detection, and neuron counting. It provides a biologically relevant benchmark for evaluating segmentation models in a context where traditional computer vision models trained on natural images often underperform.

All data is shared under the Creative Commons Attribution Share Alike 4.0 International License, encouraging both academic and methodological exploration.

C. Methodology

1. Data Preparation

1. The raw images and masks are loaded and are cut into smaller **400x400 pixel**

For each mask:

- A flood-fill operation is used to identify connected regions with intensity values greater than **127**.
- From these regions, the **area, width, and height** of each cell are calculated.

2. The distribution of cell sizes is then analyzed.

- This helps choose a good **overlap value** for splitting the images.
- An overlap of **100 pixels** was selected, since it's larger than most cells.
- This ensures that even if a cell is near the edge of a sub-image, it will still appear fully in at least one sub-image.

---

2. Data Augmentation

Two types of data augmentation are used to improve model learning. These are gradually introduced during training:

1. Random Shapes

- Ellipses and rounded rectangles are drawn in random positions.
- They are similar in **color and size** to real cells.
- Purpose: helps the model recognize **cell shapes** instead of just reacting to yellow pixel clusters.

2. Affine Transformations

- Random changes applied to the image:
  - Rotation
  - X/Y Scaling
  - Horizontal/Vertical Flipping
  - Skewing

---

3. Training Process

Since brain cells make up only about nearly **1%** of all pixels, training is done in two phases to handle the class imbalance:

Phase 1 – Filtered Data (only images with cells)

- Use only images that contain brain cells.
- This raises the cell pixel percentage to around **2%**.
- Class weights are adjusted but slightly reduced to avoid over-predicting cells.

Phase 1.1

- **Train for 5 epochs** on the filtered dataset.
- **No augmentation** is applied.

Phase 1.2

- **Train for 10 more epochs** with the following augmentation:
  - Drawing of random ellipses and rounded rectangles.
- Purpose: to teach the model that **cell shape matters** for predictions.

Phase 2 – Full Dataset

- Now train on the **entire dataset**, including images with no brain cells.
- Use both augmentation techniques:
  - Random shapes
  - Affine transformations
- Only **5 epochs** are run due to slower training times.

4. Prediction Strategy

Since the model predicts on small sections of each image, we need to stitch the results back together:

1. **Split** the input image into **400x400 pixel** sections (overlap amount is flexible).
2. **Run** predictions on each section.
3. **Resize** each predicted mask back to 400x400 pixels.
4. **Merge** all predicted masks back into one full-size image:
    - For overlapping areas, take the **minimum pixel value** to avoid overestimating.

5. Evaluation of the models

To thoroughly assess the segmentation performance of our models, we employed a range of evaluation metrics and visual tools. These include ROC curves, confusion matrices, precision-recall curves, and Dice coefficients.

- **Dice Coefficient**

The Dice coefficient is a widely used metric for evaluating image segmentation tasks. It is particularly effective for class-imbalanced images, where traditional accuracy can be misleading. Unlike accuracy, which includes true negatives in its calculation, the Dice coefficient focuses on the overlap between the predicted mask and the ground truth mask (true positives), providing a more meaningful assessment of segmentation performance.

$$\text{Dice} = \frac{2 \cdot |A \cap B|}{|A| + |B|}$$

Where:

- (A) is the predicted binary mask
- (B) is the ground truth binary mask

Higher Dice scores indicate better performance.

- **ROC Curve**

The **Receiver Operating Characteristic (ROC)** curve plots the **true positive rate (TPR)** against the **false positive rate (FPR)** at various threshold settings.

We calculated pixel-level ROC curves by flattening both predicted and ground truth masks.

- **Confusion Matrix**

A confusion matrix summarizes pixel classification into: True Positives (TP), False Positives (FP), True Negatives (TN) and False Negatives (FN).

From this matrix, we derive: Accuracy, Precision, Recall and F1 Score.

- **Precision-Recall Curve**

The precision-recall (PR) curve is particularly useful in datasets with imbalanced classes, as it focuses on performance for the positive class.

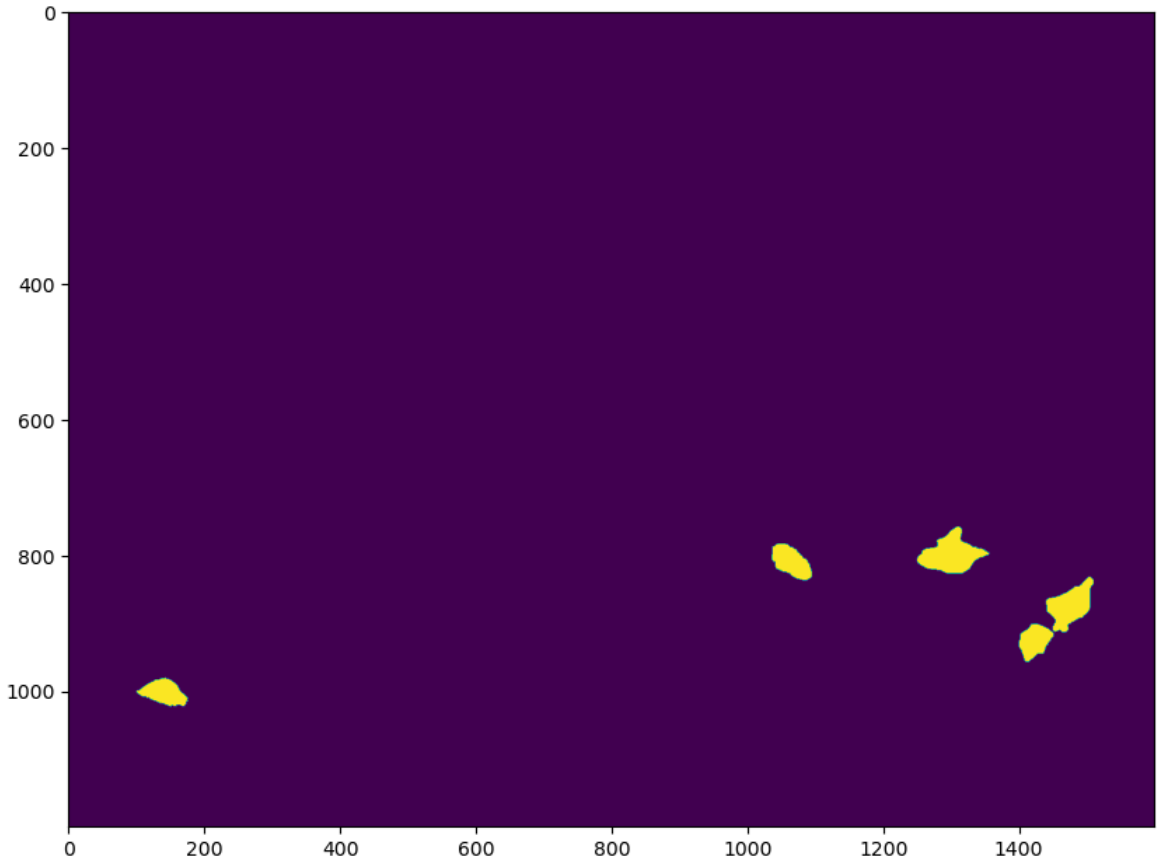
In [13]:
```
# Imports libraries and gathers all .png image and mask file paths from specifie

import numpy as np
import pandas as pd
from PIL import Image
import os

# empty lists to store full paths of image and mask files
inputPaths = []
maskPaths = []

# set the paths
images_path = r"C:\Users\gonca\Desktop\LiveCell\all_images"
masks_path = r"C:\Users\gonca\Desktop\LiveCell\all_masks"

# traverse the images directory and collect full paths of all .png image files
for dirname, _, filenames in os.walk(images_path):
    for filename in filenames:
        if filename.endswith(".png"):  # only png are considered
            fullPath = os.path.join(dirname, filename)  # construct full file pa
            inputPaths.append(fullPath)  # add the path to the list

# same thing but for masks
for dirname, _, filenames in os.walk(masks_path):
    for filename in filenames:
        if filename.endswith(".png"):
            fullPath = os.path.join(dirname, filename)
            maskPaths.append(fullPath)
```

In [15]:
```
# Loads and pairs each image with its corresponding mask, returning a dictionary

import multiprocessing as mp

# loads one image and its mask
def LoadImgProc(fileInfo):
    filename = fileInfo[0]
    imgPath, maskPath = fileInfo[1]
    img = Image.open(imgPath)
    mask = Image.open(maskPath).convert('L')  # open the mask and convert to gra
    return (filename, img, mask)

# loads all image and mask pairs
def loadImages(imgPathNames, maskPathNames):
    # dictionary of mask paths using the file name as the key
    maskFileDict = {os.path.basename(fullPath): fullPath for fullPath in maskPat

    # dictionary that matches each image path with its corresponding mask path
    imgMaskFilePairsDict = {
        os.path.basename(fullPath): (fullPath, maskFileDict[os.path.basename(ful
        for fullPath in imgPathNames
    }
```

```
    # load all image-mask pairs (sequentially for now, could be done in parallel
    imgMaskPairsList = [LoadImgProc(t) for t in imgMaskFilePairsDict.items()]

    # Return a dictionary: filename - (image, mask)
    return {filename: (img, mask) for filename, img, mask in imgMaskPairsList}

# Load all the image and mask data into a dictionary
imgMaskPairsRaw = loadImages(inputPaths, maskPaths)
```

In [17]:
```
# Extracts the loaded images and masks into separate lists and displays the firs

import matplotlib.pyplot as plt

imgListRaw = [t[0] for _, t in imgMaskPairsRaw.items()]

maskListRaw = [t[1] for _, t in imgMaskPairsRaw.items()]

# plot the first mask to check what it looks like
plt.figure(figsize=(10, 16))
plt.imshow(maskListRaw[0])
```

Out[17]: <matplotlib.image.AxesImage at 0x22e4fc45eb0>



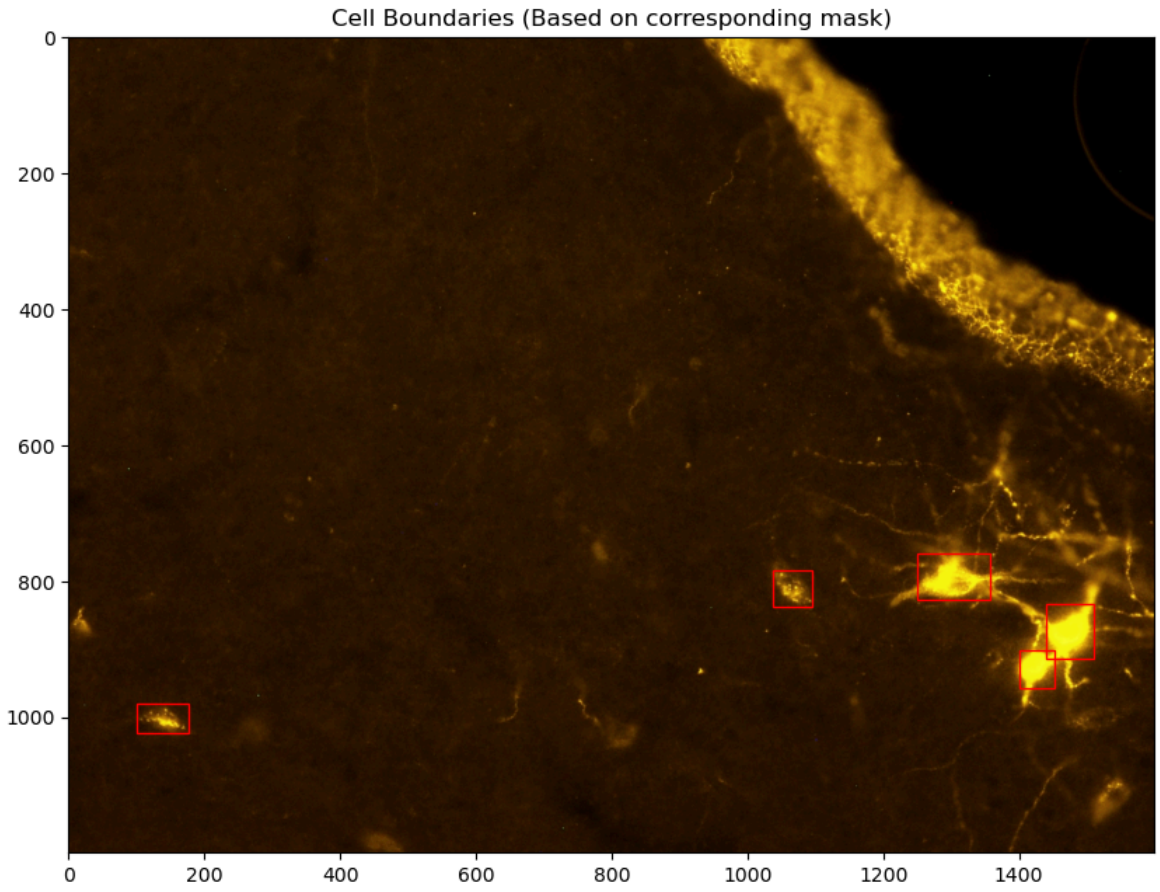
In [19]:
```
# Identifies and extracts all connected pixel groups (cells) from a binary mask

from collections import namedtuple
from collections import defaultdict

# cell info
CellLocation = namedtuple("CellLocation", ["nPixels", "x", "y", "width", "height

# perform BFS to mark all connected pixels as part of the same group
```

```
def bfs(r, c, nRows, nCols, traversable, groupAssignments, groupIndex):
    frontier = [(r,c)]
    groupAssignments[r][c] = groupIndex
    while len(frontier) > 0:
        curRow, curCol = frontier.pop() # current cell
        # check 4 neighboring cells (up, down, left, right)
        nextNodes = [
            (curRow + 1, curCol),
            (curRow - 1, curCol),
            (curRow, curCol + 1),
            (curRow, curCol - 1)
        ]
        for nextRow, nextCol in nextNodes:
            # check if the neighbor is inside the bounds
            withinRange = (
                nextRow >= 0 and nextRow < nRows and
                nextCol >= 0 and nextCol < nCols)
            unvisited = (
                withinRange and
                traversable[nextRow][nextCol] and
                groupAssignments[nextRow][nextCol] == None)
            if withinRange and unvisited:
                frontier.append((nextRow, nextCol))
                groupAssignments[nextRow][nextCol] = groupIndex
    return

# for a given binary mask, find all connected groups of pixels (cells)
def markGroups(maskArr):
    threshold = int(255 * 0.5)
    # booleans list: True = pixel is part of a cell
    isCell = np.greater(maskArr, threshold).tolist()
    nRows, nCols = maskArr.shape
    group = [[None]*nCols for _ in range(nRows)]
    groupIdx = 0
    # loop through each pixel
    for r in range(nRows):
        for c in range(nCols):
            unvisitedGroup = isCell[r][c] and (group[r][c] == None)
            if unvisitedGroup:
                bfs(r,c, nRows, nCols, isCell, group, groupIdx)
                groupIdx += 1 # move to the next group

    #Get size and bounding
    nGroups = groupIdx
    sizeList = [0] * nGroups

    rightList = [0] * nGroups
    leftList = [nCols] * nGroups

    botList = [0] * nGroups
    topList = [nRows] * nGroups

    for r in range(nRows):
        for c in range(nCols):
            if group[r][c] != None:
                groupNo = group[r][c]
                sizeList[groupNo] += 1

                rightList[groupNo] = max(rightList[groupNo], c)
                leftList[groupNo] = min(leftList[groupNo], c)
```

```
                botList[groupNo] = max(botList[groupNo], r)
                topList[groupNo] = min(topList[groupNo], r)

    # results
    cellLocList = []
    for groupNo in range(nGroups):
        size = sizeList[groupNo]
        x = leftList[groupNo]
        y = topList[groupNo]
        width = 1 + (rightList[groupNo] - x)
        height = 1 + (botList[groupNo] - y)
        cellLocList.append(CellLocation(size, x, y, width, height))

    return cellLocList
```

In [21]:

```
# raw image with red bounding boxes

import matplotlib.patches as patches

# cell locations from all masks
cellLocationList = [markGroups(np.array(mask)) for mask in maskListRaw]

imageIdx = 0 # image selected

plt.figure(figsize=(10, 16))
plt.title("Cell Boundaries (Based on corresponding mask)")
plt.imshow(imgListRaw[imageIdx])

ax = plt.gca()

# red rectangle for each detected cell
for cellLoc in cellLocationList[imageIdx]:
    rect = patches.Rectangle(
        (cellLoc.x, cellLoc.y), cellLoc.width, cellLoc.height,
        linewidth=1, edgecolor='r', facecolor='none'
    )
    ax.add_patch(rect)

plt.show()

# visually confirmation the cell detection and localization results by overlayin
```



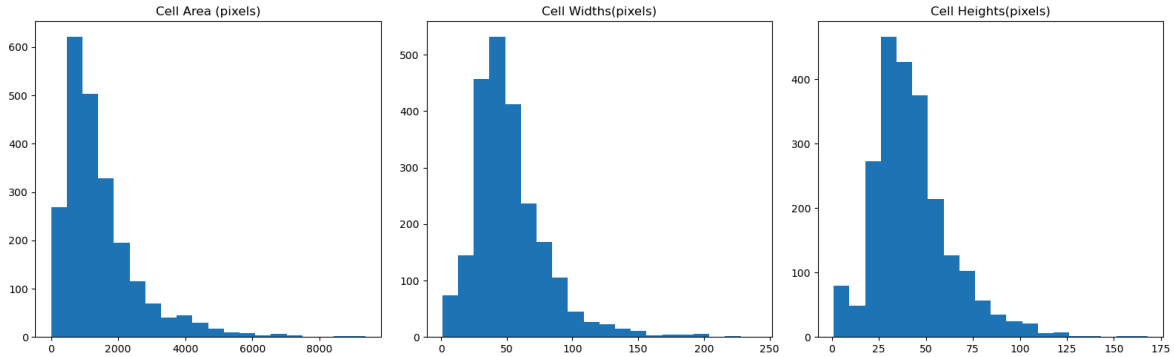
In [23]:
```
# merges the cell location results from all masks into one single list

# list of cell locations from all images into a single list
flattenedCellLocations = []

# loop through each image's list of CellLocation objects
for li in cellLocationList:
    flattenedCellLocations.extend(li)
```

In [25]:
```
# count the total number of detected cells across all images
nTotalCells = len(flattenedCellLocations)
print(nTotalCells)

# properties of each cell
sizes = [cellLoc.nPixels for cellLoc in flattenedCellLocations]
widths = [cellLoc.width for cellLoc in flattenedCellLocations]
heights = [cellLoc.height for cellLoc in flattenedCellLocations]
```

```
2270
```

In [27]:
```
# side-by-side histograms for cell sizes, widths, and heights
fig, ax = plt.subplots(1, 3, figsize=(16,5))

nBins = 20
ax[0].hist(sizes,bins=nBins)
ax[0].set_title('Cell Area (pixels)')

ax[1].hist(widths,bins=nBins)
ax[1].set_title('Cell Widths(pixels)')

ax[2].hist(heights,bins=nBins)
ax[2].set_title('Cell Heights(pixels)')
```

```
plt.tight_layout()
plt.show()

percentile = 95

sizePercentile = np.percentile(sizes, percentile)
widthPercentile = np.percentile(widths, percentile)
heightPercentile = np.percentile(heights, percentile)

print("{0}th Percentile Values for:".format(percentile))
print(" Area:{:0.2f} pixels".format(sizePercentile))
print(" Width:{:0.2f} pixels".format(widthPercentile))
print(" Height:{:0.2f} pixels".format(heightPercentile))
```




```
95th Percentile Values for:
 Area:3872.65 pixels
 Width:103.00 pixels
 Height:82.55 pixels
```

In [29]:
```
# total number of pixels marked as cells across all masks
totalNoOfMarkedPixels = sum(sizes)

# total number of pixels in all masks
overallNoOfPixels = sum((mask.size[0] * mask.size[1] for mask in maskListRaw))

# remaining pixels that are not marked as cells
totalNoOfUnmarkedPixels = overallNoOfPixels - totalNoOfMarkedPixels

# inverse freq.
markedInv = overallNoOfPixels / totalNoOfMarkedPixels
unmarkedInv = overallNoOfPixels / totalNoOfUnmarkedPixels

# normalization of the weights
normalizationCoeff = 1 / (markedInv + unmarkedInv)
classWeights = {
    0: unmarkedInv * normalizationCoeff, #unmarked pixels
    1: markedInv * normalizationCoeff    #marked pixels
}

print("Percentage of pixels which are neural cells: {0}%".format((totalNoOfMarke
print(classWeights)
```

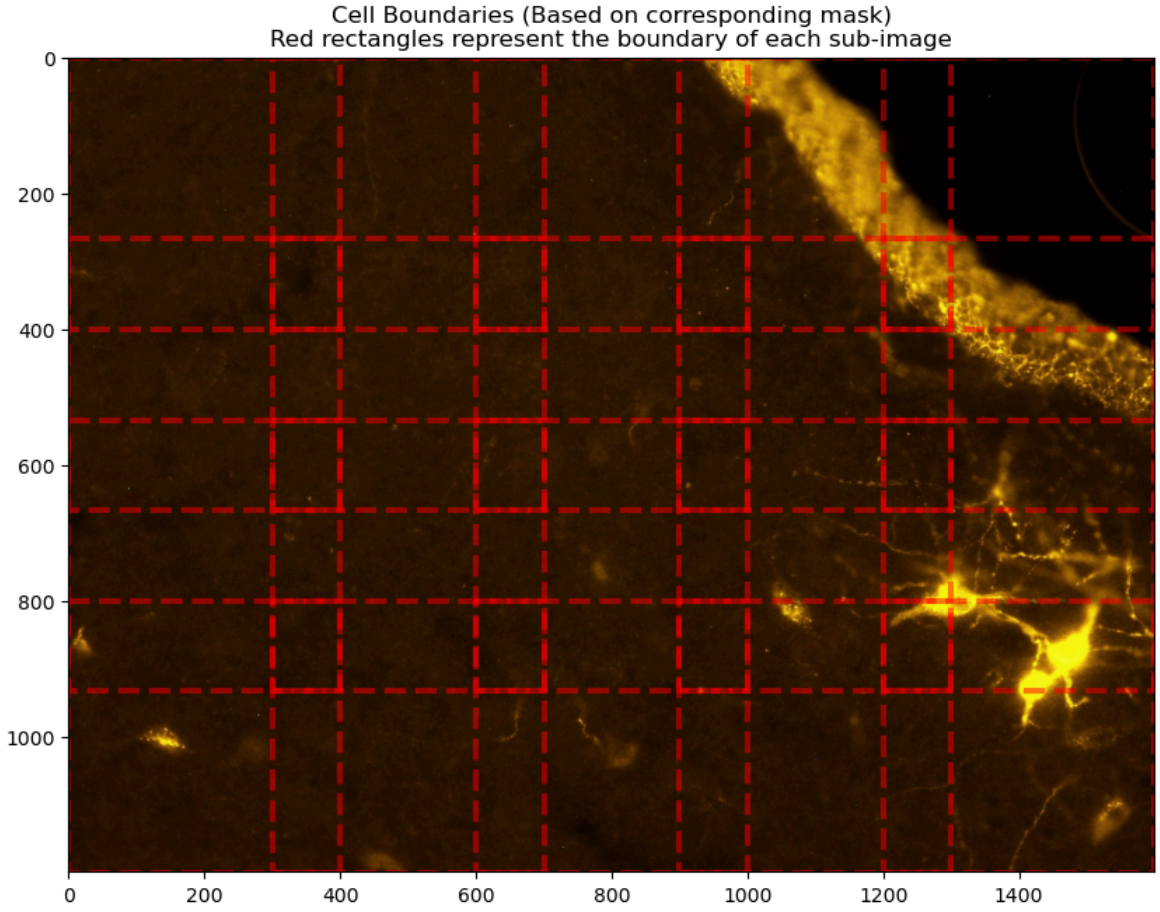
```
Percentage of pixels which are neural cells: 0.6066018845700825%
{0: 0.006066018845700824, 1: 0.9939339811542991}
```

Subdivision of source images

Each original image of 1600x1200 pixels is split into smaller images of 400x400 pixels. This creates 25 smaller images from each big one. We do this splitting ahead of time to make the data processing faster and to make it easier to separate the training and testing data.

In [32]:
```
'''
This code divides the selected image into overlapping square sub-regions (partit
ensuring full coverage with specified minimum overlap,
and visualizes these sub-image boundaries over the original image.
'''

import math

# rectangle structure with x, y, width, and height
Rect = namedtuple("Rect", ["x", "y", "width", "height"])

def EvenlySpacedPartitions(width, height, partitionLength, minOverlap):
    # minimum number of partitions needed to cover a dimension
    def minPartitions(dimLength):
        return math.ceil((dimLength - minOverlap) / (partitionLength - minOverla

    # number of rows and columns for partitions
    nRows = minPartitions(height)
    nCols = minPartitions(width)

    # ensuring partitions overlap by at least minOverlap
    def getPartitionStartingPositions(dimLength, nPartitions):
        overlap = (dimLength - nPartitions * partitionLength) / (1- nPartitions)
        advance = partitionLength - overlap
        return [advance * i for i in range(nPartitions)]

    # starting positions for rows and columns
    rowPositions = getPartitionStartingPositions(height, nRows)
    colPositions = getPartitionStartingPositions(width, nCols)
    partitions = []

     # rectangles for each sub-image partition based on starting positions
    for colStart in colPositions: #x
        for rowStart in rowPositions: #y
            partitions.append(Rect(colStart, rowStart, partitionLength, partitio
    return partitions

imageIdx = 0;

plt.figure(figsize=(10,16))

plt.title("Cell Boundaries (Based on corresponding mask)\nRed rectangles represe
plt.imshow(imgListRaw[imageIdx])

# image dimensions
width, height  = imgListRaw[imageIdx].size

# generate overlapping partitions of size 400x400 with minimum 100 pixels overla
partitions = EvenlySpacedPartitions(width, height, 400, 100)

ax = plt.gca();

# rectangles showing each sub-image partition on the image
```

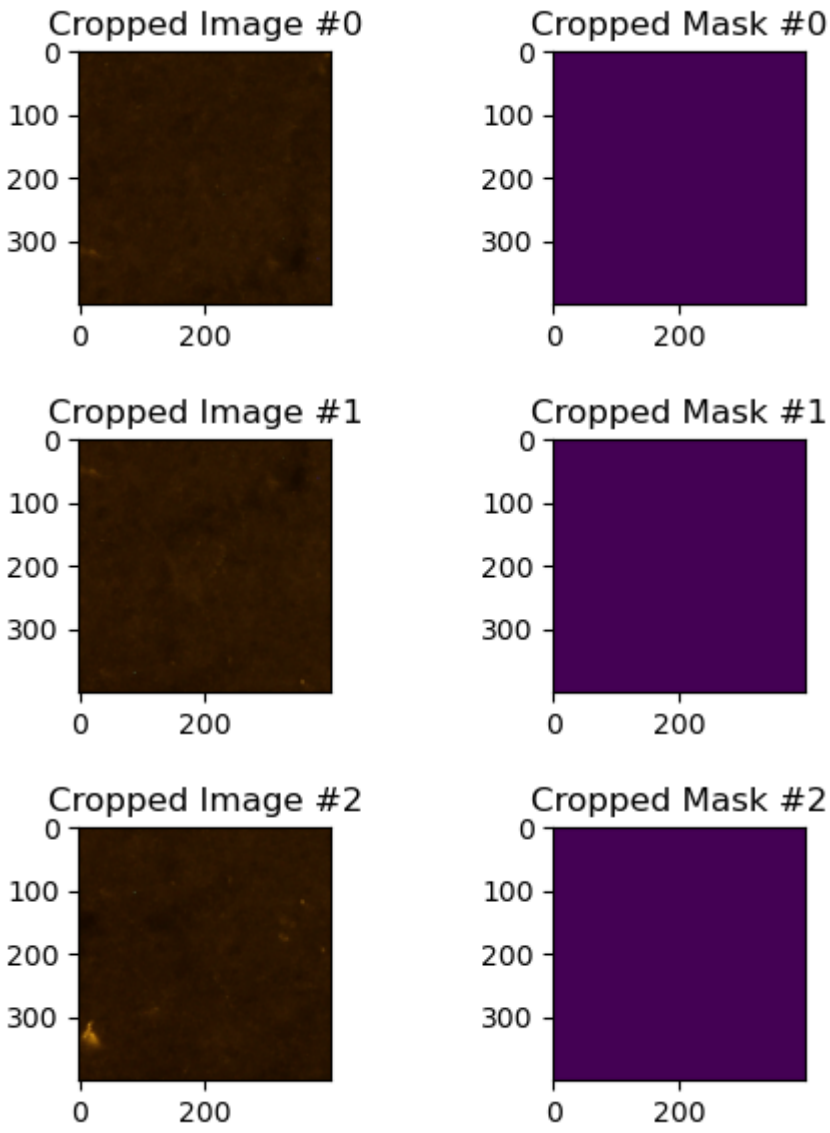
```
for partition in partitions:
    rect = patches.Rectangle((partition.x, partition.y), partition.width, partit
    ax.add_patch(rect)

plt.show()
```

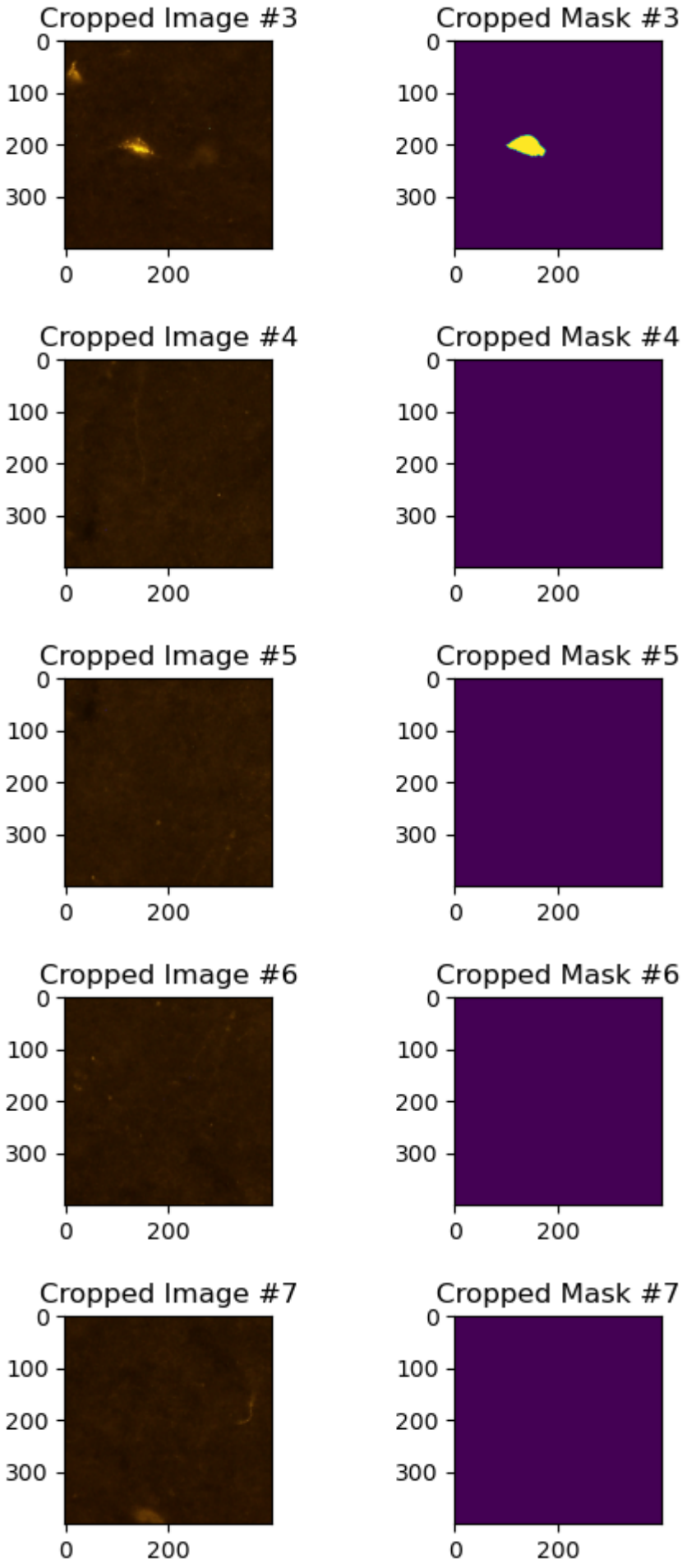


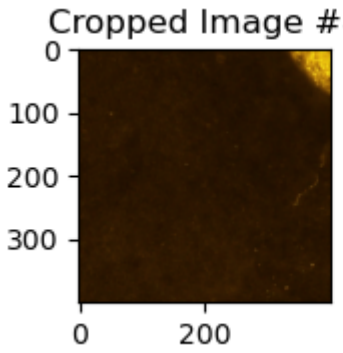
In [34]:
```
# divide large images and their corresponding masks into smaller, overlapping pa

# define a structure to hold a cropped image, its corresponding mask, and the so
CroppedImage = namedtuple("CroppedImage", ["image", "mask", "sourceRect"])

def CropImage(pilImage, pilMask, partitions):
    croppedImages = []
    for partition in partitions:
        # crop box coordinates (left, upper, right, lower)
        top, bot = (partition.y, partition.y + partition.height)
        left, right = (partition.x, partition.x + partition.width)

        # crop the image and the mask using the bounding box
        croppedImg = pilImage.crop((left, top, right, bot))
        croppedMask = pilMask.crop((left, top, right, bot))
        croppedImages.append(CroppedImage(croppedImg, croppedMask, partition))
    return croppedImages

def GetAllPartitions(pilImageList, partitionSize, minOverlap):
    partitionList = []
    for img in pilImageList:
        width, height = img.size
        # overlapping partitions for the current image
        partitions = EvenlySpacedPartitions(width, height, partitionSize, minOve
        partitionList.append(partitions)
    return partitionList
```

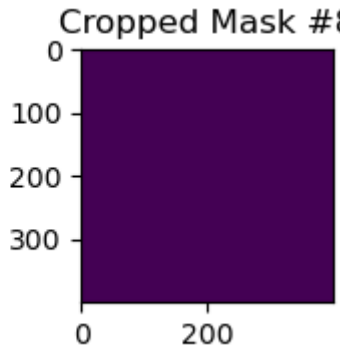
```
# partition rectangles for all images with 400x400 patches and at least 100 pixe
partitionList = GetAllPartitions(imgListRaw, 400, 100)
```

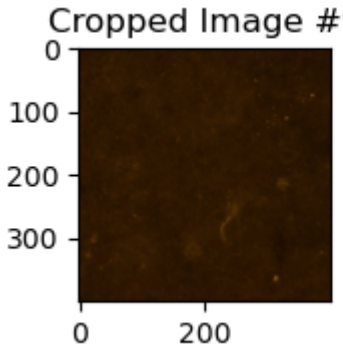
In [36]:
```
# crop the first image and its mask
sampleCrop = CropImage(imgListRaw[0], maskListRaw[0], partitionList[0])

# loop through each cropped image-mask pair and display them side-by-side
for idx, croppedImagePair in enumerate(sampleCrop):
    fig, ax = plt.subplots(1, 2, figsize=(5,2))

    ax[0].imshow(croppedImagePair.image)
    ax[0].set_title('Cropped Image #{0}'.format(idx))

    ax[1].imshow(croppedImagePair.mask)
    ax[1].set_title('Cropped Mask #{0}'.format(idx))

    plt.tight_layout()
    plt.show()
```

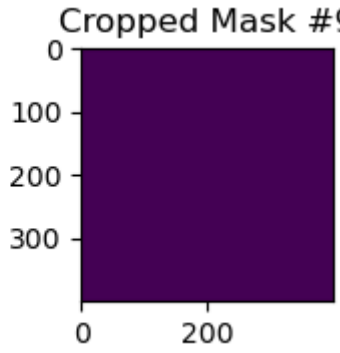


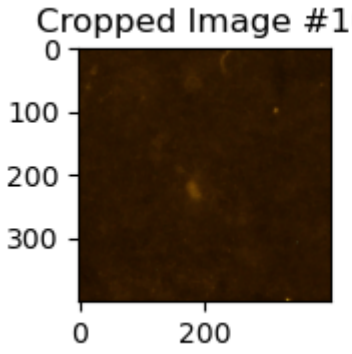

Cropped Image #3
Cropped Mask #3
Cropped Image #4
Cropped Mask #4
Cropped Image #5
Cropped Mask #5
Cropped Image #6
Cropped Mask #6
Cropped Image #7
Cropped Mask #7
0
100
200
300
0
200

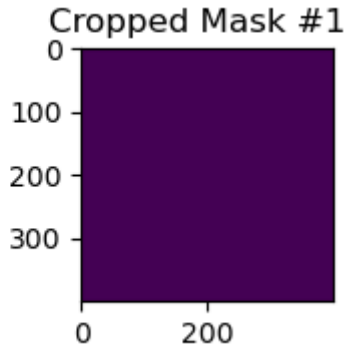

Cropped Image #8
0
100
200
300
0
200

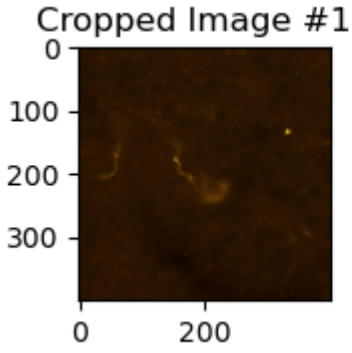

Cropped Mask #8
0
100
200
300
0
200

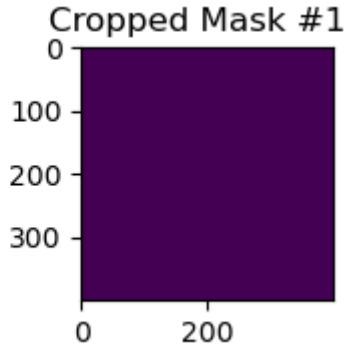

Cropped Image #9
0
100
200
300
0
200

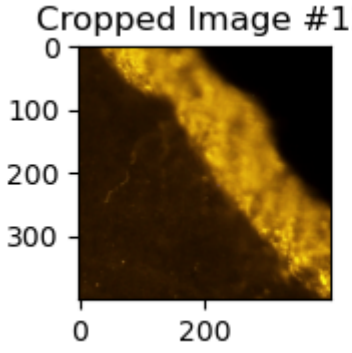

Cropped Mask #9
0
100
200
300
0
200

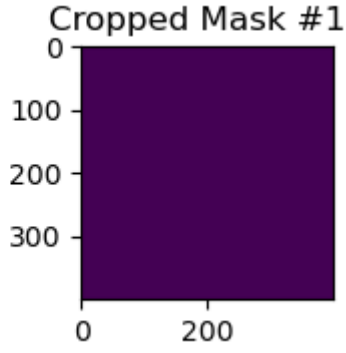

Cropped Image #10
0
100
200
300
0
200

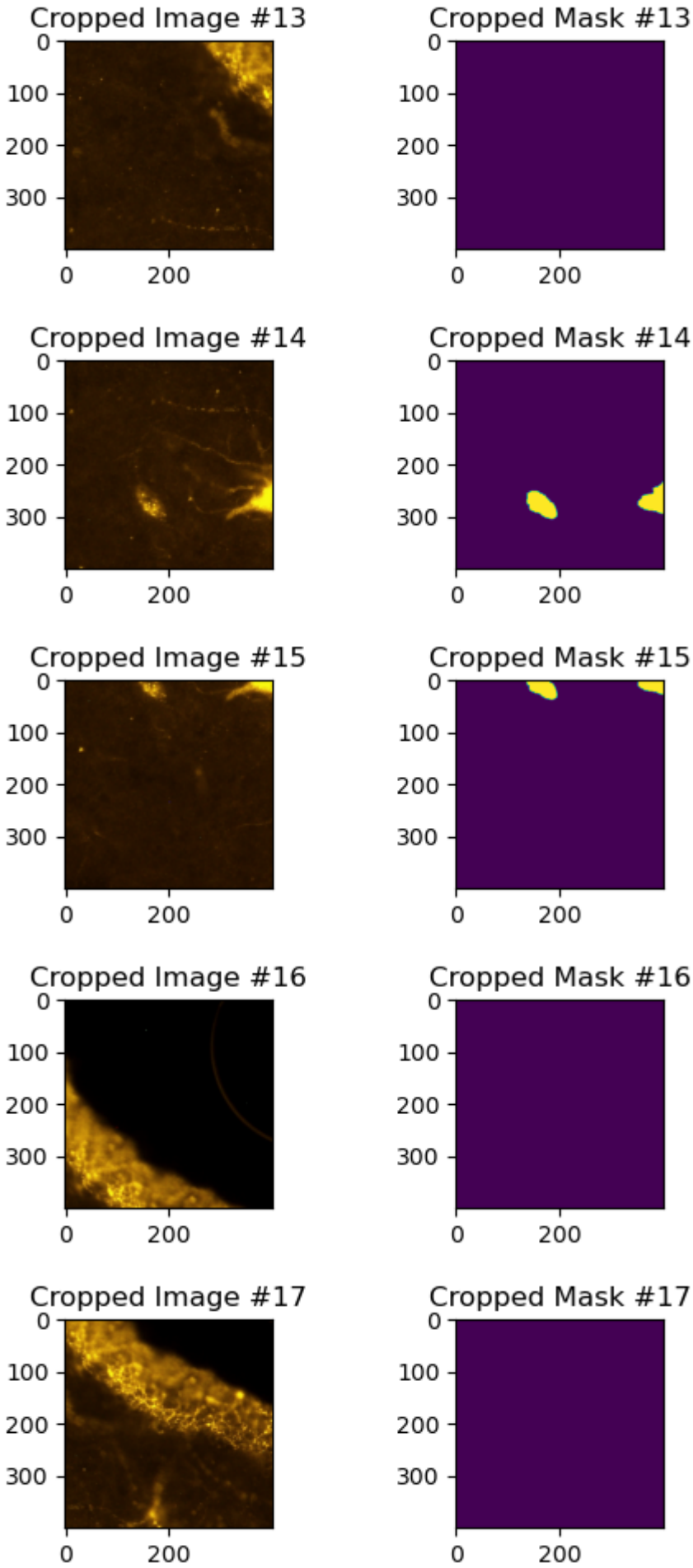

Cropped Mask #10
0
100
200
300
0
200

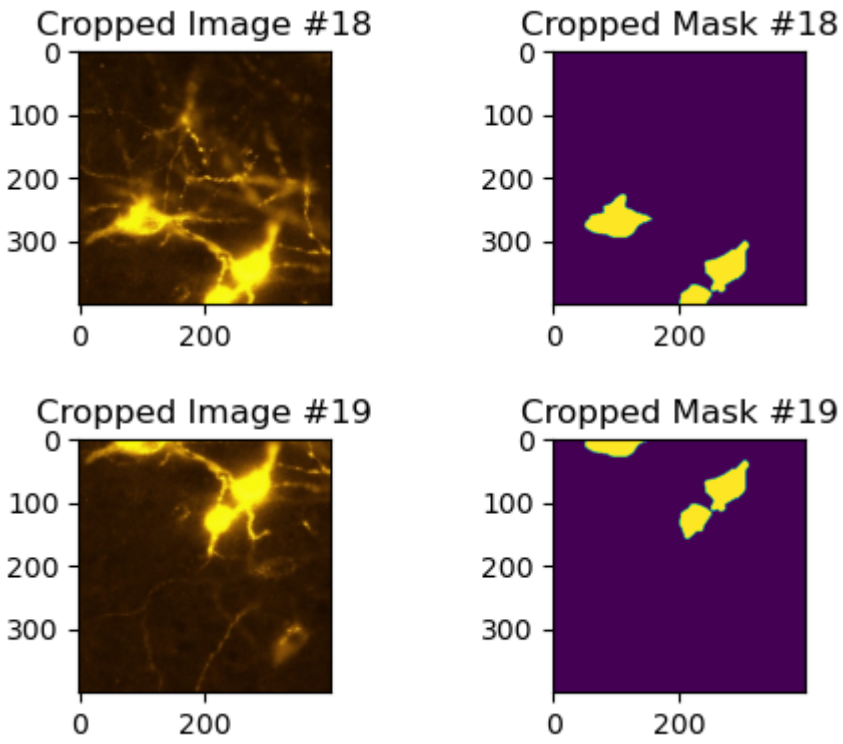

Cropped Image #11
0
100
200
300
0
200


Cropped Mask #11
0
100
200
300
0
200


Cropped Image #12
0
100
200
300
0
200


Cropped Mask #12
0
100
200
300
0
200


Cropped Image #13
Cropped Mask #13
Cropped Image #14
Cropped Mask #14
Cropped Image #15
Cropped Mask #15
Cropped Image #16
Cropped Mask #16
Cropped Image #17
Cropped Mask #17
0
100
200
300
0
200



Data Augmentation: Random Shapes

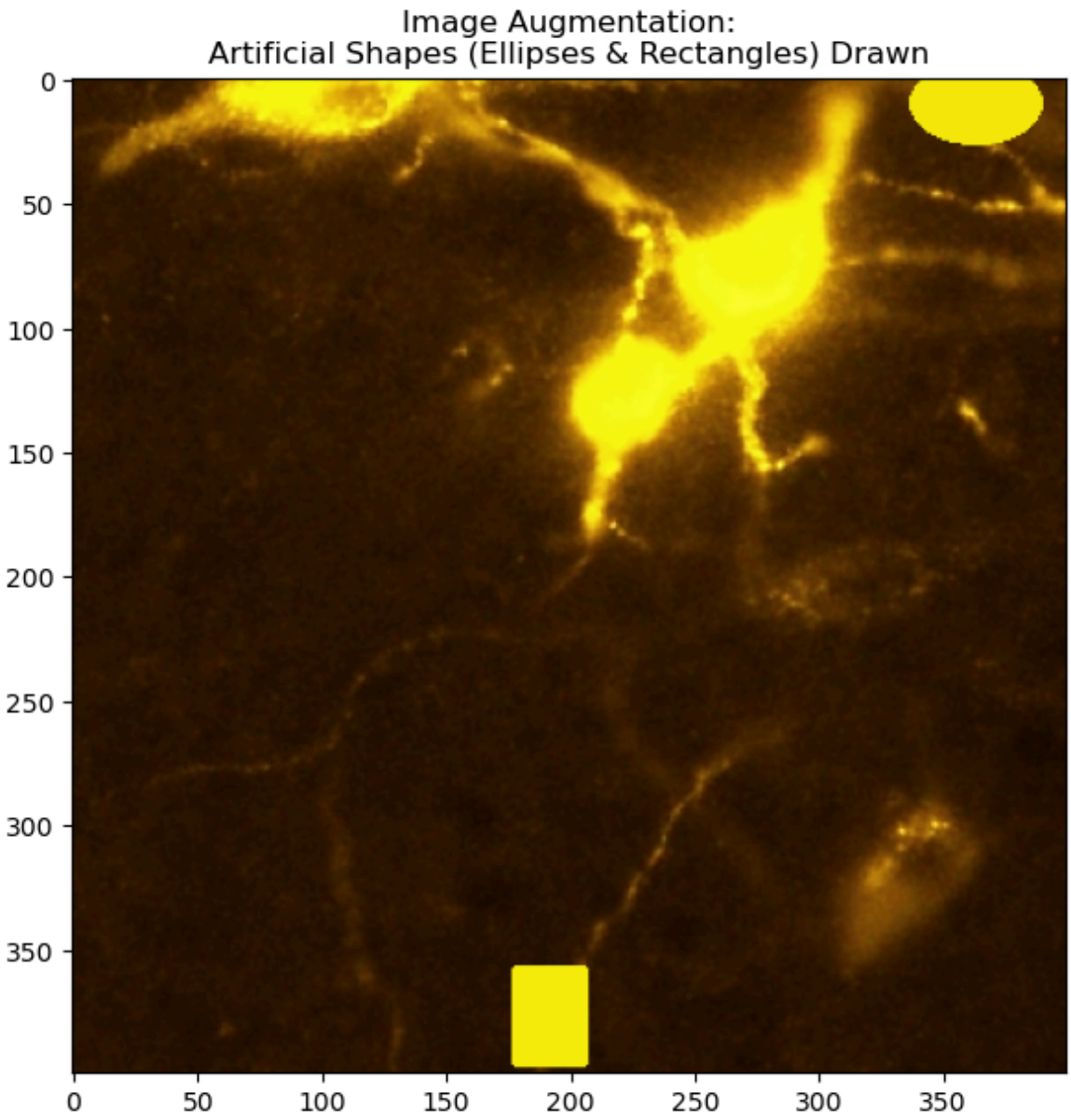
Using the sizes of cells found earlier, ellipses and rounded rectangles (with colors like the marked neurons) are randomly drawn on each smaller image without changing the mask. These added shapes help the neural network understand that shape is an important feature for making predictions.

Notes: A more precise way to create similar colors would be to analyze the colors of the marked neurons using a statistical method such as Gaussian Mixture. But instead, we chose to create RGB colors by carefully guessing two colors from the first image. To focus more on shape rather than color, the neural network could also be trained using black and white images.

In [39]:
```
from PIL import ImageDraw
from enum import Enum
import random

# define a simple RGB color structure
TColour = namedtuple("TColour", ["r", "g", "b"])

# sample colors observed in actual cell regions
darkYellow = TColour(101, 61, 6)
brightYellow = TColour(247, 237, 15)

# to interpolate between two RGB colors
def lerpColours(colour1, colour2, frac):
    r = colour1.r * (1 - frac) + colour2.r * frac
    g = colour1.g * (1 - frac) + colour2.g * frac
    b = colour1.b * (1 - frac) + colour2.b * frac
    return TColour(int(r),int(g),int(b))
```

```
# shape types: Ellipse or Rounded Rectangle
class ENShape(Enum):
    Ellipse = 0
    Rect = 1

# draw a randomly-sized and randomly-positioned shape (ellipse or rounded rect)
def DrawRandomEllipse(img, shape = ENShape.Ellipse):
    imgWidth, imgHeight = img.size

    # random center position within image bounds
    centreX, centreY = (random.uniform(0, imgWidth), random.uniform(0, imgHeight

     # random size, inspired by the distribution of real cell sizes
    shapeWidth, shapeHeight = (random.uniform(25, 75), random.uniform(25, 75))

    # bounding box from center and size
    boundingBox = (
        int(centreX - shapeWidth * 0.5),
        int(centreY - shapeHeight * 0.5),
        int(centreX + shapeWidth * 0.5),
        int(centreY + shapeHeight * 0.5)
    )

    fillColour = lerpColours(darkYellow, brightYellow, random.uniform(0, 1))

    # draw the selected shape on the image
    gc = ImageDraw.Draw(img)
    if shape == ENShape.Ellipse:
        gc.ellipse(boundingBox, fill=fillColour, outline=None, width=1)
    elif shape == ENShape.Rect:
        rad = int(random.uniform(1, min(shapeWidth, shapeHeight) * 0.3))
        gc.rounded_rectangle(boundingBox, radius=rad, fill=fillColour, outline=N

# last cropped image and copy it for augmentation
sampleImage = sampleCrop[-1].image.copy()

# synthetic ellipse and a synthetic rounded rectangle on it
DrawRandomEllipse(sampleImage, ENShape.Ellipse)
DrawRandomEllipse(sampleImage, ENShape.Rect)

# result with the drawn synthetic shapes
plt.figure(figsize=(7,7))
plt.title("Image Augmentation:\nArtificial Shapes (Ellipses & Rectangles) Drawn"
plt.imshow(sampleImage)
```

Out[39]: <matplotlib.image.AxesImage at 0x22e54672c00>



Data Augmentation: Affine Transformations

Affine transformations are a useful way to make basic changes to an image, such as:

- Moving the image up, down, left, or right (translation)
- Stretching or shrinking the image along the X or Y axis (scaling)
- Rotating the image around any point (by combining rotation with translation)
- Slanting the image to make it look like a trapezoid (skewing)
- Flipping the image horizontally or vertically (done by using negative scaling)

All these changes can be combined into a single 3x3 matrix. To apply multiple transformations, we multiply matrices together, one for each operation (like rotation or scaling). The order of multiplication matters and may need to be reversed depending on how the matrices are set up (row-major vs. column-major).

These transformations are applied to both the image and its mask so they stay matched.

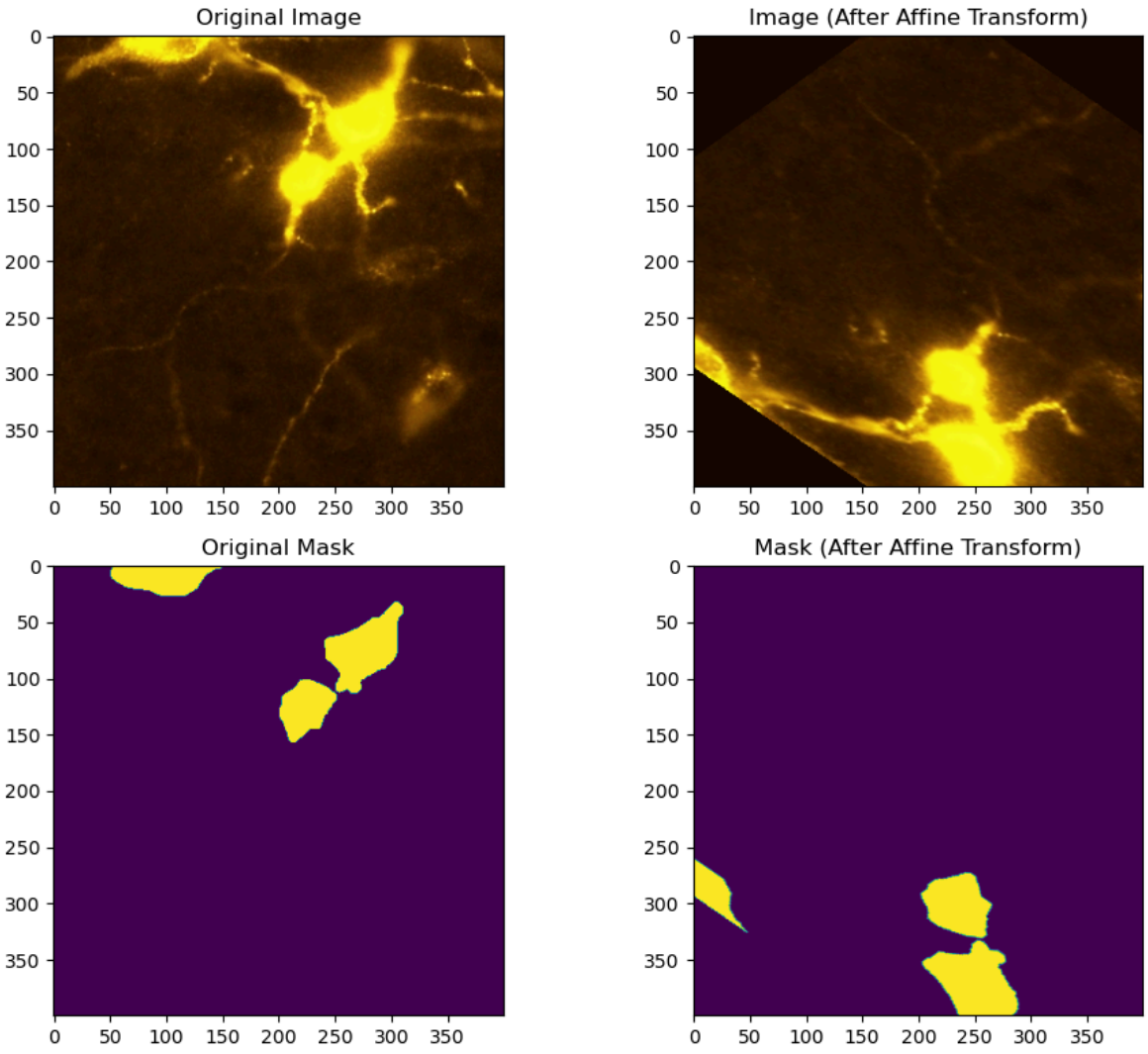
Note: TensorFlow's method is easier to use but has some limitations. For example, it always rotates around the center and doesn't allow you to change the pivot point. It also only fills empty areas left by the transformation with a single solid color. Lastly, if the data pipeline involves converting images from PIL to NumPy arrays, TensorFlow doesn't give a speed advantage, since both still run on the CPU.

In [42]:
```
# translation by x, y
def TranslateXForm(x,y):
    return np.array(
        [[1,0,x],
         [0,1,y],
         [0,0,1]]
    );

# rotation by degrees
def RotateXForm(deg):
    rad = math.radians(deg)
    return np.array(
        [[math.cos(rad),-math.sin(rad),0],
         [math.sin(rad), math.cos(rad),0],
         [0,0,1]]
    );

# shearing along x and y axes
def ShearXForm(cx, cy):
    return np.array(
        [[1,cx,0],
         [cy,1,0],
         [0,0,1]]
    );

# scaling along x and y axes
def ScaleXForm(xScale, yScale):
    return np.array(
        [[xScale,0,0],
         [0,yScale,0],
         [0,0,1]]
    );

# random combination of affine transformations
def GetRandomImageXForm(imgWidth, imgHeight):
    xform = np.identity(3)

    # translate origin to the center of the image
    xform = np.matmul(TranslateXForm(imgWidth * -0.5, imgHeight * -0.5), xform)

    # apply random shear
    xform = np.matmul(ShearXForm(random.uniform(0, 0.20), random.uniform(0, 0.20

    # apply random rotation
    xform = np.matmul(RotateXForm(random.uniform(0, 360)), xform)

    # apply random scaling
    xform = np.matmul(xform, ScaleXForm(random.uniform(0.8, 1.5), random.uniform

   # apply random horizontal and vertical flipping
    yFlipCoeff = 1 if bool(random.getrandbits(1)) else -1
    xFlipCoeff = 1 if bool(random.getrandbits(1)) else -1
```

```
    xform = np.matmul(ScaleXForm(xFlipCoeff, yFlipCoeff), xform)

    xform = np.matmul(TranslateXForm(imgWidth * 0.5, imgHeight * 0.5), xform) #S
    return xform

# colors for augmented images
imgBgColour1 = TColour(20, 8, 0)
imgBgColour2 = TColour(49, 25, 1)

# using PIL
def ApplyAffineXForm(image, xform, bgColour):
    invXForm = np.linalg.inv(xform)
    unpackedXForm = (*invXForm[0],*invXForm[1])  # flatten the first two rows

    xformedImg = image.transform(
        image.size,
        Image.AFFINE,
        data=unpackedXForm,
        resample=Image.BILINEAR,
        fillcolor=bgColour)
    return xformedImg
```

In [44]:

```
# sample image and its corresponding mask from the cropped dataset
sampleImage = sampleCrop[-1].image
sampleMask = sampleCrop[-1].mask

# random affine transformation matrix based on image size
augmentingXForm = GetRandomImageXForm(*sampleImage.size)

# random background color
bgColour = lerpColours(imgBgColour1, imgBgColour2, random.uniform(0, 1))

xformedImg = ApplyAffineXForm(sampleImage,augmentingXForm, bgColour)
xformedMask = ApplyAffineXForm(sampleMask,augmentingXForm, 0)

# display the original and transformed image-mask pairs
fig, ax = plt.subplots(2, 2, figsize=(10,8))

ax[0,0].imshow(sampleImage)
ax[0,0].set_title('Original Image')

ax[0,1].imshow(xformedImg)
ax[0,1].set_title('Image (After Affine Transform)')

ax[1,0].imshow(sampleMask)
ax[1,0].set_title('Original Mask')

ax[1,1].imshow(xformedMask)
ax[1,1].set_title('Mask (After Affine Transform)')

plt.tight_layout()
plt.show()
```



In [46]:
```
# crops all images and their masks into smaller pieces according to the precompu

partitionedImagesSet = [CropImage(img, mask, partitionInfo) for img, mask, parti
```

In [48]:
```
flattenedImages = []
flattenedMasks = []
for imageMaskPairList in partitionedImagesSet:
    for imageMaskPair in imageMaskPairList:
        flattenedImages.append(imageMaskPair.image)
        flattenedMasks.append(imageMaskPair.mask)
```

In [50]:
```
print(len(flattenedImages))
```

```
5660
```

Data Pipeline

A sequence-based data generator is used here because the augmentation methods can create many different versions of each image. The training data is shuffled after every epoch.

In [53]:
```
'''
custom data generator that enables augmented images for training. When enabled,
copied and augmentation techniques are applied. After the training concludes, th
```

```
'''

from tensorflow.keras.utils import Sequence
from tensorflow.keras.preprocessing.image import apply_affine_transform

subImgSize = 400
finalMaskSize = 50


# draws a random number of ellipses and rectangles directly on the input image.
def drawRandomShape(pilImage):
    nEllipses = random.randint(0, 2)
    for _ in range(nEllipses):
        DrawRandomEllipse(pilImage, ENShape.Ellipse)
    nRects = random.randint(0, 2)
    for _ in range(nRects):
        DrawRandomEllipse(pilImage, ENShape.Rect)
    return pilImage

class ImageSequence(Sequence):

    def __init__(self, x_set, y_set, batch_size, mask_final_dim):
        self.x, self.y = x_set, y_set
        self.batch_size = batch_size
        self.shuffleIndices()
        self.mask_dim = (mask_final_dim, mask_final_dim) #Used to downscale the
        self.bDrawShapes = False
        self.bAffineTransform = False

     # randomize sample order for each epoch
    def shuffleIndices(self):
        nSamples = len(self.x)
        indices = np.arange(nSamples)
        np.random.shuffle(indices)
        self.indices = indices

    def on_epoch_end(self):
        self.shuffleIndices()

    def enableFeatures(self, bDrawShapes, bAffineTransform):
        self.bDrawShapes = bDrawShapes
        self.bAffineTransform = bAffineTransform

    # returns total number of batches per epoch
    def __len__(self):
        return math.floor(len(self.x) / self.batch_size)

    def __getitem__(self, idx):
        startingIdx = idx * self.batch_size
        endingIdx = (idx + 1) * self.batch_size
        selectedIndices = self.indices[startingIdx:endingIdx]

        splitImages = [self.x[idx] for idx in selectedIndices]
        splitMasks = [self.y[idx] for idx in selectedIndices]

        imgWithRandomShapes = [
            drawRandomShape(img.copy()) if self.bDrawShapes else img
            for img in splitImages]

         # downscale masks to target size
```

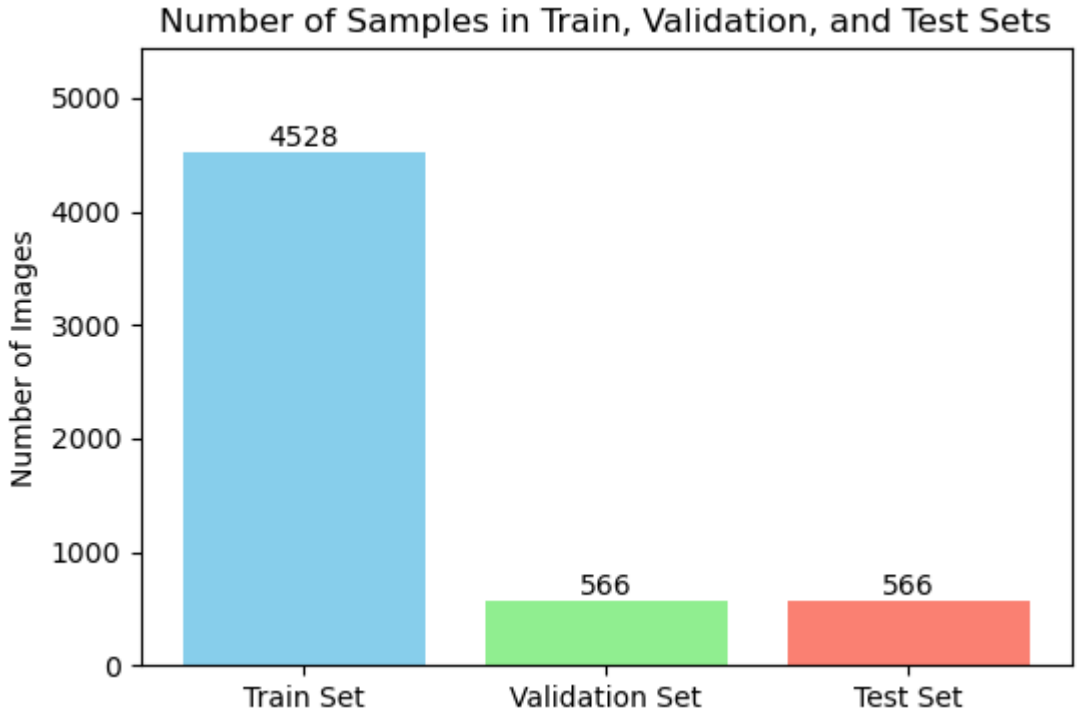
```
        masksDownscaled = [tMask.resize(self.mask_dim, Image.NEAREST ) for tMask

        # convert images and masks to NumPy arrays
        x_np = [np.array(img) for img in imgWithRandomShapes]
        y_np = [np.array(img) for img in masksDownscaled]

        x_augmented = []
        y_augmented = []

        for x, y in zip(x_np, y_np):
            rotation = random.uniform(0, 360)
            shear = random.uniform(-10, 10)
            yFlipCoeff = 1 if bool(random.getrandbits(1)) else -1
            xFlipCoeff = 1 if bool(random.getrandbits(1)) else -1
            xScale = random.uniform(0.8, 1.5) * xFlipCoeff
            yScale = random.uniform(0.8, 1.5) * yFlipCoeff

            # random grayscale background fill for augmented image
            bgColor = random.uniform(0,50)

            x_aug = None
            if self.bAffineTransform:
                x_aug = apply_affine_transform(x, theta=rotation, shear=shear, z
            else:
                x_aug = x

            original_y_shape = y.shape
            y_reshaped = np.reshape(y, (*original_y_shape, 1))
            y_aug = None
            if self.bAffineTransform:
                y_aug = apply_affine_transform(y_reshaped, theta=rotation, shear
            else:
                y_aug = y_reshaped

            y_aug_reshaped = np.reshape(y_aug, original_y_shape)
            x_augmented.append(x_aug)
            y_augmented.append(y_aug_reshaped)

        return (np.array(x_augmented), np.array(y_augmented) / 255)
```

In [55]:
```
# data split into training, validation, test

from sklearn.model_selection import train_test_split

# split off 80% train, 20% temp
img_train, img_temp, mask_train, mask_temp = train_test_split(flattenedImages, f

# split temp 50/50 to get 10% val and 10% test
img_val, img_test, mask_val, mask_test = train_test_split(img_temp, mask_temp, t

print(f"Train: {len(img_train)} images")
print(f"Validation: {len(img_val)} images")
print(f"Test: {len(img_test)} images")
```

```
Train: 4528 images
Validation: 566 images
Test: 566 images
```

In [57]:
```
# visual of split
```

```
# counts of training, validation, and testing samples
counts = [len(img_train), len(img_val), len(img_test)]
labels = ['Train Set', 'Validation Set', 'Test Set']

plt.figure(figsize=(6,4))
bars = plt.bar(labels, counts, color=['skyblue', 'lightgreen', 'salmon'])
plt.title('Number of Samples in Train, Validation, and Test Sets')
plt.ylabel('Number of Images')

for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height + 1, str(height), ha='cente

plt.ylim(0, max(counts)*1.2)
plt.show()
```



Training Set Filtering

To reduce the problem of class imbalance, the training set is first filtered to keep only the smaller images that actually contain neurons. This raises the percentage of neuron pixels from 0.6% in the full dataset to 2% in the filtered set. The filtering is done by checking if the mask for each image has at least one pixel that is not zero.

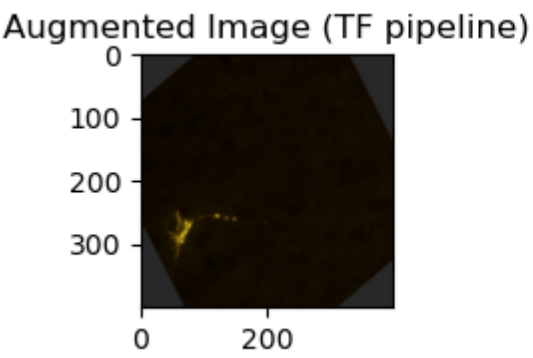
In [60]:
```
# filter out those within the training set that actually have cells, most images

trainSetWithCells = [(img, mask) for img, mask in
                     zip(img_train, mask_train)
                     if (np.sum(np.array(mask)) > 255 * 5)]
img_train_with_cells, mask_train_with_cells = zip(*trainSetWithCells)

# count number of background (unmarked) pixels: pixels with intensity ≤ 122
nUnmarkedPixels = sum([np.sum(np.array(mask) <= 122) for mask in mask_train_with

# count number of cell (marked) pixels: pixels with intensity > 122
```

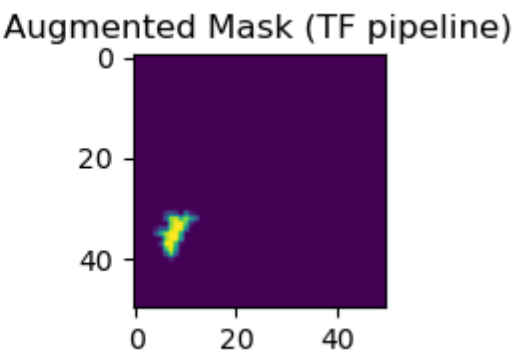
```
nMarkedPixels = sum([np.sum(np.array(mask) > 122) for mask in mask_train_with_ce

# total number of pixels across all masks
totalPixels = nMarkedPixels + nUnmarkedPixels

# inverse frequency for each class: more weight for rarer class
invMarked = totalPixels / nMarkedPixels
invUnmarked = totalPixels / nUnmarkedPixels

# normalize the weights so they sum to 1
normalizingCoeff = 1 / (invMarked + invUnmarked)

lossWeights = [invUnmarked * normalizingCoeff, invMarked  * normalizingCoeff]

print(len(img_train_with_cells))
print("Unmarked: {0}, marked {1}".format(nUnmarkedPixels, nMarkedPixels))
print("Percentage of pixels which are neural cells (filtered training set): {0}%
print(lossWeights)
```

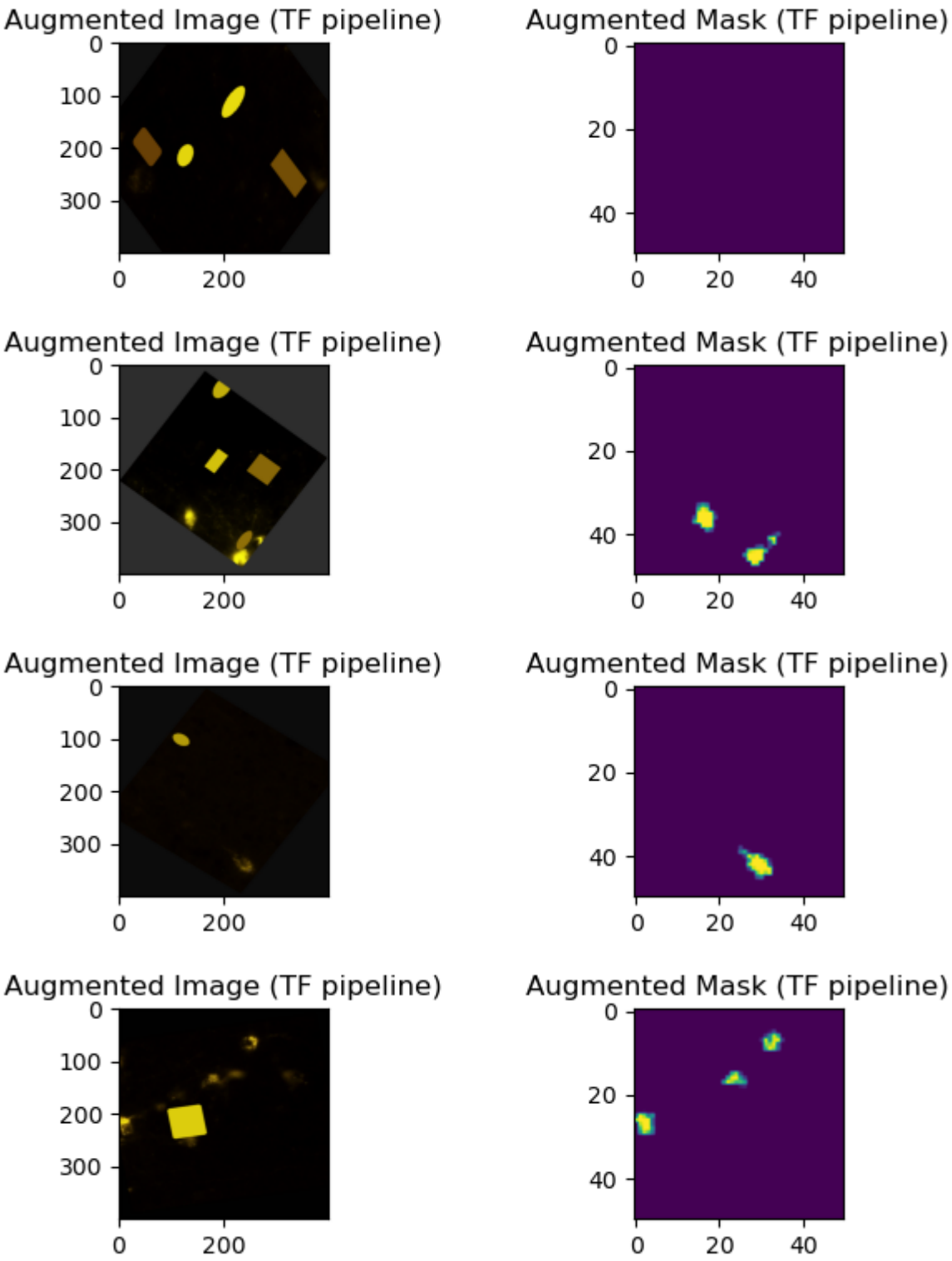
```
1491
Unmarked: 233833043, marked 4726957
Percentage of pixels which are neural cells (filtered training set): 1.9814541415
157612%
[0.019814541415157615, 0.9801854585848424]
```

In [62]:

```
batch_size = 5

# data generator using images and masks with cells
generator = ImageSequence(img_train_with_cells, mask_train_with_cells, batch_siz

# data augmentation features
generator.enableFeatures(True, True)

# first batch of images and masks
images, masks = generator.__getitem__(0)

# image and its corresponding mask
for img, mask in zip(images, masks):
    fig, ax = plt.subplots(1, 2, figsize=(7,2))

    ax[0].imshow(img)
    ax[0].set_title('Augmented Image (TF pipeline)')

    ax[1].imshow(mask)
    ax[1].set_title('Augmented Mask (TF pipeline)')

    plt.tight_layout()
    plt.show()
```







In [64]:
```
# filter for Validation and Test images

valSetWithCells = [(img, mask) for img, mask in zip(img_val, mask_val) if (np.su
if valSetWithCells:
    img_val_with_cells, mask_val_with_cells = zip(*valSetWithCells)
else:
    img_val_with_cells, mask_val_with_cells = [], []

# filter test set for images with cells
testSetWithCells = [(img, mask) for img, mask in zip(img_test, mask_test) if (np
if testSetWithCells:
    img_test_with_cells, mask_test_with_cells = zip(*testSetWithCells)
else:
```

```
    img_test_with_cells, mask_test_with_cells = [], []

print(f"Training images with cells: {len(img_train_with_cells)}")
print(f"Validation images with cells: {len(img_val_with_cells)}")
print(f"Test images with cells: {len(img_test_with_cells)}")
```

```
Training images with cells: 1491
Validation images with cells: 181
Test images with cells: 199
```

In [66]:
```
batch_size = 5

# Training generator
train_generator = ImageSequence(img_train_with_cells, mask_train_with_cells, bat
train_generator.enableFeatures(True, True)  # data augmentation on training

# Validation generator (usually no augmentation)
val_generator = ImageSequence(img_val_with_cells, mask_val_with_cells, batch_siz
val_generator.enableFeatures(False, False)

# Optionally, test generator if needed
test_generator = ImageSequence(img_test_with_cells, mask_test_with_cells, batch_
test_generator.enableFeatures(False, False)
```

FCN Model Implementation

This model uses a straightforward architecture with five convolutional layers that gradually reduce the image size. Unlike many advanced image segmentation networks, it doesn't follow a downsampling-then-upsampling (encoder-decoder) structure, as a lower-resolution output mask is sufficient for the task.

A custom cross-entropy loss function is used, where the weight of the neuron class is reduced by a factor of 3 to prevent the model from over-predicting neuron regions, especially the yellow-highlighted areas.

Given the strong class imbalance in the dataset, the primary evaluation metrics are precision and recall and dice coefficient for neuron cell detection, rather than overall accuracy.

In [112…
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras import backend as K

keras.backend.clear_session()

# loss function from https://stackoverflow.com/questions/46009619/keras-weighted
def weighted_binary_crossentropy(zero_weight, one_weight):
    def weighted_binary_crossentropy(y_true, y_pred):
        b_ce = keras.backend.binary_crossentropy(y_true, y_pred)
        # weighted calc
        weight_vector = y_true * one_weight + (1 - y_true) * zero_weight
        weighted_b_ce = weight_vector * b_ce
        return keras.backend.mean(weighted_b_ce)
```

```
    return weighted_binary_crossentropy

model = keras.Sequential(
    [
        layers.InputLayer(input_shape=(subImgSize, subImgSize, 3)), # input crop

        layers.Lambda(lambda x : x / 255), # normalization
        layers.Conv2D(196, 5, strides=(2,2), activation='relu'), # feature extra
        layers.MaxPooling2D(2), # keeps most important features

        layers.Conv2D(256, 5,  activation='relu'), # bigger receptive field, mor
        layers.MaxPooling2D(2),

        layers.Conv2D(512, 3, activation='relu'),
        layers.Conv2D(512, 1, activation='relu'), # 1x1 convolution acts as a fe


        layers.Conv2DTranspose(1,finalMaskSize - 44,activation="sigmoid") # deco
    ]
)

def dice_coefficient(y_true, y_pred, smooth=1e-6):
    y_true = K.cast(y_true, 'float32')
    y_pred = K.cast(y_pred, 'float32')
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (K.sum(y_true_f) + K.sum(y_pred_f) + s

# adam optimizer
opt = keras.optimizers.Adam(learning_rate=0.001)

# compile the model with the custom loss
model.compile(
    optimizer=opt,
    loss=weighted_binary_crossentropy(lossWeights[0], lossWeights[1] * 0.33), #
    metrics=['accuracy', dice_coefficient]
)
model.summary()
```

```
WARNING:tensorflow:From C:\Users\gonca\anaconda3\Lib\site-packages\keras\src\back
end\common\global_state.py:82: The name tf.reset_default_graph is deprecated. Ple
ase use tf.compat.v1.reset_default_graph instead.
```

```
C:\Users\gonca\anaconda3\Lib\site-packages\keras\src\layers\core\input_layer.py:2
7: UserWarning: Argument `input_shape` is deprecated. Use `shape` instead.
  warnings.warn(
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lambda (Lambda)	(None, 400, 400, 3)	0
conv2d (Conv2D)	(None, 198, 198, 196)	14,896
max_pooling2d (MaxPooling2D)	(None, 99, 99, 196)	0
conv2d_1 (Conv2D)	(None, 95, 95, 256)	1,254,656
max_pooling2d_1 (MaxPooling2D)	(None, 47, 47, 256)	0
conv2d_2 (Conv2D)	(None, 45, 45, 512)	1,180,160
conv2d_3 (Conv2D)	(None, 45, 45, 512)	262,656
conv2d_transpose (Conv2DTranspose)	(None, 50, 50, 1)	18,433

Total params: 2,730,801 (10.42 MB)

Trainable params: 2,730,801 (10.42 MB)

Non-trainable params: 0 (0.00 B)

Training - [Phase 1.1]

The model is trained for 5 epochs on the filtered training set without any data augmentation. At this stage, the CNN tends to produce false positives, marking any area with even a slight yellow tint as a neuron.

In [114…

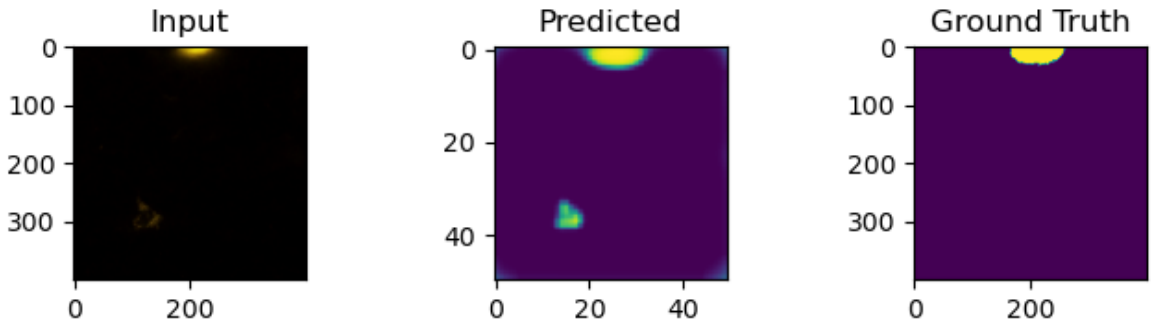
```
# with the filtered dataset
batch_size = 50

# data generator for training
generator = ImageSequence(img_train_with_cells, mask_train_with_cells, batch_siz

# train
history1 = model.fit(
    x=generator,
    epochs=5,
    validation_data=val_generator
)
```

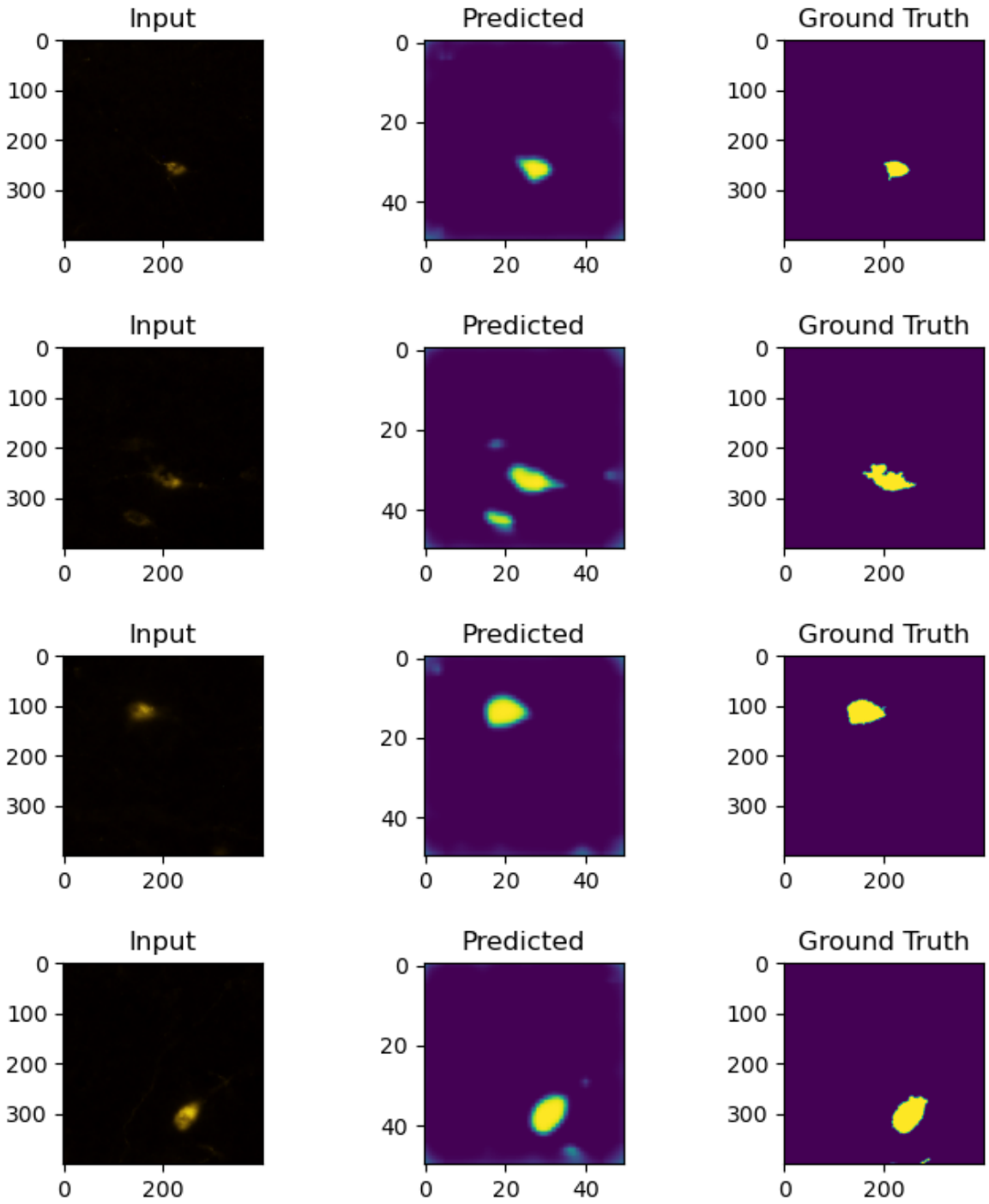
```
Epoch 1/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 636s 22s/step - accuracy: 0.7294 - dice_coefficient:
0.0803 - loss: 0.0138 - val_accuracy: 0.9898 - val_dice_coefficient: 0.1144 - val
_loss: 0.0025
Epoch 2/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 639s 22s/step - accuracy: 0.9710 - dice_coefficient:
0.3091 - loss: 0.0044 - val_accuracy: 0.9835 - val_dice_coefficient: 0.1750 - val
_loss: 0.0018
Epoch 3/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 626s 22s/step - accuracy: 0.9670 - dice_coefficient:
0.3575 - loss: 0.0038 - val_accuracy: 0.9925 - val_dice_coefficient: 0.2624 - val
_loss: 0.0014
Epoch 4/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 610s 21s/step - accuracy: 0.9691 - dice_coefficient:
0.4101 - loss: 0.0033 - val_accuracy: 0.9871 - val_dice_coefficient: 0.2495 - val
_loss: 0.0013
Epoch 5/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 633s 22s/step - accuracy: 0.9660 - dice_coefficient:
0.4132 - loss: 0.0031 - val_accuracy: 0.9907 - val_dice_coefficient: 0.2707 - val
_loss: 0.0012
```

In [129…

```python
# with training data
def previewResults():
    testCases = np.array([np.array(img) for img in img_train_with_cells[:5]])
    testGroundTruth = mask_train_with_cells[:5]
    predictions = model.predict(testCases)

    for img, pred, actual in zip(testCases, predictions, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')

        ax[1].imshow(pred)
        ax[1].set_title('Predicted')

        ax[2].imshow(actual)
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
        plt.show()

previewResults()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 742ms/step
```



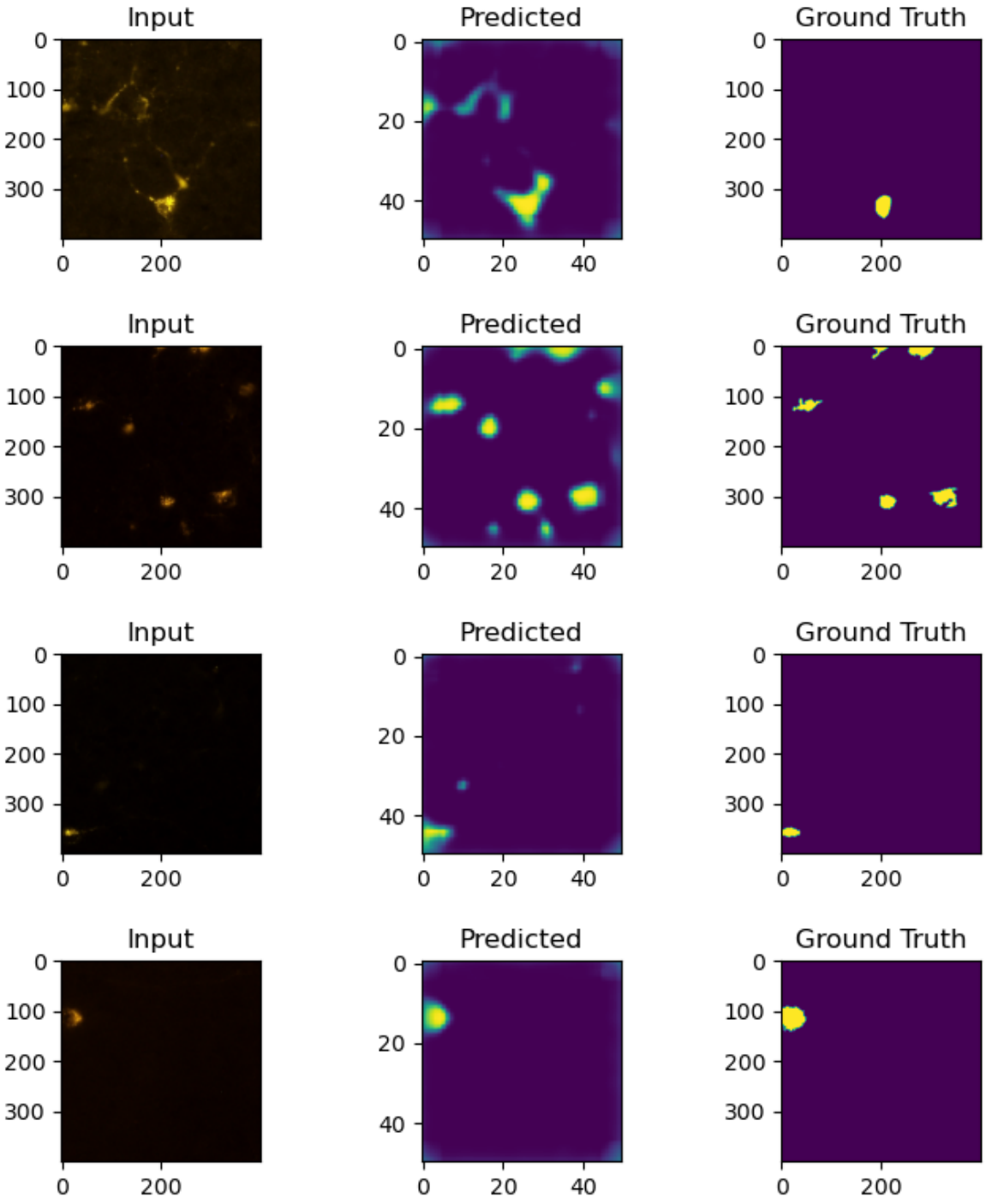


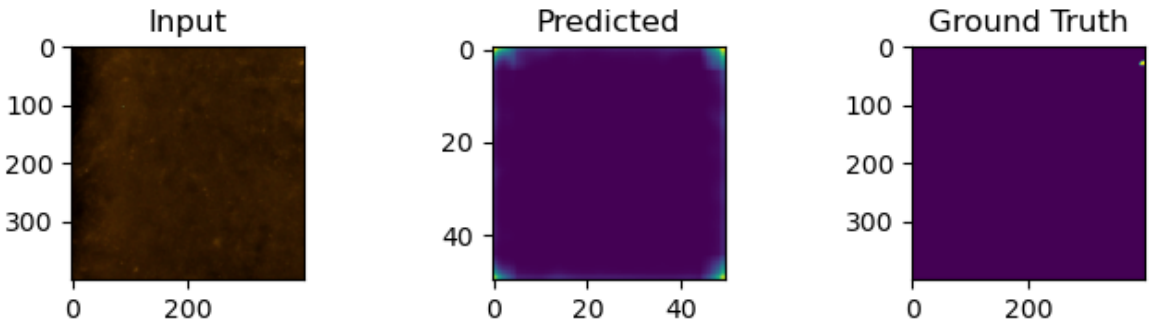
In [131…

```
# with validation data

def previewResults():
    testCases = np.array([np.array(img) for img in img_val_with_cells[:5]])
    testGroundTruth = mask_val_with_cells[:5]
    predictions = model.predict(testCases)

    for img, pred, actual in zip(testCases, predictions, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')

        ax[1].imshow(pred)
        ax[1].set_title('Predicted')

        ax[2].imshow(actual)
```

```
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
        plt.show()

previewResults()
```

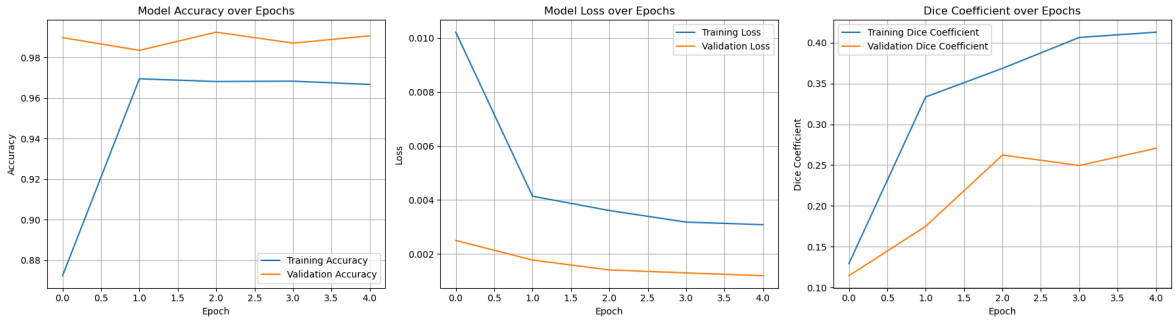
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 493ms/step
```





In [133…

```
plt.figure(figsize=(18, 5))

# Accuracy
plt.subplot(1, 3, 1)
plt.plot(history1.history['accuracy'], label='Training Accuracy')
plt.plot(history1.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss
plt.subplot(1, 3, 2)
plt.plot(history1.history['loss'], label='Training Loss')
plt.plot(history1.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Dice Coefficient
plt.subplot(1, 3, 3)
plt.plot(history1.history['dice_coefficient'], label='Training Dice Coefficient'
plt.plot(history1.history['val_dice_coefficient'], label='Validation Dice Coeffi
plt.title('Dice Coefficient over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Dice Coefficient')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Accuracy: The gap between training and validation accuracy is relatively small throughout, which is a good sign that the model isn't severely overfitting early on.

Loss: The validation loss is generally slightly higher than the training loss, which is expected.

Dice coeff: The validation Dice coefficient is consistently higher than the training Dice coefficient after epoch 1. This could indicate that the model is generalizing relatively well to unseen data based on this specific metric, or it might be related to the specific characteristics of our validation set.

The first training session appears promising. The FCN model is clearly learning, as evidenced by the decreasing loss and increasing accuracy and Dice coefficient. There are no immediate signs of severe overfitting or underfitting.

Training - [Phase 1.2]

The model is trained for 10 more epochs on the filtered dataset. During this phase, random shapes are added to the input images to help the network learn the typical shapes of brain cells. This leads to some noticeable improvement.

In [74]:
```
generator.enableFeatures(True, False) # enable random shapes

history = model.fit(
    x=generator,
    epochs=10,
    validation_data=val_generator)
```

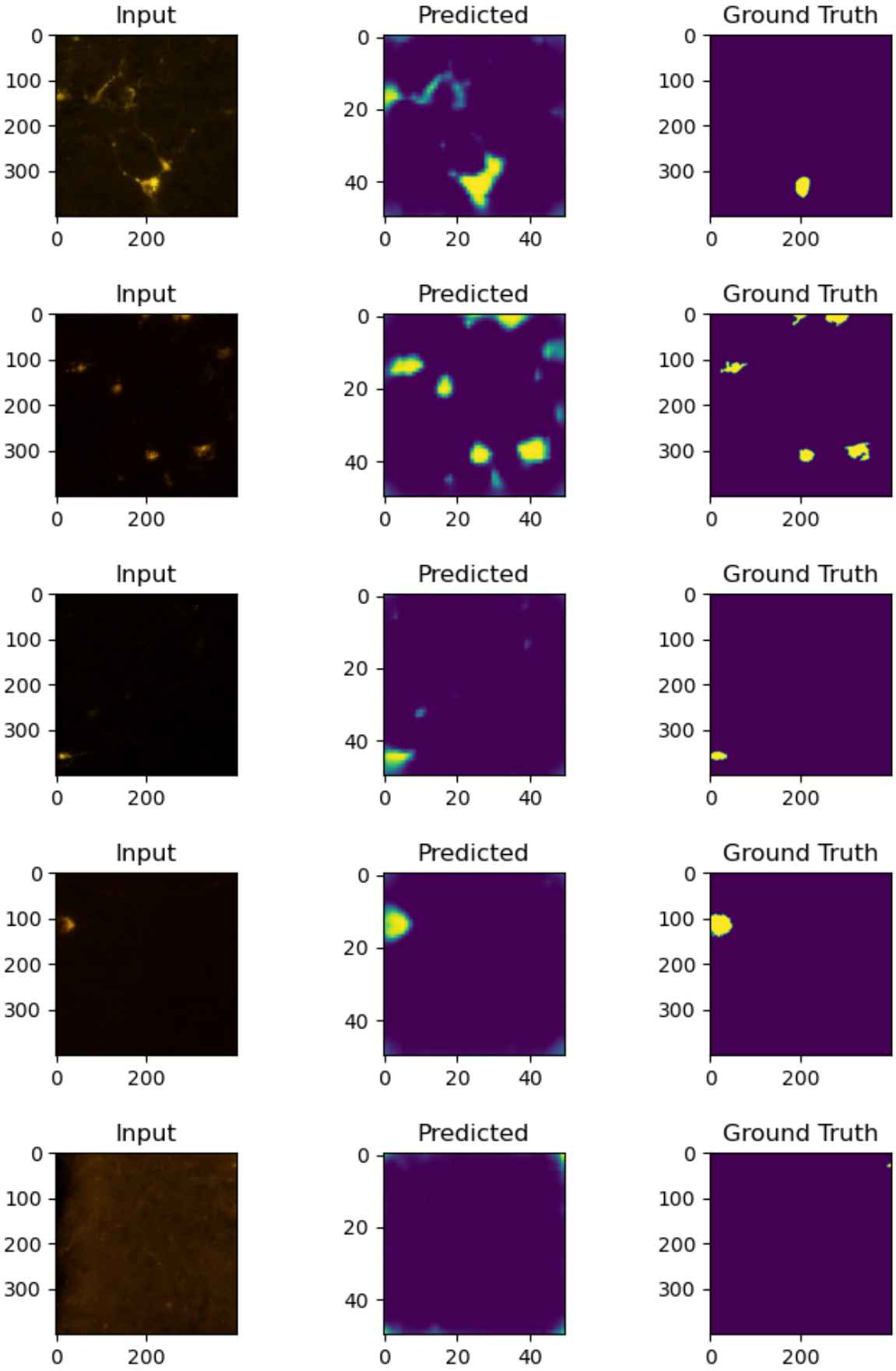
```
Epoch 1/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 584s 20s/step - accuracy: 0.9455 - dice_coefficient:
0.2838 - loss: 0.0050 - val_accuracy: 0.9757 - val_dice_coefficient: 0.4903 - val
_loss: 0.0030
Epoch 2/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 587s 20s/step - accuracy: 0.9613 - dice_coefficient:
0.3706 - loss: 0.0036 - val_accuracy: 0.9667 - val_dice_coefficient: 0.3786 - val
_loss: 0.0033
Epoch 3/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 607s 21s/step - accuracy: 0.9591 - dice_coefficient:
0.3455 - loss: 0.0037 - val_accuracy: 0.9756 - val_dice_coefficient: 0.5004 - val
_loss: 0.0027
Epoch 4/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 584s 20s/step - accuracy: 0.9678 - dice_coefficient:
0.4120 - loss: 0.0031 - val_accuracy: 0.9729 - val_dice_coefficient: 0.4619 - val
_loss: 0.0026
Epoch 5/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 761s 26s/step - accuracy: 0.9690 - dice_coefficient:
0.4167 - loss: 0.0029 - val_accuracy: 0.9777 - val_dice_coefficient: 0.5119 - val
_loss: 0.0027
Epoch 6/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 736s 25s/step - accuracy: 0.9690 - dice_coefficient:
0.4196 - loss: 0.0031 - val_accuracy: 0.9786 - val_dice_coefficient: 0.4831 - val
_loss: 0.0029
Epoch 7/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 647s 22s/step - accuracy: 0.9687 - dice_coefficient:
0.4245 - loss: 0.0030 - val_accuracy: 0.9779 - val_dice_coefficient: 0.5167 - val
_loss: 0.0026
Epoch 8/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 594s 20s/step - accuracy: 0.9676 - dice_coefficient:
0.4067 - loss: 0.0032 - val_accuracy: 0.9737 - val_dice_coefficient: 0.4489 - val
_loss: 0.0026
Epoch 9/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 629s 22s/step - accuracy: 0.9697 - dice_coefficient:
0.4224 - loss: 0.0029 - val_accuracy: 0.9784 - val_dice_coefficient: 0.5276 - val
_loss: 0.0025
Epoch 10/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 578s 20s/step - accuracy: 0.9705 - dice_coefficient:
0.4353 - loss: 0.0028 - val_accuracy: 0.9727 - val_dice_coefficient: 0.5002 - val
_loss: 0.0024
```

In [76]:

```python
# second training images
def previewResults():
    testCases = np.array([np.array(img) for img in img_val_with_cells[:5]])
    testGroundTruth = mask_val_with_cells[:5]
    predictions = model.predict(testCases)

    for img, pred, actual in zip(testCases, predictions, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')

        ax[1].imshow(pred)
        ax[1].set_title('Predicted')

        ax[2].imshow(actual)
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
```

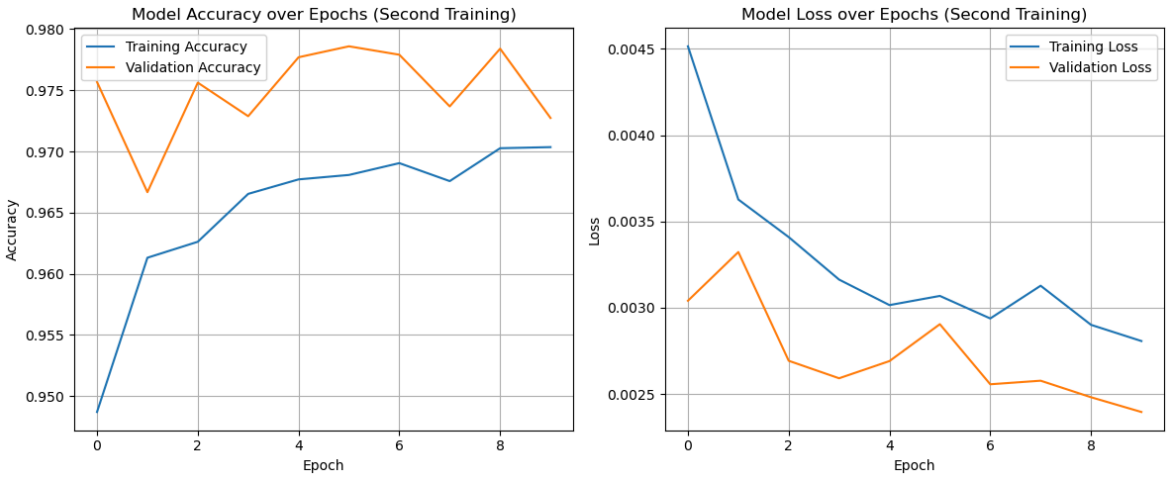
```
        plt.show()

previewResults()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 506ms/step
```



In [80]:
```
# second training
plt.figure(figsize=(12, 5))
```

```
# accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs (Second Training)')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs (Second Training)')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Accuracy: Validation accuracy is generally higher and fluctuates more, sometimes dropping below the peak, which suggests it might be finding some more challenging examples in the validation set or that there's slight overfit on some epochs. The gap between training and validation accuracy is relatively small.

Loss: The validation loss is consistently lower than the training loss, which is a positive sign for generalization. Both curves show some fluctuations, but the overall trend is downward.

The second training session for the FCN model demonstrates continued learning and improved performance compared to the start of the session. The consistent decrease in both training and validation loss, coupled with the rising accuracy, indicates that the model is becoming more proficient at the task. The fact that validation loss is generally lower than training loss and validation accuracy is often higher suggests that the FCN is generalizing quite well to unseen data within this session, although the validation accuracy does show some minor dips. The training seems stable, and the model is effectively converging.

Training - [Phase 2]

The model is trained for 5 epochs on the full training dataset, with all augmentations turned on (affine transformations and random shapes).

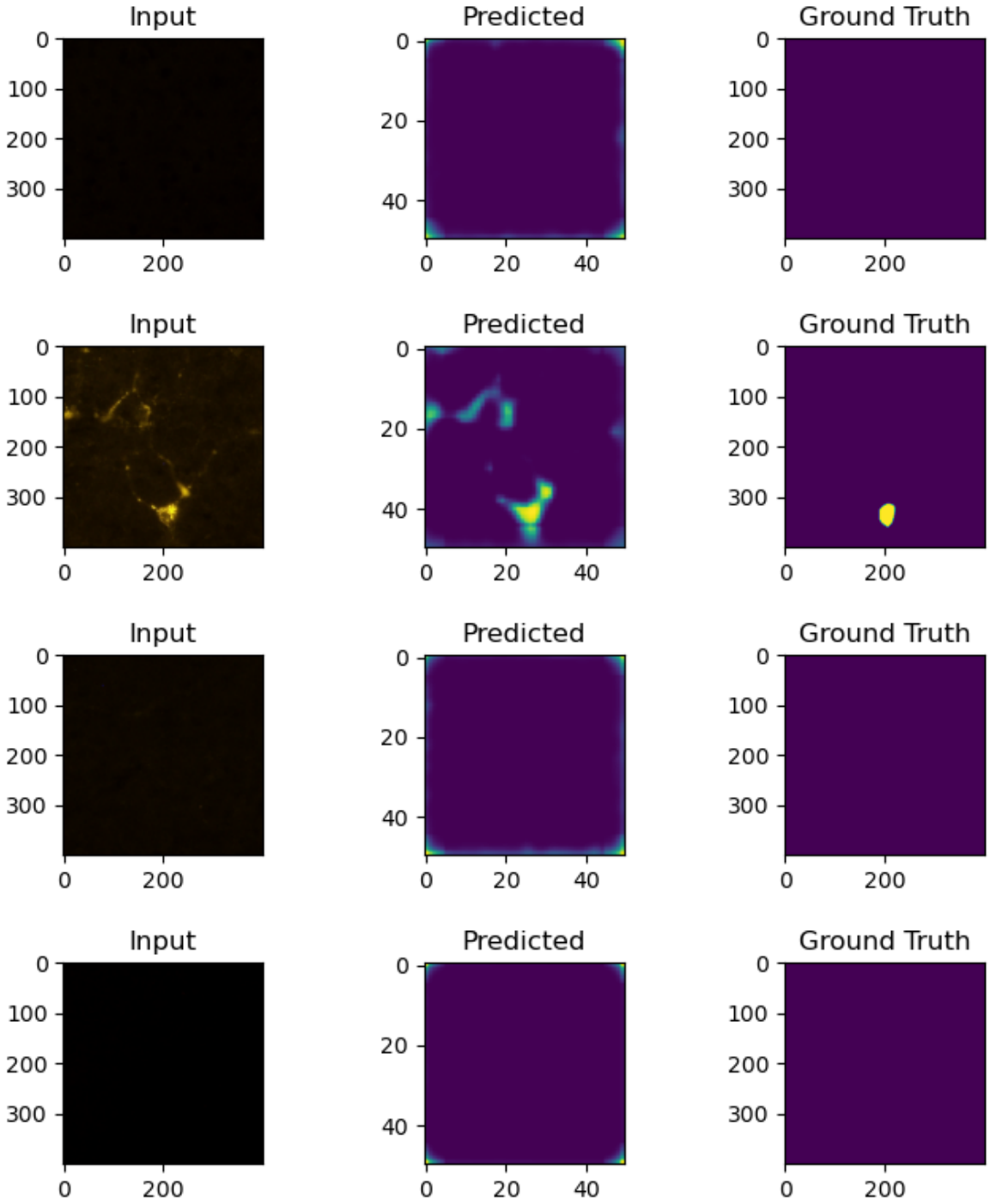
In [165…
```
generator = ImageSequence(img_train, mask_train, batch_size, finalMaskSize) #use
generator.enableFeatures(True, True) #Enable Random shapes + transforms

val_generatorce = ImageSequence(img_val, mask_val, batch_size, finalMaskSize)

history3 = model.fit(
    x=generator,
    epochs=5,
    validation_data=val_generator)
```

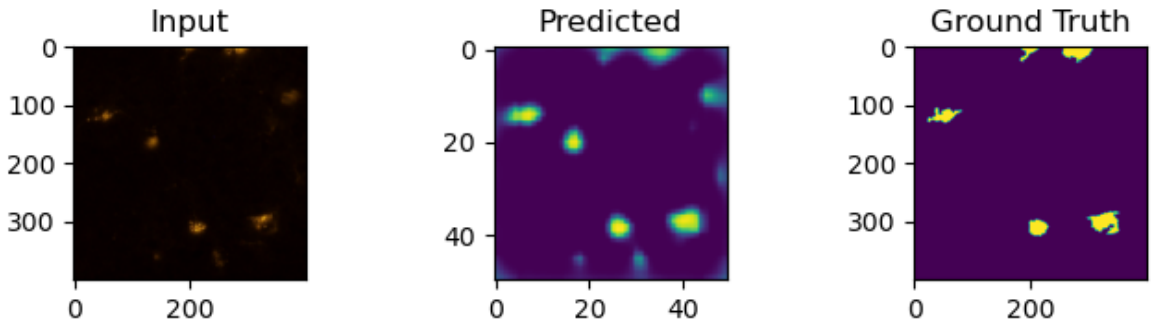
```
C:\Users\gonca\anaconda3\Lib\site-packages\keras\src\trainers\data_adapters\py_da
taset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__
init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_mult
iprocessing`, `max_queue_size`. Do not pass these arguments to `fit()`, as they w
ill be ignored.
  self._warn_if_super_not_called()
Epoch 1/5
90/90 ━━━━━━━━━━━━━━━━━━━━ 1813s 20s/step - accuracy: 0.9772 - dice_coefficient:
0.1313 - loss: 0.0025 - val_accuracy: 0.9923 - val_dice_coefficient: 0.2783 - val
_loss: 0.0014
Epoch 2/5
90/90 ━━━━━━━━━━━━━━━━━━━━ 1725s 19s/step - accuracy: 0.9883 - dice_coefficient:
0.2789 - loss: 0.0013 - val_accuracy: 0.9925 - val_dice_coefficient: 0.3063 - val
_loss: 0.0013
Epoch 3/5
90/90 ━━━━━━━━━━━━━━━━━━━━ 1814s 20s/step - accuracy: 0.9902 - dice_coefficient:
0.3200 - loss: 0.0011 - val_accuracy: 0.9939 - val_dice_coefficient: 0.3404 - val
_loss: 0.0014
Epoch 4/5
90/90 ━━━━━━━━━━━━━━━━━━━━ 1788s 20s/step - accuracy: 0.9897 - dice_coefficient:
0.3399 - loss: 0.0012 - val_accuracy: 0.9904 - val_dice_coefficient: 0.2595 - val
_loss: 0.0012
Epoch 5/5
90/90 ━━━━━━━━━━━━━━━━━━━━ 1803s 20s/step - accuracy: 0.9900 - dice_coefficient:
0.3216 - loss: 0.0011 - val_accuracy: 0.9936 - val_dice_coefficient: 0.2849 - val
_loss: 0.0014
```

In [181…
```
# third training images
def previewResults():
    testCases = np.array([np.array(img) for img in img_val[:5]])
    testGroundTruth = mask_val[:5]
    predictions = model.predict(testCases)

    for img, pred, actual in zip(testCases, predictions, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')

        ax[1].imshow(pred)
        ax[1].set_title('Predicted')
```

```
        ax[2].imshow(actual)
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
        plt.show()

previewResults()
```

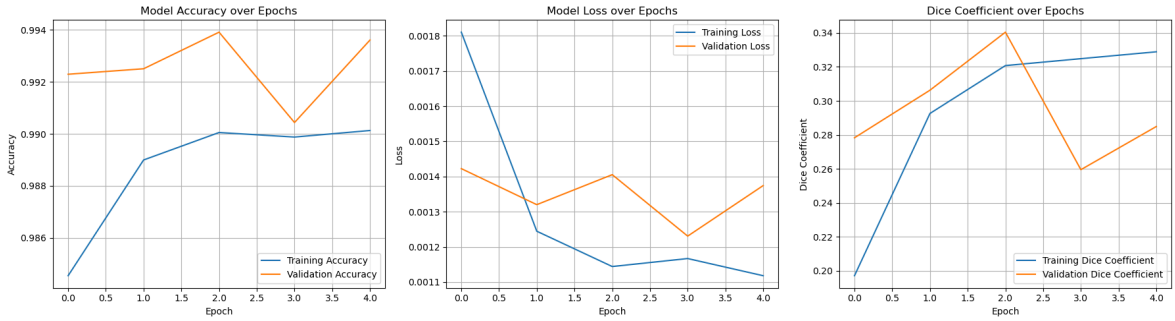
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 486ms/step
```





In [183…

```
plt.figure(figsize=(18, 5))

# Accuracy
plt.subplot(1, 3, 1)
plt.plot(history3.history['accuracy'], label='Training Accuracy')
plt.plot(history3.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss
plt.subplot(1, 3, 2)
plt.plot(history3.history['loss'], label='Training Loss')
plt.plot(history3.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Dice Coefficient
plt.subplot(1, 3, 3)
plt.plot(history3.history['dice_coefficient'], label='Training Dice Coefficient'
plt.plot(history3.history['val_dice_coefficient'], label='Validation Dice Coeffi
plt.title('Dice Coefficient over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Dice Coefficient')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Accuracy: The overall range for both is very high (above 0.985), indicating that the model is correctly classifying a large proportion of pixels, likely dominated by the background.

Loss: Validation loss, however, is very erratic, showing significant dips and spikes. It starts higher than training loss, dips lower around epoch 3, and then rises again, indicating instability in its generalization to unseen data.

Dice coeff: The discrepancy and instability between training and validation Dice suggest issues with generalization despite the promising peak.

The overall high accuracy but fluctuating Dice coefficient might indicate that while the model is very good at identifying the dominant background class, its performance on the less frequent foreground (cells) is inconsistent and still needs improvement for robust segmentation. Further investigation into the cause of the validation metric fluctuations (e.g., data distribution, learning rate, or potential overfitting after epoch 2) would be beneficial.

Model Evaluation

All test sub-images are passed through the model to get predictions. A threshold of 0.5 is applied to turn the output into binary values (0 or 1). These predictions are then flattened and combined into a single 1-D array, which is used to generate evaluation metrics using SciPy's classification report.

In [198…
```
predste = model.predict(np.array(img_test))
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 54s 3s/step
```

In [200…
```
preds_binary = (predste > 0.5).astype('float32')
```

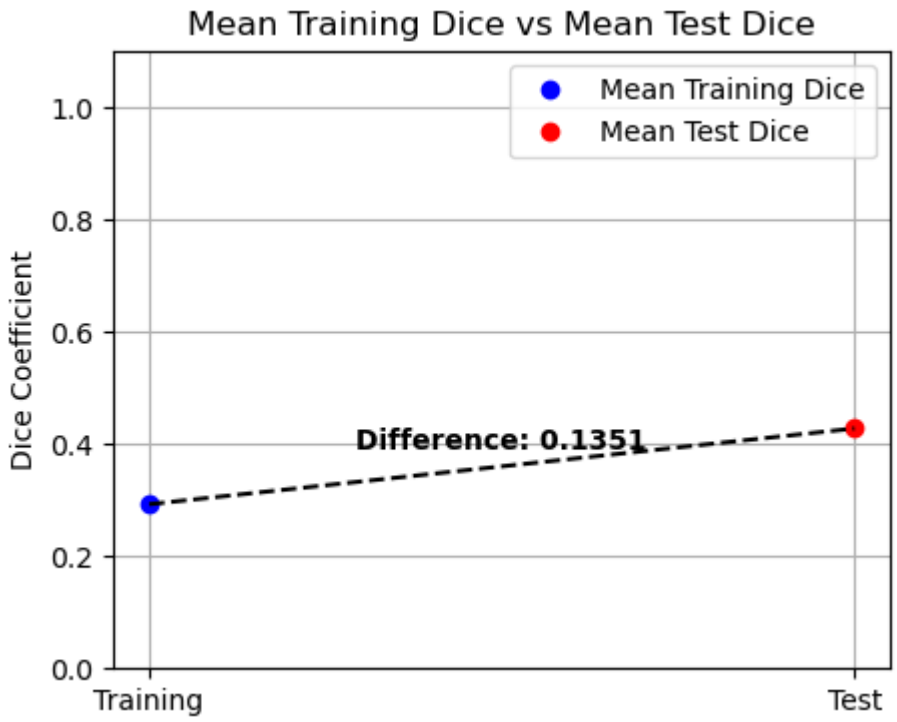
In [202…
```
mask_test_np = np.array([np.array(m) for m in mask_test])

mask_test_tensor = tf.convert_to_tensor(mask_test_np, dtype=tf.float32)
mask_test_tensor = tf.expand_dims(mask_test_tensor, axis=-1)

# Resize all masks to (finalMaskSize, finalMaskSize)
mask_test_resized = tf.image.resize(
    mask_test_tensor,
    size=(finalMaskSize, finalMaskSize),
    method='nearest'
).numpy()
```

In [204…
```
mask_test_binary = (mask_test_resized > 0.5).astype('float32')
```

In [281…
```
def dice_coef_np(y_true, y_pred, smooth=1e-6):
    y_true_f = y_true.flatten()
    y_pred_f = y_pred.flatten()
    intersection = np.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (np.sum(y_true_f) + np.sum(y_pred_f) +

# Calculate average Dice score
dice_scores = [dice_coef_np(y_t, y_p) for y_t, y_p in zip(mask_test_binary, pred
mean_dice = np.mean(dice_scores)
```

In [283…

```
import matplotlib.pyplot as plt
import numpy as np

# training history
train_dice = history3.history['dice_coefficient']

# Mean Dice coefficient during training (average over all epochs)
mean_dice_train = np.mean(train_dice)
mean_dice_test = mean_dice

plt.figure(figsize=(5,4))


plt.plot([0], [mean_dice_train], 'bo', label='Mean Training Dice')
plt.plot([1], [mean_dice_test], 'ro', label='Mean Test Dice')


plt.plot([0, 1], [mean_dice_train, mean_dice_test], 'k--')

# Calculate difference and plot as text
diff = mean_dice_test - mean_dice_train
mid_x = 0.5
mid_y = (mean_dice_train + mean_dice_test) / 2
plt.text(mid_x, mid_y + 0.03, f'Difference: {diff:.4f}', ha='center', fontsize=1

plt.xticks([0, 1], ['Training', 'Test'])
plt.ylabel('Dice Coefficient')
plt.title('Mean Training Dice vs Mean Test Dice')
plt.legend()
plt.ylim(0, 1.1)
plt.grid(True)
plt.show()
```



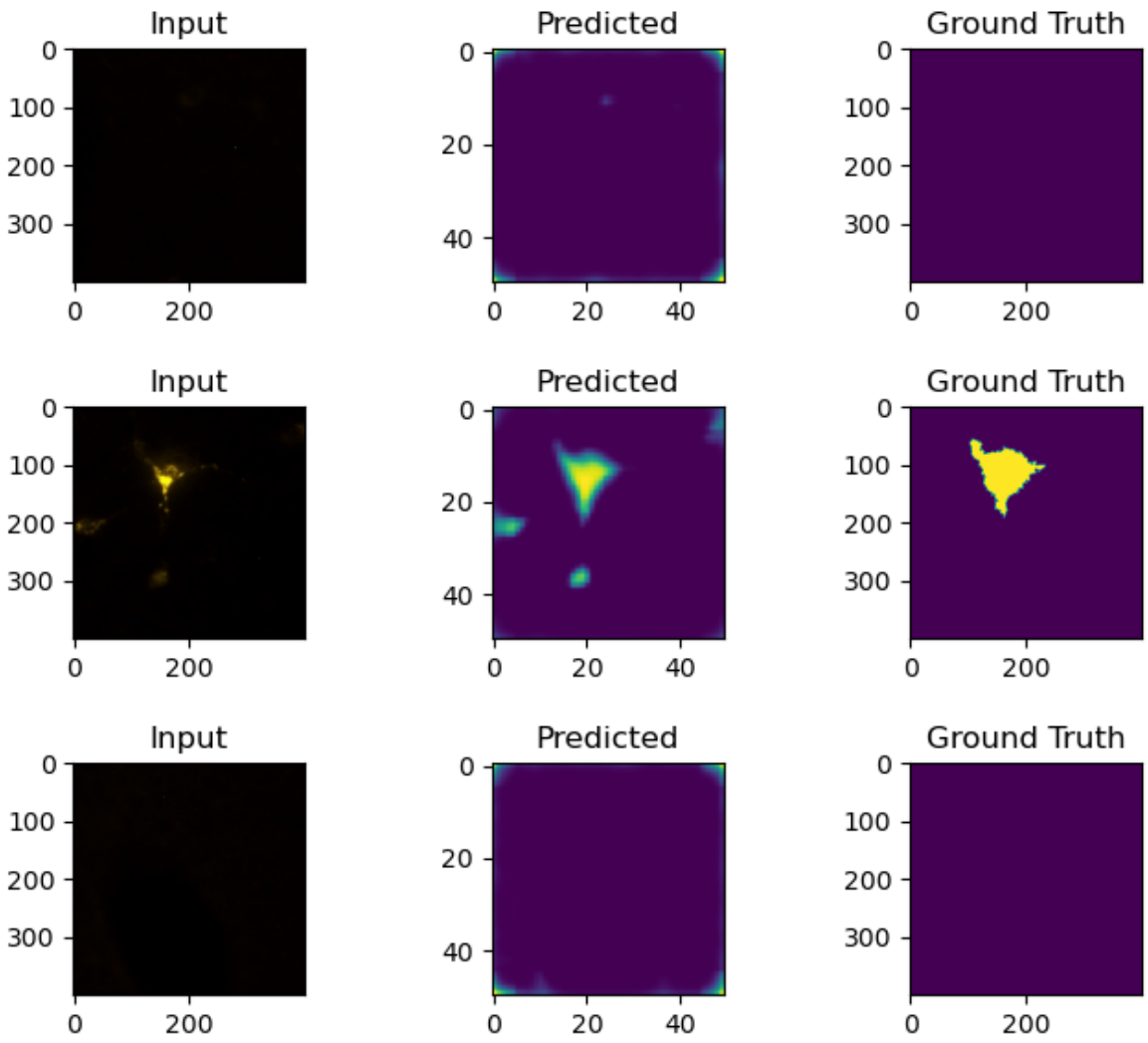
The difference of 0.1351 indicates that the model performed considerably better on the test data than on the training data, which is highly unusual and counter-intuitive in machine learning. Potential Reasons for this Unusual Result:

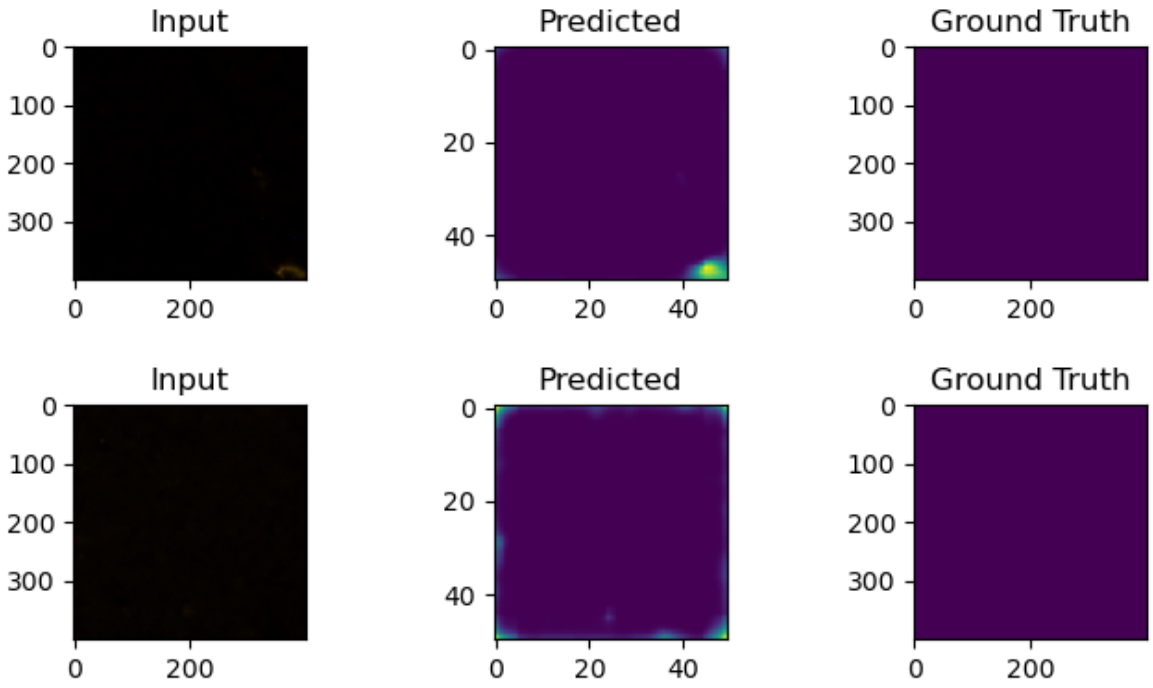
- Easier Test Set: The test set might be significantly "easier" than the training set.
- Harder Training Set: Conversely, the training set might contain particularly challenging examples that drag down the average training Dice, while the test set represents a more typical or easier distribution.
- Augmentation issues

In [210…
```
#img_test, mask_test
predictions = model.predict(np.array([np.array(img) for img in img_test]))
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 54s 3s/step
```

In [212…
```
# prediction with test dataset
import itertools

for img, pred, actual in itertools.islice(zip(img_test, predictions, mask_test),
    fig, ax = plt.subplots(1, 3, figsize=(7,2))
    ax[0].imshow(img)
    ax[0].set_title('Input')

    ax[1].imshow(pred)
    ax[1].set_title('Predicted')
    ax[2].imshow(actual)
    ax[2].set_title('Ground Truth')
    plt.tight_layout()
    plt.show()
```



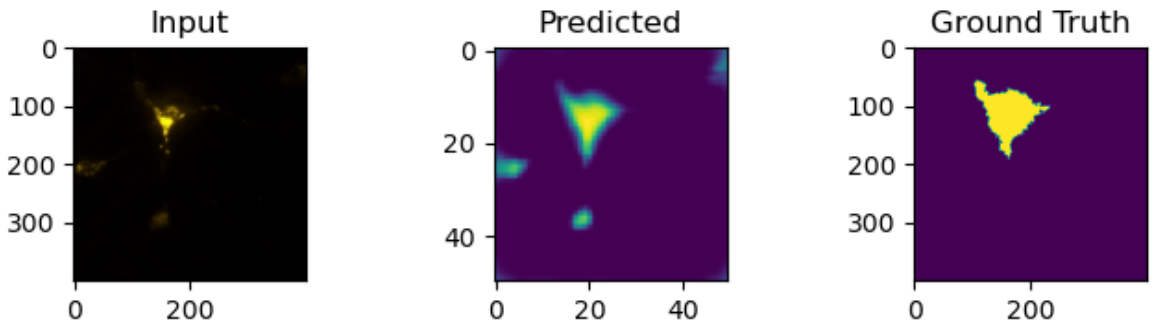


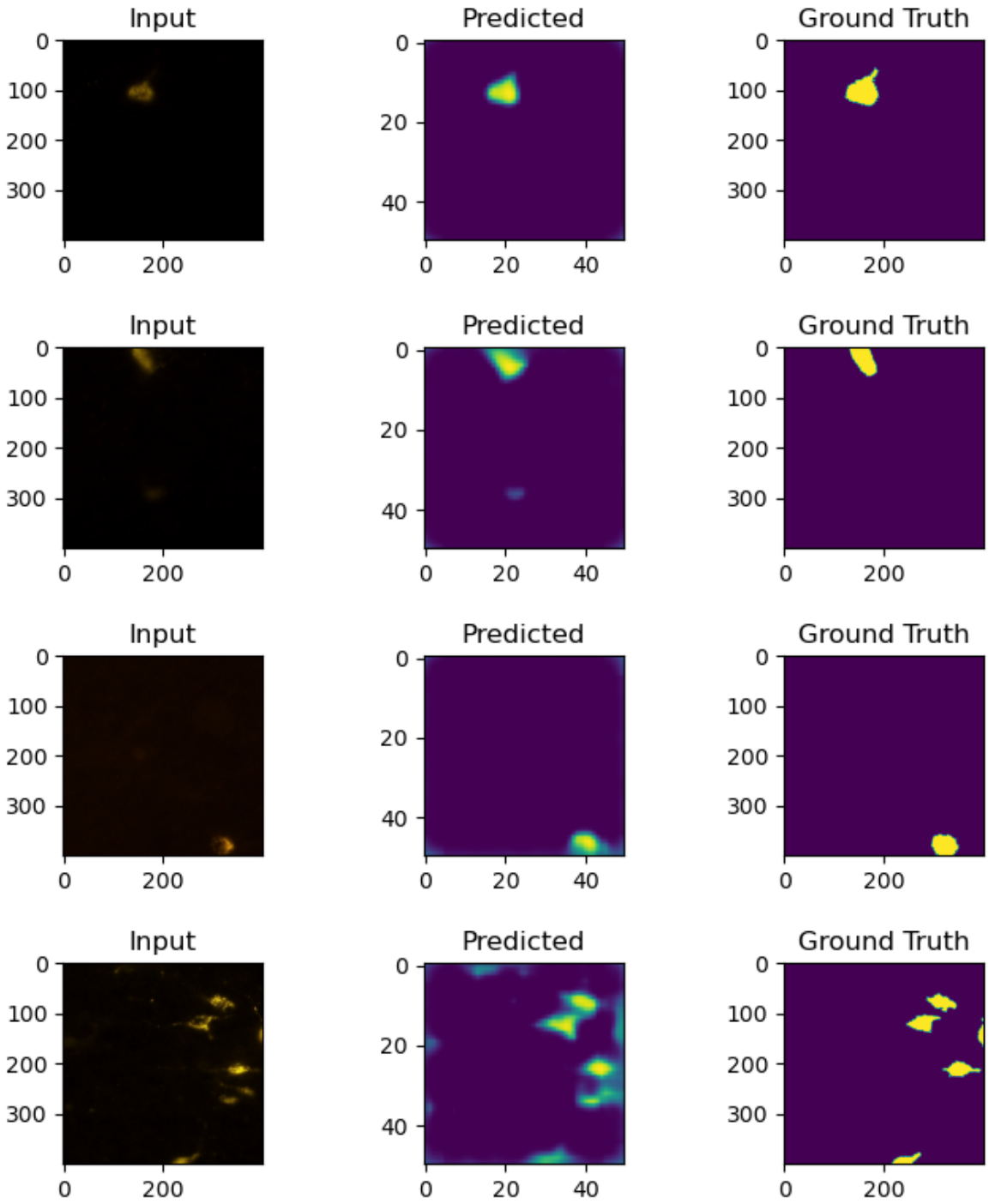
The FCN model appears to be competent at identifying the presence or absence of foreground objects. It excels at classifying vast background areas correctly. However, when foreground objects are present (Row 2), the model struggles with precise segmentation and boundary delineation, often resulting in over-segmentation or blob-like predictions that are larger and less accurate than the ground truth.

In [214…
```
#img_test, mask_test (with cells only)
predictions2 = model.predict(np.array([np.array(img) for img in img_test_with_ce
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 19s 3s/step
```

In [216…
```
for img, pred, actual in itertools.islice(zip(img_test_with_cells, predictions2,
    fig, ax = plt.subplots(1, 3, figsize=(7,2))
    ax[0].imshow(img)
    ax[0].set_title('Input')

    ax[1].imshow(pred)
    ax[1].set_title('Predicted')
    ax[2].imshow(actual)
    ax[2].set_title('Ground Truth')
    plt.tight_layout()
    plt.show()
```



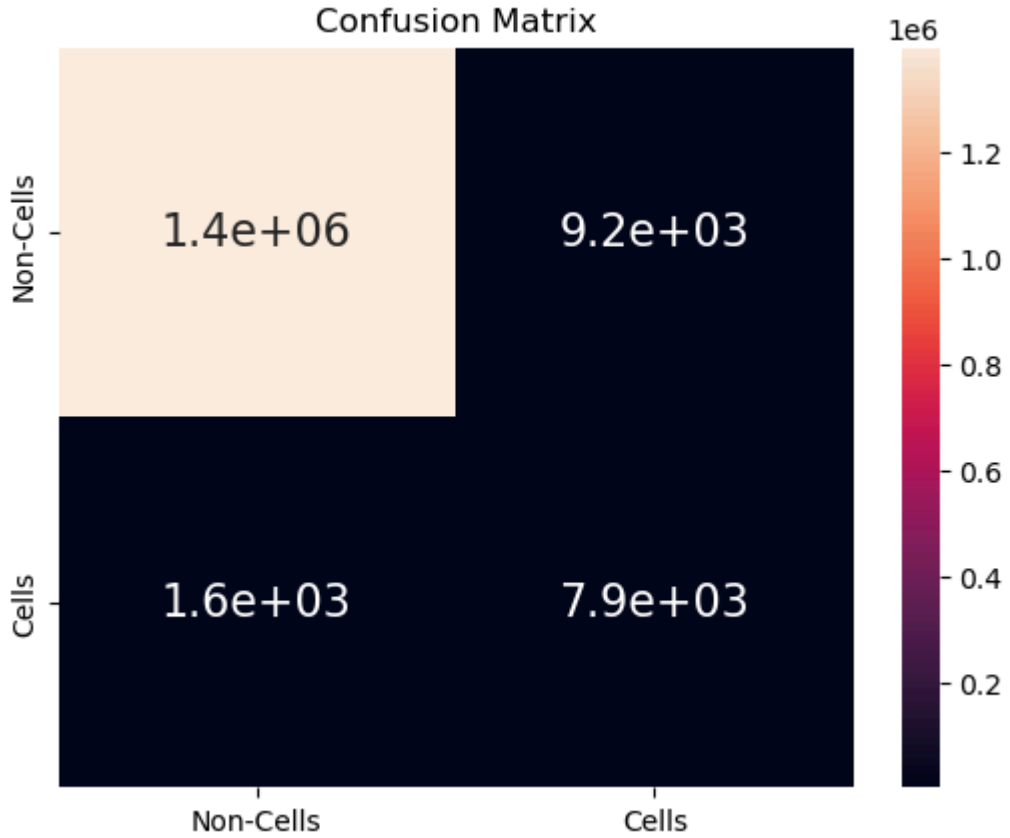


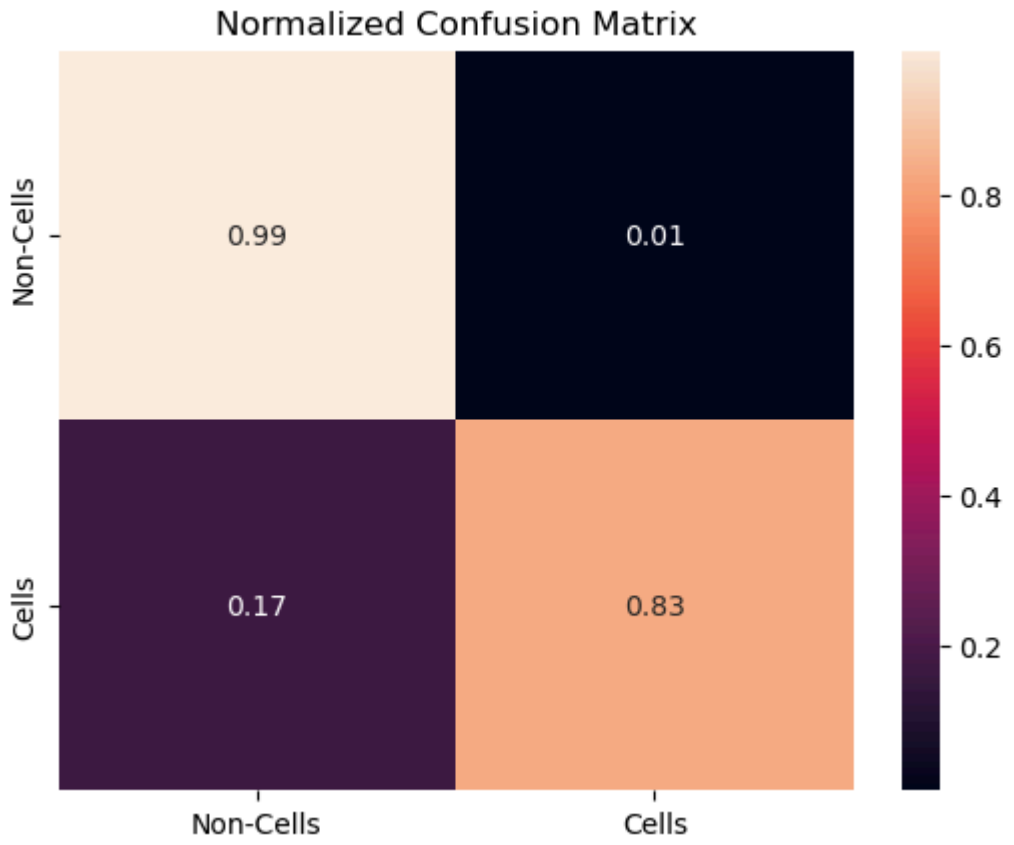
In [218…

```
# confusion matrix for accuracy (false positives)
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
import seaborn as sn

def computeMetrics(pred, groundTruthImages):
    actualList = []
    predList = []
    for p, groundTruth in zip(pred, groundTruthImages):
        resized = groundTruth.resize((finalMaskSize,finalMaskSize), Image.BILINE
        resizedThreshold = np.array(resized, dtype=np.uint8) > 122 #0-255
        predThreshold = (p > 0.5).astype(np.uint8) #0-1
        actualList.append(np.reshape(resizedThreshold, -1))
        predList.append(np.reshape(predThreshold, -1))
    actual = np.concatenate((actualList), axis=0)
    predictions = np.concatenate((predList), axis=0)
    return (predictions, actual)
```

```
pred1D, actual1D = computeMetrics(predictions, mask_test)
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
import seaborn as sn

labels= ["Non-Cells", "Cells"]
print(classification_report(actual1D, pred1D, target_names=labels))
confusionMatrix = confusion_matrix(actual1D, pred1D)

df_cm = pd.DataFrame(confusionMatrix, index=labels, columns=labels)
sn.heatmap(df_cm, annot=True, annot_kws={"size": 16}).set_title("Confusion Matri
plt.show()

confMatrixNormalized = confusionMatrix.astype('float') / confusionMatrix.sum(axi
df_cm_norm = pd.DataFrame(confMatrixNormalized, index=labels, columns=labels)
sn.heatmap(df_cm_norm, annot=True, fmt=".2f")
plt.title("Normalized Confusion Matrix")
plt.show()
```

```
              precision    recall  f1-score   support

   Non-Cells       1.00      0.99      1.00   1405519
       Cells       0.46      0.83      0.59      9481

    accuracy                           0.99   1415000
   macro avg       0.73      0.91      0.80   1415000
weighted avg       1.00      0.99      0.99   1415000
```





This FCN model is highly effective at identifying background pixels (Non-Cells), achieving near-perfect precision and recall for this dominant class. However, its performance on the minority "Cells" class is mixed and less precise.

The model demonstrates good recall (0.83) for cells, meaning it successfully detects a large proportion of the actual cells present. This is a positive for not missing too many targets.

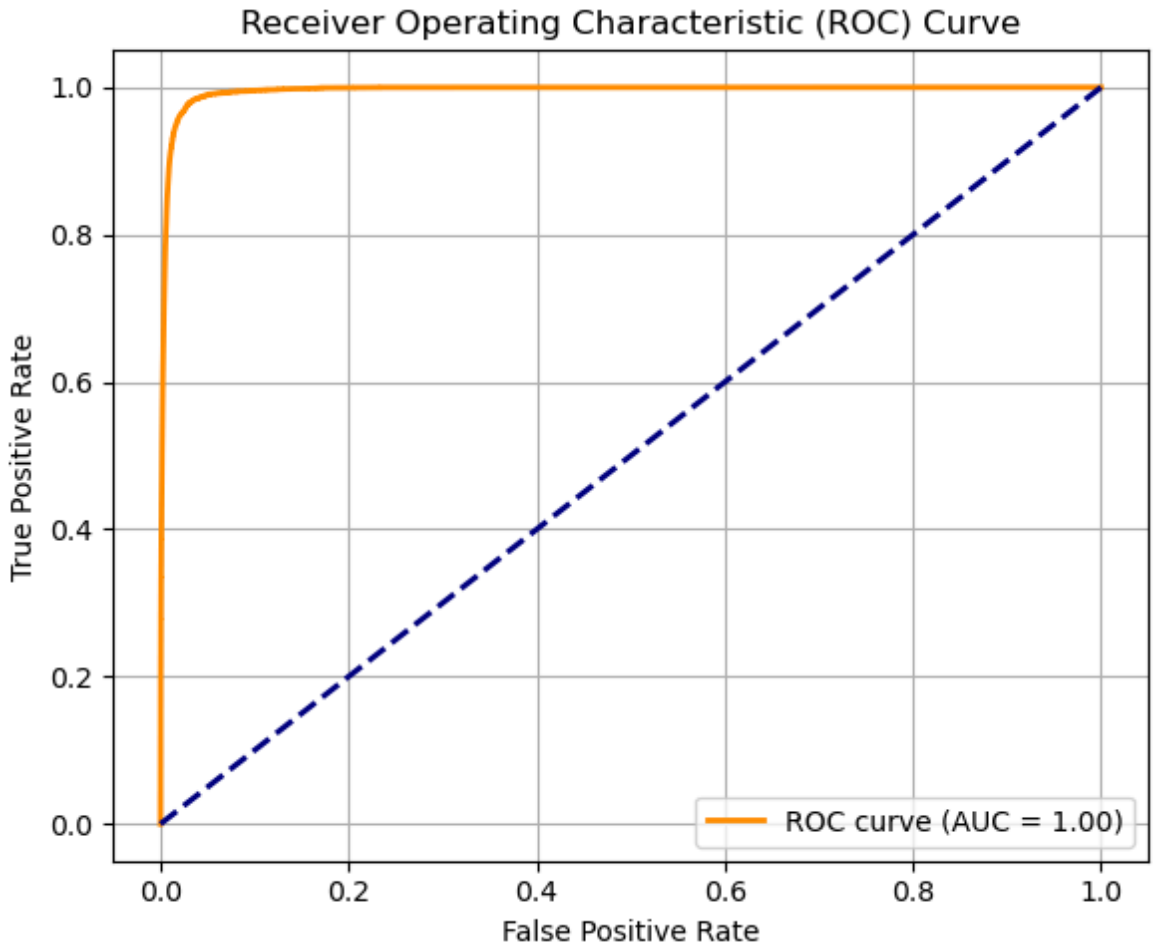
The major weakness lies in its low precision (0.46) for cells. This indicates a high rate of false positives for cells, meaning a substantial number of pixels predicted as "Cells" are actually background. This translates to over-segmentation or "blobby" predictions where the model includes surrounding background into the cell region, or predicts cells where none exist. The visual examples previously seen (where the FCN predicted larger, less precise blobs for cells) perfectly align with this quantitative finding.

In summary, the FCN is good at finding cells but struggles with outlining them precisely and avoiding false positives. Its overall accuracy is inflated by its strong performance on the abundant background.

In [220…

```
# roc curve

from sklearn.metrics import roc_curve, auc

def flattenPredictions(predictions, groundTruthImages):
    actualList = []
    predProbsList = []

    for pred, groundTruth in zip(predictions, groundTruthImages):
        # resize ground truth to match prediction
```

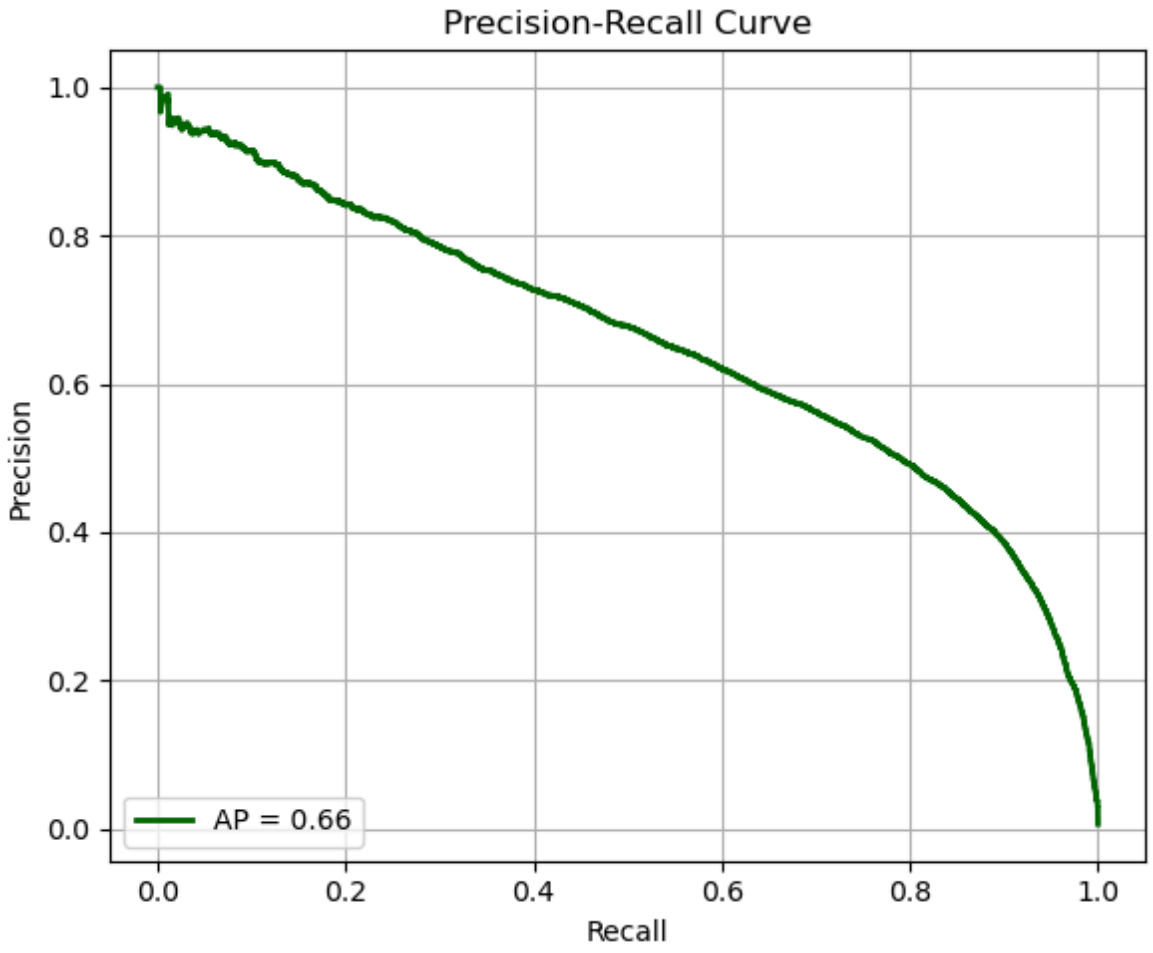
```
        resized = groundTruth.resize((finalMaskSize, finalMaskSize), Image.BILIN
        actualMask = (np.array(resized, dtype=np.uint8) > 122).astype(np.uint8)

        # flatten
        actualList.append(actualMask.flatten())
        predProbsList.append(pred.flatten())  # Use raw probabilities, rather th

    actual = np.concatenate(actualList)
    predProbs = np.concatenate(predProbsList)

    return predProbs, actual

# raw prediction values and actual labels
predProbs1D, actual1D = flattenPredictions(predictions, mask_test)

# ROC and AUC
fpr, tpr, _ = roc_curve(actual1D, predProbs1D)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()
```



The ROC curve's AUC of 1.00 is likely an artifact of the pixel-level evaluation on a highly imbalanced dataset, where the model's perfect classification of the overwhelming background class dominates the metric. While it suggests excellent separation of individual foreground and background pixels, it does not fully represent the quality of the foreground segmentation.

In [222…

```
# precision recall curve
from sklearn.metrics import precision_recall_curve, average_precision_score

precision, recall, _ = precision_recall_curve(actual1D, predProbs1D)
avg_precision = average_precision_score(actual1D, predProbs1D)

plt.figure(figsize=(6, 5))
plt.plot(recall, precision, color='darkgreen', lw=2, label=f'AP = {avg_precision
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve')
plt.legend(loc='lower left')
plt.grid(True)
plt.tight_layout()
plt.show()
```



the Precision-Recall curve and its Average Precision (AP = 0.66) offer a more truthful and critical evaluation of the FCN's ability to segment the actual "Cells." The AP of 0.66 indicates that the model has decent performance in balancing the identification of true cells with avoiding false positives, but there's a clear trade-off. As recall increases (finding more cells), precision tends to drop (making more mistakes).

U-Net Model Implementation

As mentioned in the introduction, models like U-Net are specifically designed for biomedical image segmentation tasks. Unlike traditional fully convolutional networks (FCNs), U-Net incorporates skip connections that pass feature maps directly from the encoder to the corresponding decoder layers. This architectural design helps retain spatial information that is often lost during downsampling and significantly enhances boundary localization, an essential factor when segmenting fine structures such as neuronal cells.

After implementing an FCN trained with a custom class-weighted cross-entropy loss, we extended our experiment by introducing a U-Net model combined with the Dice loss function. Dice loss is particularly effective in addressing class imbalance—a common issue in biomedical segmentation, where the region of interest (e.g., brain cells) often comprises only a small portion of the image. Moreover, Dice loss pairs well with the U-Net architecture, enhancing its ability to produce precise segmentations in such scenarios. By comparing the performance of both models under similar training conditions, our goal was to determine which combination of architecture and loss function delivers more accurate and reliable segmentation results on our fluorescent neuronal cell dataset.

Initially, we built a more complex and robust U-Net model. However, due to extremely long training times (nearly three hours per epoch), we were forced to simplify the model. Unfortunately, we may have gone too far in this simplification, potentially compromising the model's performance...

In [68]:
```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras import backend as K

# Dice loss
def dice_loss(y_true, y_pred):
    smooth = 1e-6
    y_true_f = tf.reshape(y_true, [-1])
    y_pred_f = tf.reshape(y_pred, [-1])
    intersection = tf.reduce_sum(y_true_f * y_pred_f)
    return 1 - (2. * intersection + smooth) / (tf.reduce_sum(y_true_f) + tf.redu


def dice_coefficient(y_true, y_pred, smooth=1e-6):
    y_true = K.cast(y_true, 'float32')
    y_pred = K.cast(y_pred, 'float32')
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (K.sum(y_true_f) + K.sum(y_pred_f) + s

# Define U-Net model with 400x400 input and 50x50 output
def build_unet_model(input_size=(400, 400, 3), output_size=(50, 50, 1)):
```

```
    inputs = keras.Input(shape=input_size)

    x = layers.Rescaling(1./255)(inputs)

     # Encoder (fewer filters)
    c1 = layers.Conv2D(16, 3, activation='relu', padding='same')(x)
    c1 = layers.Conv2D(16, 3, activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D(2)(c1)

    c2 = layers.Conv2D(32, 3, activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(32, 3, activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D(2)(c2)

    c3 = layers.Conv2D(64, 3, activation='relu', padding='same')(p2)
    c3 = layers.Conv2D(64, 3, activation='relu', padding='same')(c3)

    # Decoder
    u4 = layers.Conv2DTranspose(32, 3, strides=2, padding='same', activation='re
    u4 = layers.concatenate([u4, c2])
    c4 = layers.Conv2D(32, 3, activation='relu', padding='same')(u4)

    u5 = layers.Conv2DTranspose(16, 3, strides=2, padding='same', activation='re
    u5 = layers.concatenate([u5, c1])
    c5 = layers.Conv2D(16, 3, activation='relu', padding='same')(u5)

    d6 = layers.Conv2D(1, 3, activation='sigmoid', padding='same')(c5)
    output = layers.Resizing(50, 50)(d6)

    model = keras.Model(inputs=inputs, outputs=output)
    return model

# Compile model
unet_model = build_unet_model()
unet_model.compile(optimizer='adam', loss=dice_loss, metrics=['accuracy', dice_c
unet_model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 400, 400, 3)	0	-
rescaling (Rescaling)	(None, 400, 400, 3)	0	input_layer[0][0]
conv2d (Conv2D)	(None, 400, 400, 16)	448	rescaling[0][0]
conv2d_1 (Conv2D)	(None, 400, 400, 16)	2,320	conv2d[0][0]
max_pooling2d (MaxPooling2D)	(None, 200, 200, 16)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 200, 200, 32)	4,640	max_pooling2d[0]…
conv2d_3 (Conv2D)	(None, 200, 200, 32)	9,248	conv2d_2[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 100, 100, 32)	0	conv2d_3[0][0]
conv2d_4 (Conv2D)	(None, 100, 100, 64)	18,496	max_pooling2d_1[…
conv2d_5 (Conv2D)	(None, 100, 100, 64)	36,928	conv2d_4[0][0]
conv2d_transpose (Conv2DTranspose)	(None, 200, 200, 32)	18,464	conv2d_5[0][0]
concatenate (Concatenate)	(None, 200, 200, 64)	0	conv2d_transpose… conv2d_3[0][0]
conv2d_6 (Conv2D)	(None, 200, 200, 32)	18,464	concatenate[0][0]
conv2d_transpose_1 (Conv2DTranspose)	(None, 400, 400, 16)	4,624	conv2d_6[0][0]
concatenate_1 (Concatenate)	(None, 400, 400, 32)	0	conv2d_transpose… conv2d_1[0][0]
conv2d_7 (Conv2D)	(None, 400, 400, 16)	4,624	concatenate_1[0]…
conv2d_8 (Conv2D)	(None, 400, 400, 1)	145	conv2d_7[0][0]
resizing (Resizing)	(None, 50, 50, 1)	0	conv2d_8[0][0]

Total params: 118,401 (462.50 KB)

Trainable params: 118,401 (462.50 KB)

Non-trainable params: 0 (0.00 B)

Training

Similar approach as done in FCN.

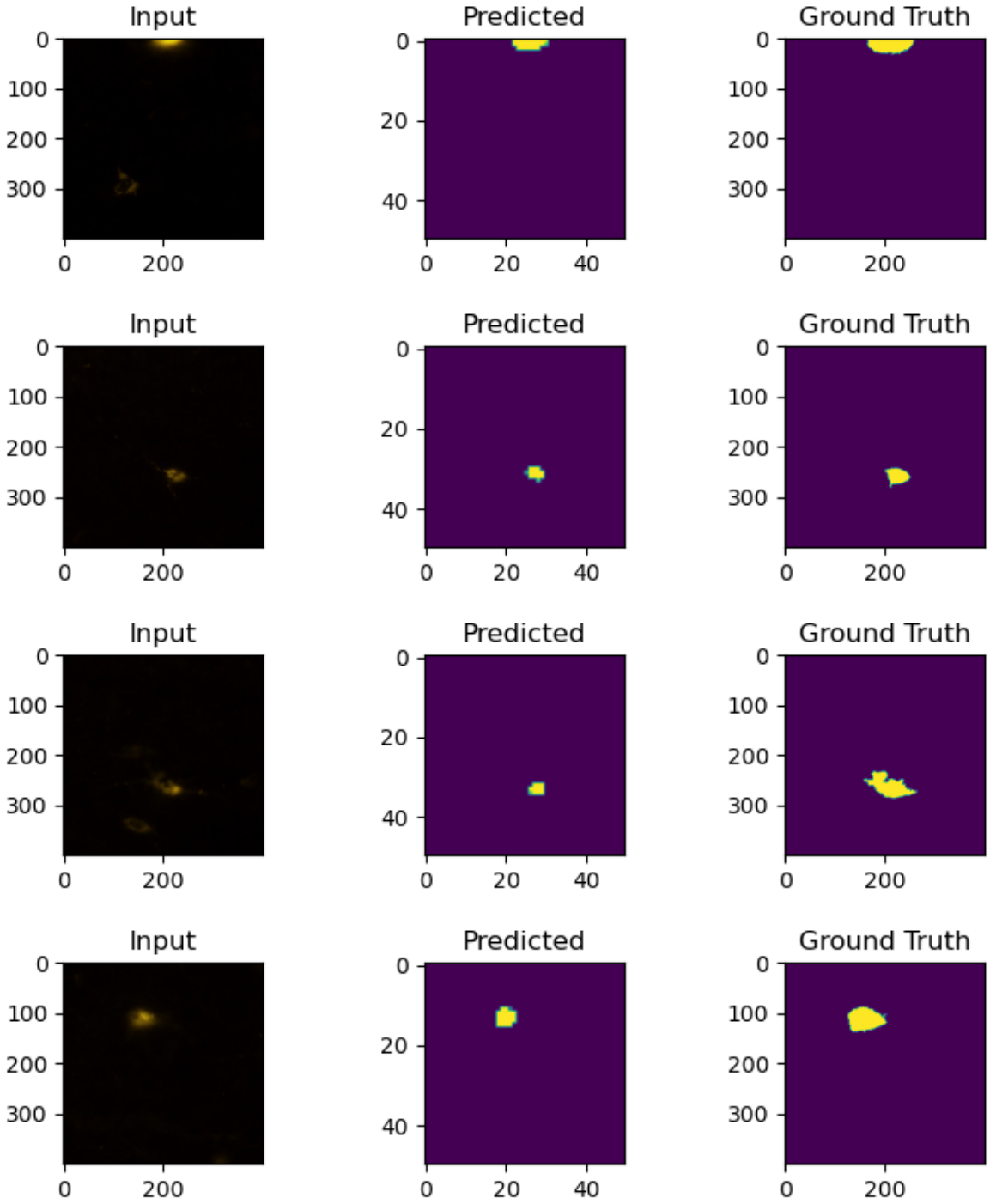
In [63]:
```
# training 1.1

batch_size = 50

unet_generator = ImageSequence(img_train_with_cells, mask_train_with_cells, batc
val_generator = ImageSequence(img_val_with_cells, mask_val_with_cells, batch_siz

history_unet1 = unet_model.fit(
    x=unet_generator,
    epochs=5,
    validation_data=val_generator
)
```

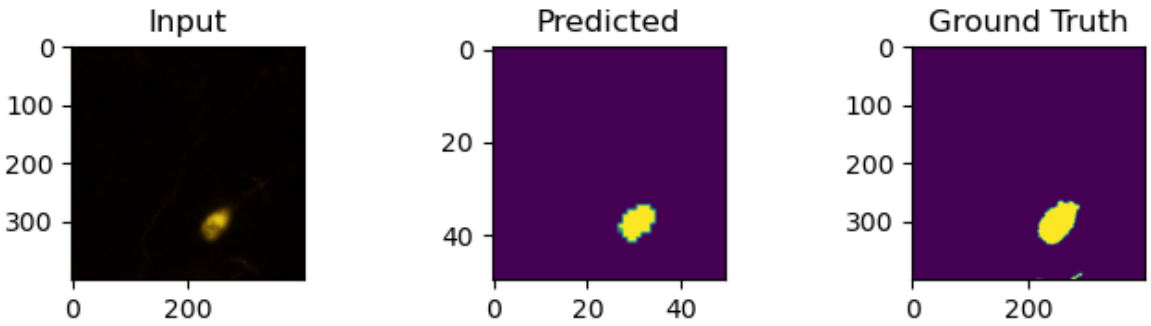
```
C:\Users\gonca\anaconda3\Lib\site-packages\keras\src\trainers\data_adapters\py_da
taset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__
init__(**kwargs)` in its constructor. `**kwargs` can include `workers`, `use_mult
iprocessing`, `max_queue_size`. Do not pass these arguments to `fit()`, as they w
ill be ignored.
  self._warn_if_super_not_called()
Epoch 1/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 240s 7s/step - accuracy: 0.2911 - dice_coefficient: 0.
0478 - loss: 0.9522 - val_accuracy: 0.8096 - val_dice_coefficient: 0.0684 - val_l
oss: 0.9316
Epoch 2/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 205s 7s/step - accuracy: 0.8846 - dice_coefficient: 0.
1181 - loss: 0.8819 - val_accuracy: 0.9786 - val_dice_coefficient: 0.5891 - val_l
oss: 0.4109
Epoch 3/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 219s 8s/step - accuracy: 0.9740 - dice_coefficient: 0.
5129 - loss: 0.4871 - val_accuracy: 0.9809 - val_dice_coefficient: 0.5391 - val_l
oss: 0.4609
Epoch 4/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 234s 8s/step - accuracy: 0.9789 - dice_coefficient: 0.
5279 - loss: 0.4721 - val_accuracy: 0.9815 - val_dice_coefficient: 0.5700 - val_l
oss: 0.4300
Epoch 5/5
29/29 ━━━━━━━━━━━━━━━━━━━━ 204s 7s/step - accuracy: 0.9792 - dice_coefficient: 0.
5439 - loss: 0.4561 - val_accuracy: 0.9783 - val_dice_coefficient: 0.5506 - val_l
oss: 0.4494
```

In [65]:
```
def previewResults():
    testCasesU = np.array([np.array(img) for img in img_train_with_cells[:5]])
    testGroundTruth = mask_train_with_cells[:5]
    predictions = unet_model.predict(testCasesU)

    for img, pred, actual in zip(testCasesU, predictions, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')
```

```
        ax[1].imshow(pred)
        ax[1].set_title('Predicted')

        ax[2].imshow(actual)
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
        plt.show()

previewResults()
```

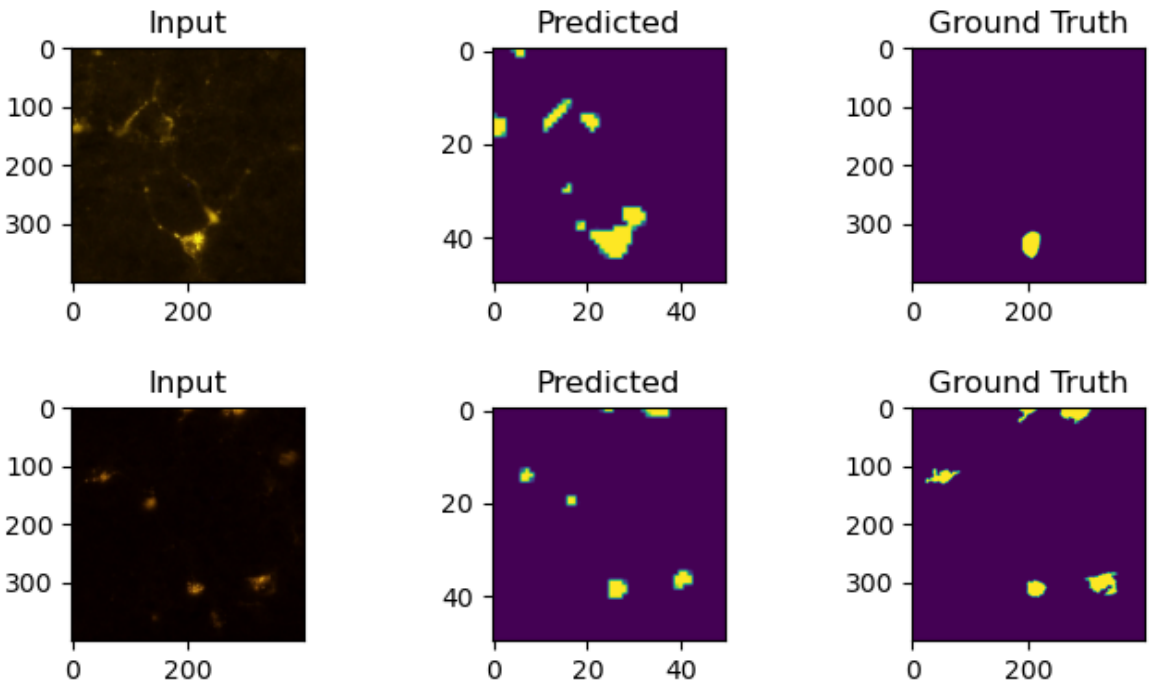
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 756ms/step
```

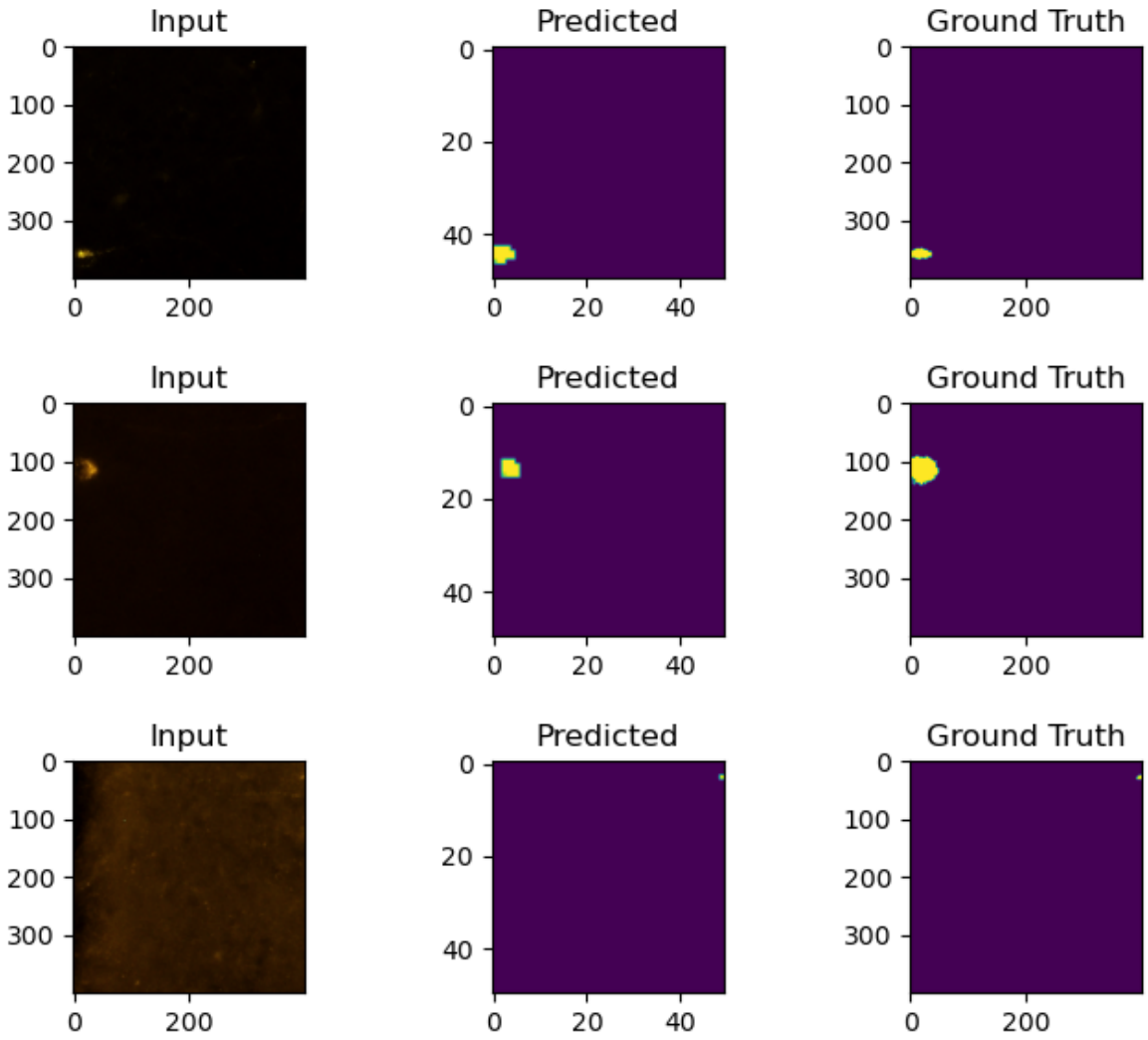




In [66]:
```
def previewResults():
    testCasesU = np.array([np.array(img) for img in img_val_with_cells[:5]])
    testGroundTruth = mask_val_with_cells[:5]
    predictions = unet_model.predict(testCasesU)

    for img, pred, actual in zip(testCasesU, predictions, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')

        ax[1].imshow(pred)
        ax[1].set_title('Predicted')

        ax[2].imshow(actual)
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
        plt.show()

previewResults()
```

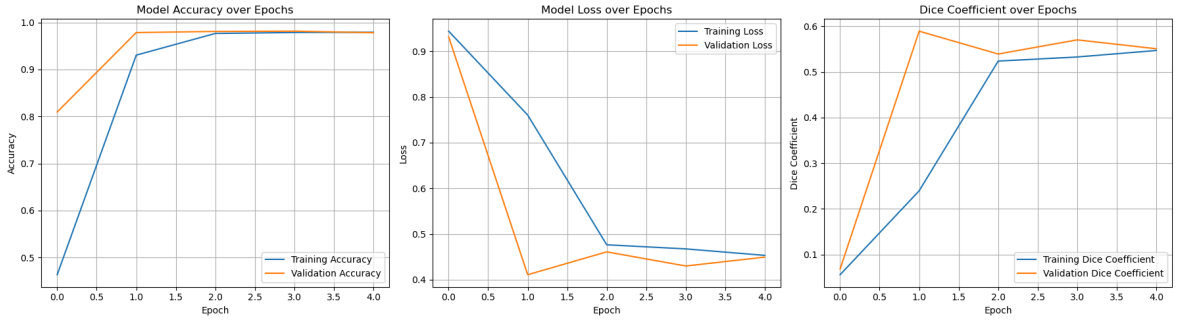
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 256ms/step
```





In [71]:
```
plt.figure(figsize=(18, 5))

# Accuracy
plt.subplot(1, 3, 1)
plt.plot(history_unet1.history['accuracy'], label='Training Accuracy')
plt.plot(history_unet1.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss
plt.subplot(1, 3, 2)
plt.plot(history_unet1.history['loss'], label='Training Loss')
plt.plot(history_unet1.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Dice Coefficient
plt.subplot(1, 3, 3)
plt.plot(history_unet1.history['dice_coefficient'], label='Training Dice Coeffic
plt.plot(history_unet1.history['val_dice_coefficient'], label='Validation Dice C
plt.title('Dice Coefficient over Epochs')
```

```
plt.xlabel('Epoch')
plt.ylabel('Dice Coefficient')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Accuracy: The training and validation accuracy curves are very close, indicating excellent generalization and minimal overfitting within these first few epochs.

Loss: Validation loss drops sharply, showing an interesting dip at epoch 1 before continuing to decrease and stabilize at a very low level, close to the training loss. This indicates the model is quickly learning to minimize errors on both seen and unseen data.

Dice coeff: Both training and validation Dice coefficients rise sharply since the first epoch.

This first training session for U-Net model is highly successful. The model demonstrates extremely fast learning and convergence, achieving high accuracy and a strong Dice coefficient very early on.

In [88]:
```
# training 1.2
unet_generator.enableFeatures(True, False) # enable random shapes

history_unet2 = unet_model.fit(
    x=unet_generator,
    epochs=10,
    validation_data=val_generator)
```

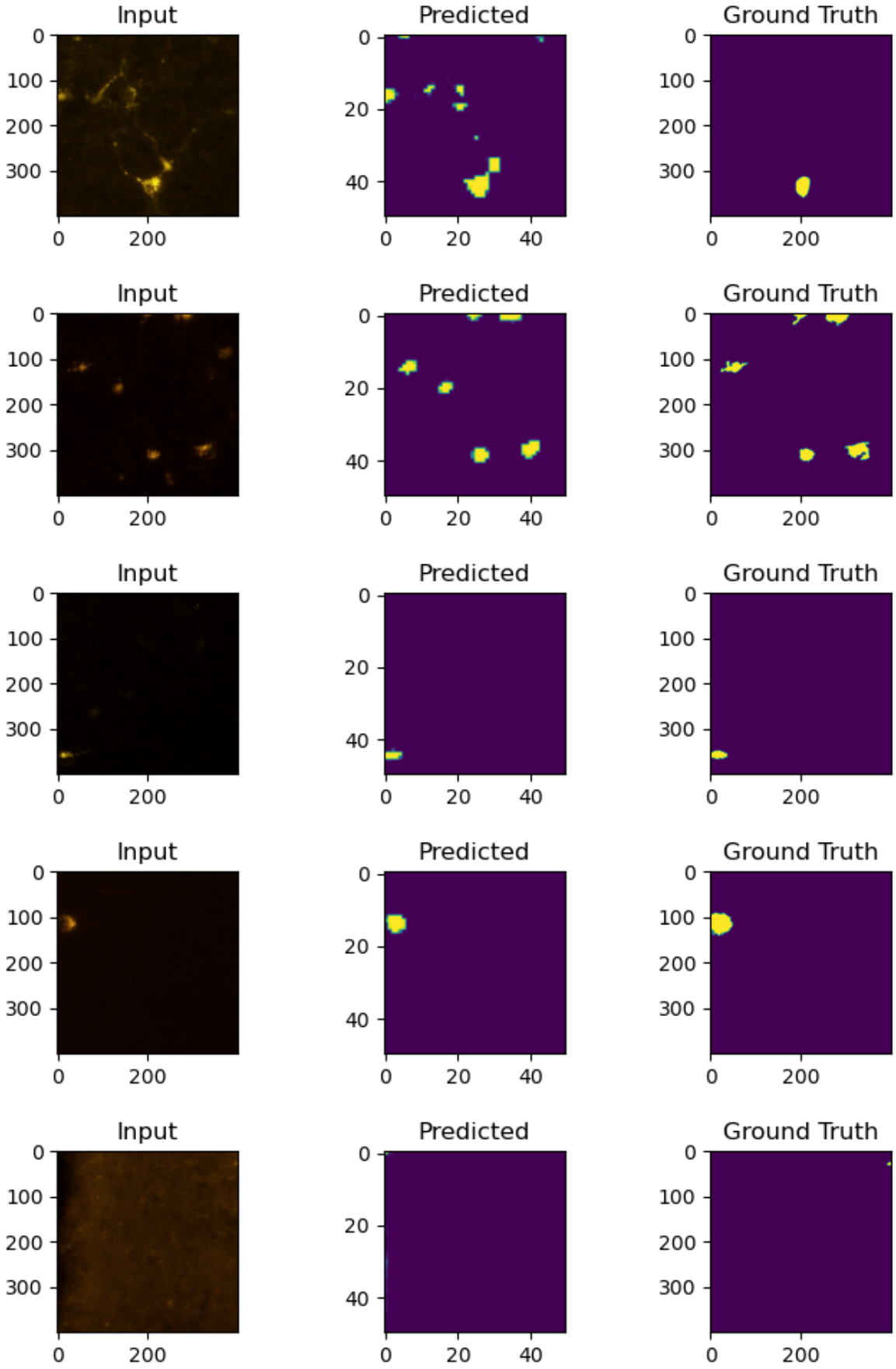
```
Epoch 1/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 205s 7s/step - accuracy: 0.9361 - dice_coefficient: 0.
2901 - loss: 0.7099 - val_accuracy: 0.9821 - val_dice_coefficient: 0.5838 - val_l
oss: 0.4162
Epoch 2/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 204s 7s/step - accuracy: 0.9490 - dice_coefficient: 0.
3252 - loss: 0.6748 - val_accuracy: 0.9836 - val_dice_coefficient: 0.5102 - val_l
oss: 0.4898
Epoch 3/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 204s 7s/step - accuracy: 0.9624 - dice_coefficient: 0.
2949 - loss: 0.7051 - val_accuracy: 0.9762 - val_dice_coefficient: 0.5451 - val_l
oss: 0.4549
Epoch 4/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 184s 6s/step - accuracy: 0.9576 - dice_coefficient: 0.
4030 - loss: 0.5970 - val_accuracy: 0.9826 - val_dice_coefficient: 0.5298 - val_l
oss: 0.4702
Epoch 5/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 177s 6s/step - accuracy: 0.9755 - dice_coefficient: 0.
4824 - loss: 0.5176 - val_accuracy: 0.9839 - val_dice_coefficient: 0.5737 - val_l
oss: 0.4263
Epoch 6/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 183s 6s/step - accuracy: 0.9804 - dice_coefficient: 0.
5480 - loss: 0.4520 - val_accuracy: 0.9818 - val_dice_coefficient: 0.5792 - val_l
oss: 0.4208
Epoch 7/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 186s 6s/step - accuracy: 0.9809 - dice_coefficient: 0.
5510 - loss: 0.4490 - val_accuracy: 0.9834 - val_dice_coefficient: 0.5869 - val_l
oss: 0.4131
Epoch 8/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 196s 7s/step - accuracy: 0.9800 - dice_coefficient: 0.
5600 - loss: 0.4400 - val_accuracy: 0.9843 - val_dice_coefficient: 0.6209 - val_l
oss: 0.3791
Epoch 9/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 182s 6s/step - accuracy: 0.9821 - dice_coefficient: 0.
5879 - loss: 0.4121 - val_accuracy: 0.9843 - val_dice_coefficient: 0.6237 - val_l
oss: 0.3763
Epoch 10/10
29/29 ━━━━━━━━━━━━━━━━━━━━ 181s 6s/step - accuracy: 0.9827 - dice_coefficient: 0.
6009 - loss: 0.3991 - val_accuracy: 0.9859 - val_dice_coefficient: 0.6298 - val_l
oss: 0.3702
```

In [134…

```python
# second training images
def previewResults():
    testCasesU = np.array([np.array(img) for img in img_val_with_cells[:5]])
    testGroundTruth = mask_val_with_cells[:5]
    predictions = unet_model.predict(testCasesU)

    for img, pred, actual in zip(testCasesU, predictions, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')

        ax[1].imshow(pred)
        ax[1].set_title('Predicted')

        ax[2].imshow(actual)
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
```

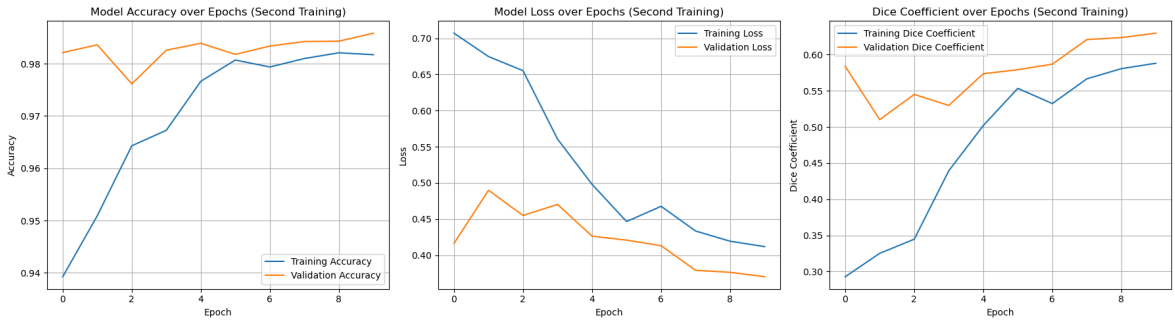
```
        plt.show()

previewResults()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 355ms/step
```



In [90]:
```
# second training
plt.figure(figsize=(18, 5))  # 3 wide subplots
```

```
# Accuracy
plt.subplot(1, 3, 1)
plt.plot(history_unet2.history['accuracy'], label='Training Accuracy')
plt.plot(history_unet2.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs (Second Training)')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss
plt.subplot(1, 3, 2)
plt.plot(history_unet2.history['loss'], label='Training Loss')
plt.plot(history_unet2.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs (Second Training)')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Dice Coefficient
plt.subplot(1, 3, 3)
plt.plot(history_unet2.history['dice_coefficient'], label='Training Dice Coeffic
plt.plot(history_unet2.history['val_dice_coefficient'], label='Validation Dice C
plt.title('Dice Coefficient over Epochs (Second Training)')
plt.xlabel('Epoch')
plt.ylabel('Dice Coefficient')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Accuracy: Validation accuracy is generally higher and fluctuates more, suggesting good generalization but perhaps some sensitivity to the validation set's characteristics.

Loss: The validation loss is consistently lower than the training loss, which is a positive sign for generalization.

Dice coeff: The validation Dice coefficient is notably higher than the training Dice coefficient, reinforcing the observation of good generalization on the validation set.

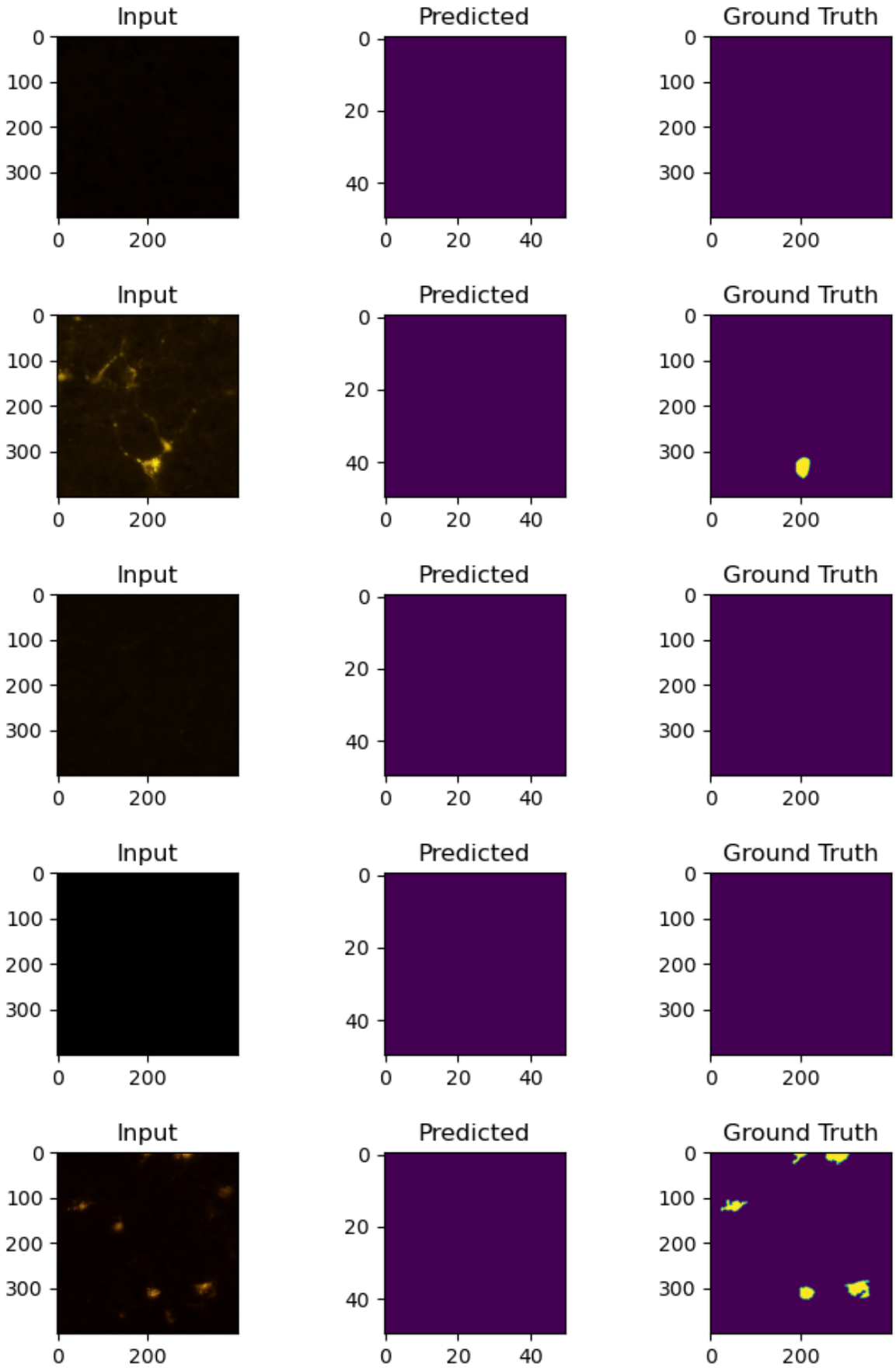
The U-Net's second session shows strong learning, with validation metrics matching or exceeding training (good generalization). No signs of overfitting—further training may still improve the training Dice score. Results are promising...

In [74]:
```
# last training

unet_generator = ImageSequence(img_train, mask_train, batch_size, finalMaskSize)
unet_generator.enableFeatures(True, True) #Enable Random shapes + transforms
val_generator = ImageSequence(img_val, mask_val, batch_size, finalMaskSize)

history_unet3 = unet_model.fit(
    x=unet_generator,
    epochs=5,
    validation_data=val_generator)
```

```
Epoch 1/5
905/905 ━━━━━━━━━━━━━━━━━━━━ 619s 684ms/step - accuracy: 0.9927 - dice_coefficien
t: 0.1596 - loss: 0.8404 - val_accuracy: 0.9936 - val_dice_coefficient: 0.1593 -
val_loss: 0.8407
Epoch 2/5
905/905 ━━━━━━━━━━━━━━━━━━━━ 628s 694ms/step - accuracy: 0.9933 - dice_coefficien
t: 0.1637 - loss: 0.8363 - val_accuracy: 0.9936 - val_dice_coefficient: 0.1062 -
val_loss: 0.8938
Epoch 3/5
905/905 ━━━━━━━━━━━━━━━━━━━━ 623s 688ms/step - accuracy: 0.9930 - dice_coefficien
t: 0.1616 - loss: 0.8384 - val_accuracy: 0.9936 - val_dice_coefficient: 0.0885 -
val_loss: 0.9115
Epoch 4/5
905/905 ━━━━━━━━━━━━━━━━━━━━ 607s 671ms/step - accuracy: 0.9926 - dice_coefficien
t: 0.1565 - loss: 0.8435 - val_accuracy: 0.9936 - val_dice_coefficient: 0.1593 -
val_loss: 0.8407
Epoch 5/5
905/905 ━━━━━━━━━━━━━━━━━━━━ 632s 698ms/step - accuracy: 0.9935 - dice_coefficien
t: 0.1752 - loss: 0.8248 - val_accuracy: 0.9936 - val_dice_coefficient: 0.1504 -
val_loss: 0.8496
```

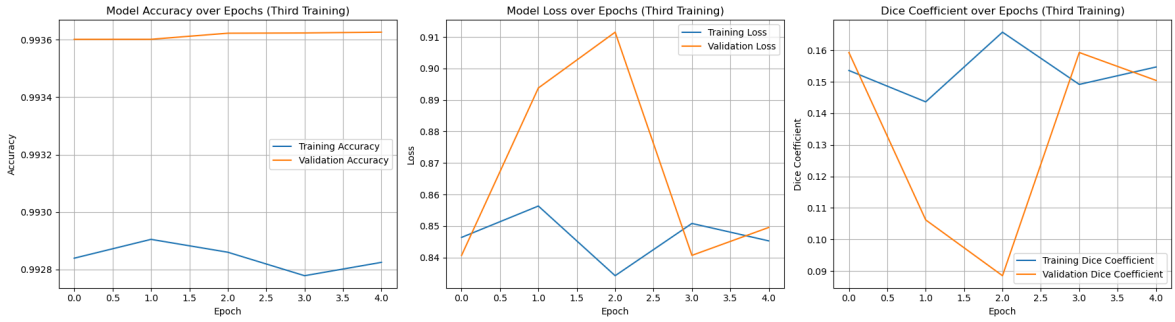
In [80]:
```
def previewResults():
    testCasesU = np.array([np.array(img) for img in img_val[:5]])
    testGroundTruth = mask_val[:5]
    predictions5 = unet_model.predict(testCasesU)

    for img, pred, actual in zip(testCasesU, predictions5, testGroundTruth):
        fig, ax = plt.subplots(1, 3, figsize=(7,2))

        ax[0].imshow(img)
        ax[0].set_title('Input')

        ax[1].imshow(pred)
        ax[1].set_title('Predicted')

        ax[2].imshow(actual)
        ax[2].set_title('Ground Truth')

        plt.tight_layout()
        plt.show()

previewResults()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 224ms/step
```



In [78]:
```
# plots last training

plt.figure(figsize=(18, 5))

# Accuracy
plt.subplot(1, 3, 1)
```

```
plt.plot(history_unet3.history['accuracy'], label='Training Accuracy')
plt.plot(history_unet3.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs (Third Training)')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# Loss
plt.subplot(1, 3, 2)
plt.plot(history_unet3.history['loss'], label='Training Loss')
plt.plot(history_unet3.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs (Third Training)')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Dice Coefficient
plt.subplot(1, 3, 3)
plt.plot(history_unet3.history['dice_coefficient'], label='Training Dice Coeffic
plt.plot(history_unet3.history['val_dice_coefficient'], label='Validation Dice C
plt.title('Dice Coefficient over Epochs (Third Training)')
plt.xlabel('Epoch')
plt.ylabel('Dice Coefficient')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



Accuracy: When we introduce images without cells, the background becomes the dominant class. A model can achieve very high accuracy simply by predicting "background" for almost all pixels. This high accuracy can be misleading if our primary interest is segmenting the foreground (cells).

Loss: The high and fluctuating loss values are strong indicators of a problem. The increase in loss, especially the validation loss, directly contradicts effective learning.

Dice coeff: The Dice coefficient is the most appropriate metric for imbalance segmentation tasks. A low Dice coefficient, particularly a validation Dice coefficient that is low and erratic, strongly indicates that the model is failing to accurately segment the foreground (cells).

Based on the plots and the visual examples, the "last training" session for U-Net model was unsuccessful in learning to segment cells, despite achieving high accuracy. This is

primarily due to the introduction of a large number of images without cells, which created an extreme class imbalance. The model has learned a "null" prediction strategy, overwhelmingly predicting background for all pixels to minimize overall loss, thus effectively ignoring and failing to detect any foreground cells. This is clearly evidenced by the very low and erratic Dice coefficient, the increasing loss values, and the visual confirmation of the model predicting solid background even when cells are present in the input image.

Model Evaluation

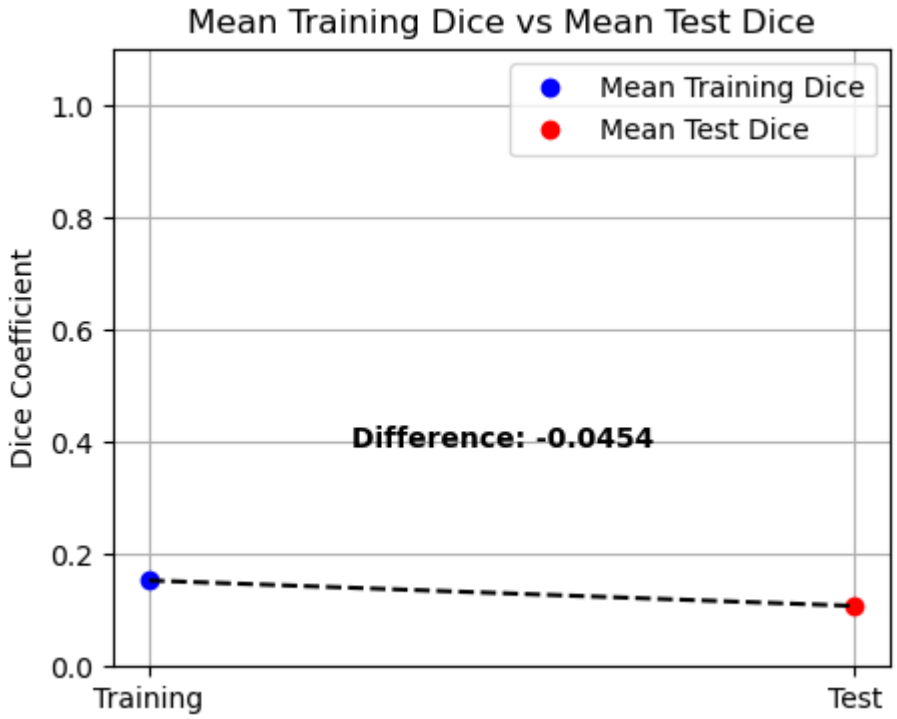
In [289…
```
# Convert list to NumPy array
img_test_array = np.array(img_test).astype(np.float32) / 255.0


print(img_test_array.shape)

# Predict
preds_unet = unet_model.predict(img_test_array)
```

```
(566, 400, 400, 3)
18/18 ━━━━━━━━━━━━━━━━━━━━ 18s 922ms/step
```

In [309…
```
# Binarize predictions
preds_unet_binary = (preds_unet > 0.5).astype(np.uint8)

# Compute Dice scores
dice_scores_unet = [dice_coef_np(y_t, y_p) for y_t, y_p in zip(mask_test_binary,

# Mean Dice
mean_dice_test_unet = np.mean(dice_scores_unet)
```

In [311…
```
# training history
train_diceu = history_unet3.history['dice_coefficient']

# Mean Dice coefficient during training (average over all epochs)
mean_dice_trainu = np.mean(train_diceu)
mean_dice_testu = mean_dice_test_unet

plt.figure(figsize=(5,4))

# Plot points
plt.plot([0], [mean_dice_trainu], 'bo', label='Mean Training Dice')
plt.plot([1], [mean_dice_testu], 'ro', label='Mean Test Dice')

# Draw line connecting the points
plt.plot([0, 1], [mean_dice_trainu, mean_dice_testu], 'k--')

# Calculate difference and plot as text
diff = mean_dice_testu - mean_dice_trainu
mid_x = 0.5
mid_y = (mean_dice_train + mean_dice_test) / 2
plt.text(mid_x, mid_y + 0.03, f'Difference: {diff:.4f}', ha='center', fontsize=1

plt.xticks([0, 1], ['Training', 'Test'])
plt.ylabel('Dice Coefficient')
plt.title('Mean Training Dice vs Mean Test Dice')
```

```
plt.legend()
plt.ylim(0, 1.1)
plt.grid(True)
plt.show()
```



The U-Net model, based on this evaluation, is currently performing very poorly at image segmentation, achieving very low Dice coefficients on both training and test sets. This severe degradation in performance, especially when contrasted with the promising results of our initial U-Net training (where Dice coefficients were much higher), strongly points to a fundamental issue introduced in subsequent training runs, likely the one where we began using "full images, not only those with cells."

As discussed previously, including a vast number of images without cells introduces an extreme class imbalance, causing the U-Net model to become biased towards predicting the dominant "Non-Cells" (background) class. The previous visual examples of the U-Net predicting a solid purple (background) mask even when cells were present directly supports this conclusion. The low Dice coefficient is a direct consequence of this "null" prediction strategy, where the model fails to capture the minority foreground class.

In [226…

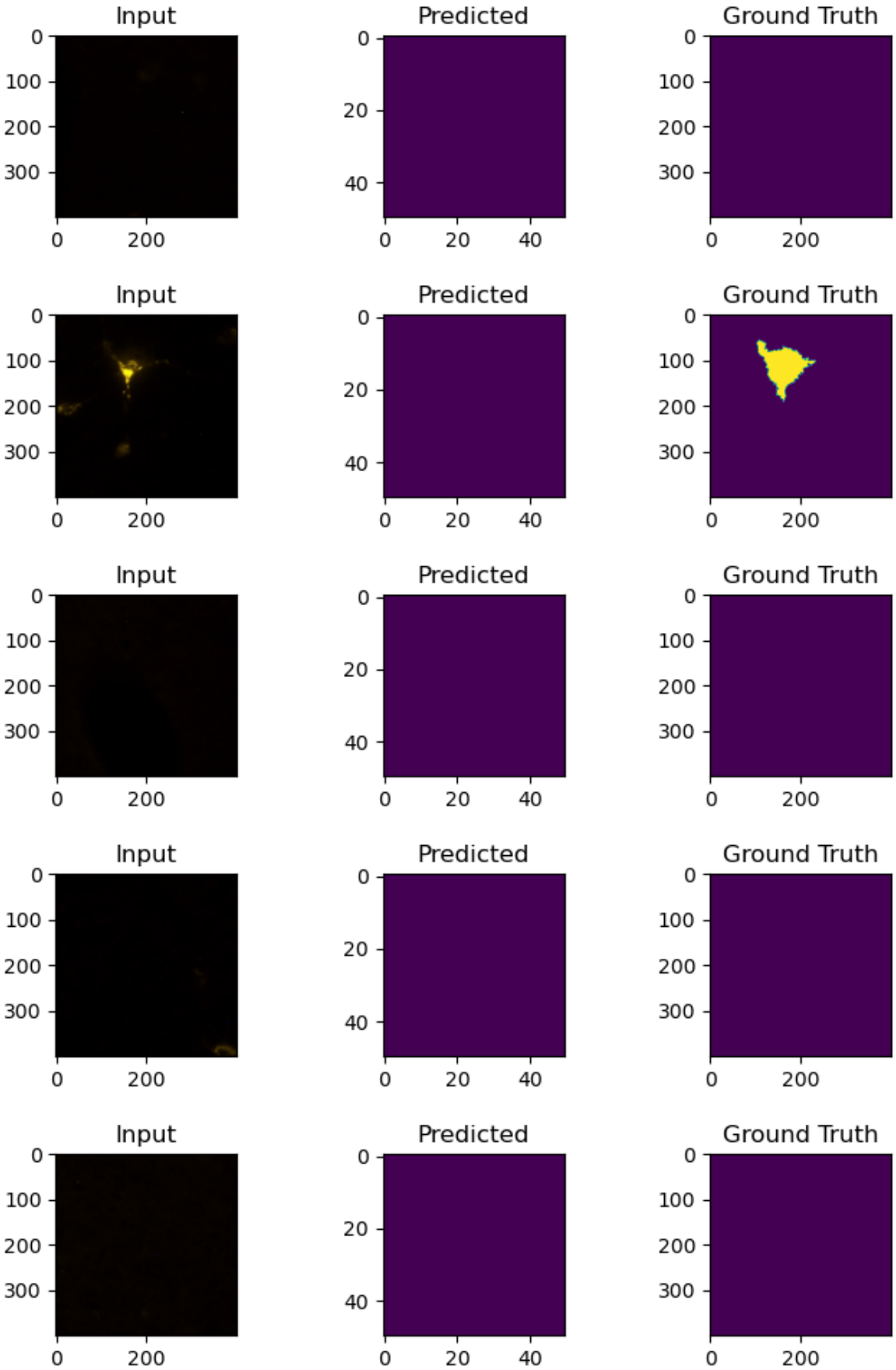
```
#img_test, mask_test (with cells only)
predictionsunet = unet_model.predict(np.array([np.array(img) for img in img_test
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 18s 973ms/step
```

In [228…

```
# model evaluation

# prediction with test dataset
import itertools

for img, pred, actual in itertools.islice(zip(img_test, predictionsunet, mask_te
    fig, ax = plt.subplots(1, 3, figsize=(7,2))
    ax[0].imshow(img)
    ax[0].set_title('Input')
```

```
    ax[1].imshow(pred)
    ax[1].set_title('Predicted')
    ax[2].imshow(actual)
    ax[2].set_title('Ground Truth')
    plt.tight_layout()
    plt.show()
```

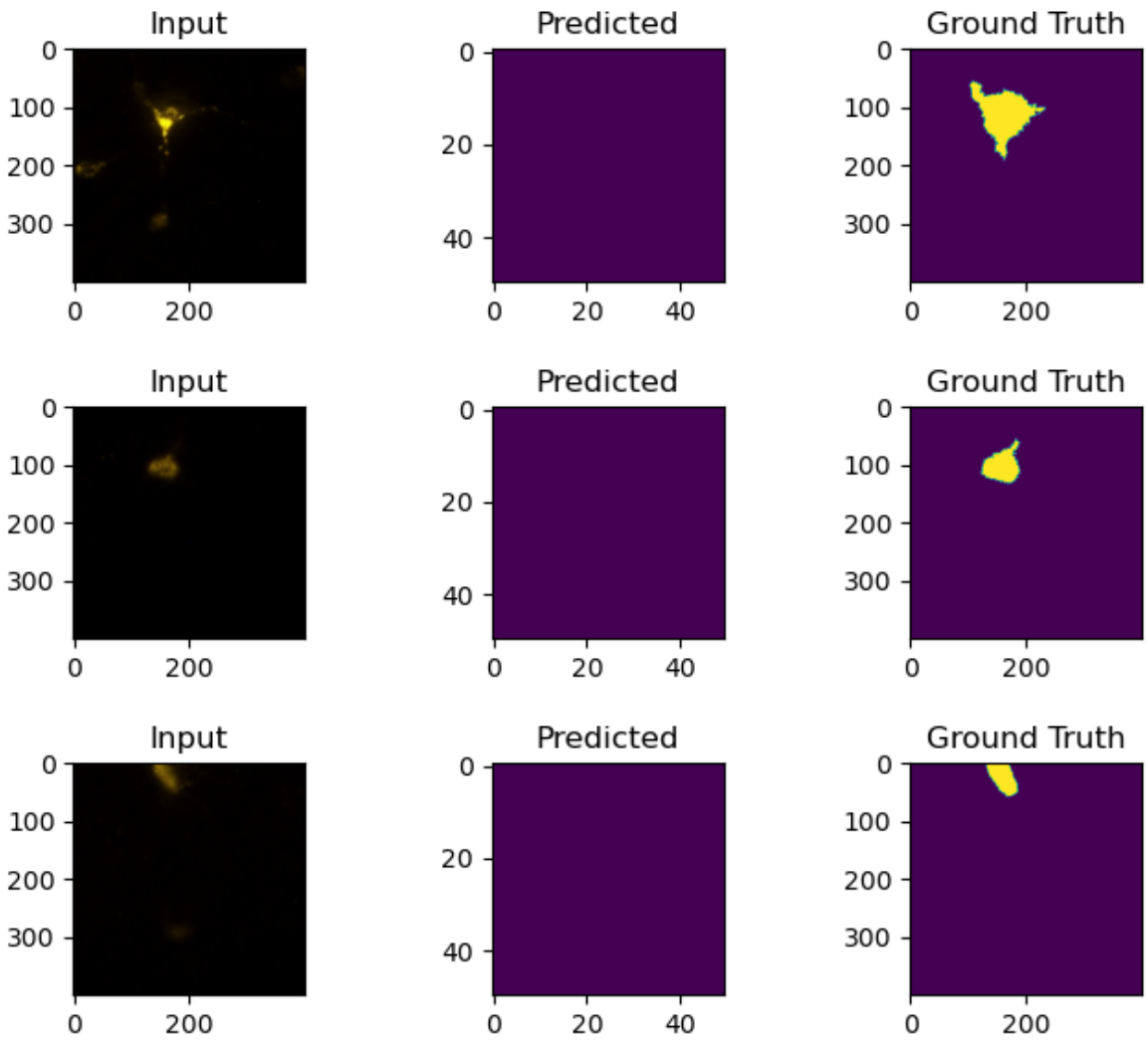


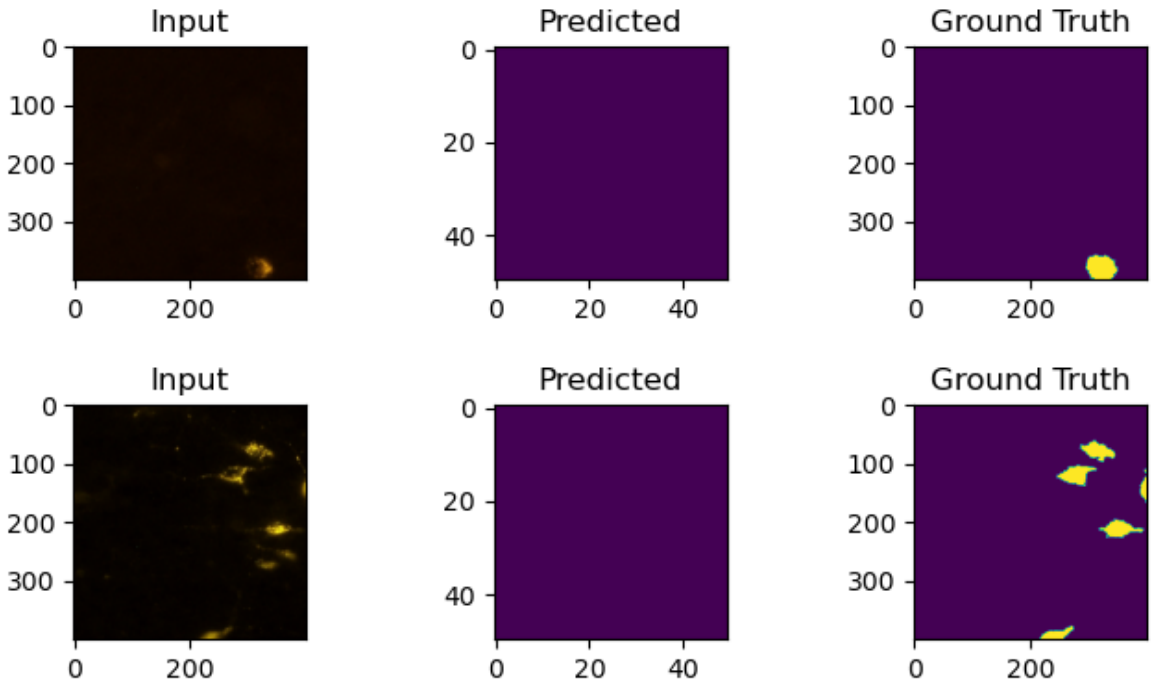
The U-Net model, in its current state after the training run that included full images (many without cells), has learned to primarily predict "background" regardless of whether cells are actually present. It prioritizes minimizing errors on the vast majority of background pixels, effectively ignoring the minority "Cells" class. This strategy leads to high overall accuracy but renders the model useless for the actual task of segmenting cells.

In [230…
```
predictionsunet2 = unet_model.predict(np.array([np.array(img) for img in img_tes
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 6s 885ms/step
```

In [232…
```
for img, pred, actual in itertools.islice(zip(img_test_with_cells, predictionsun
    fig, ax = plt.subplots(1, 3, figsize=(7,2))
    ax[0].imshow(img)
    ax[0].set_title('Input')

    ax[1].imshow(pred)
    ax[1].set_title('Predicted')
    ax[2].imshow(actual)
    ax[2].set_title('Ground Truth')
    plt.tight_layout()
    plt.show()
```





same showing of the awful results of U-net model.

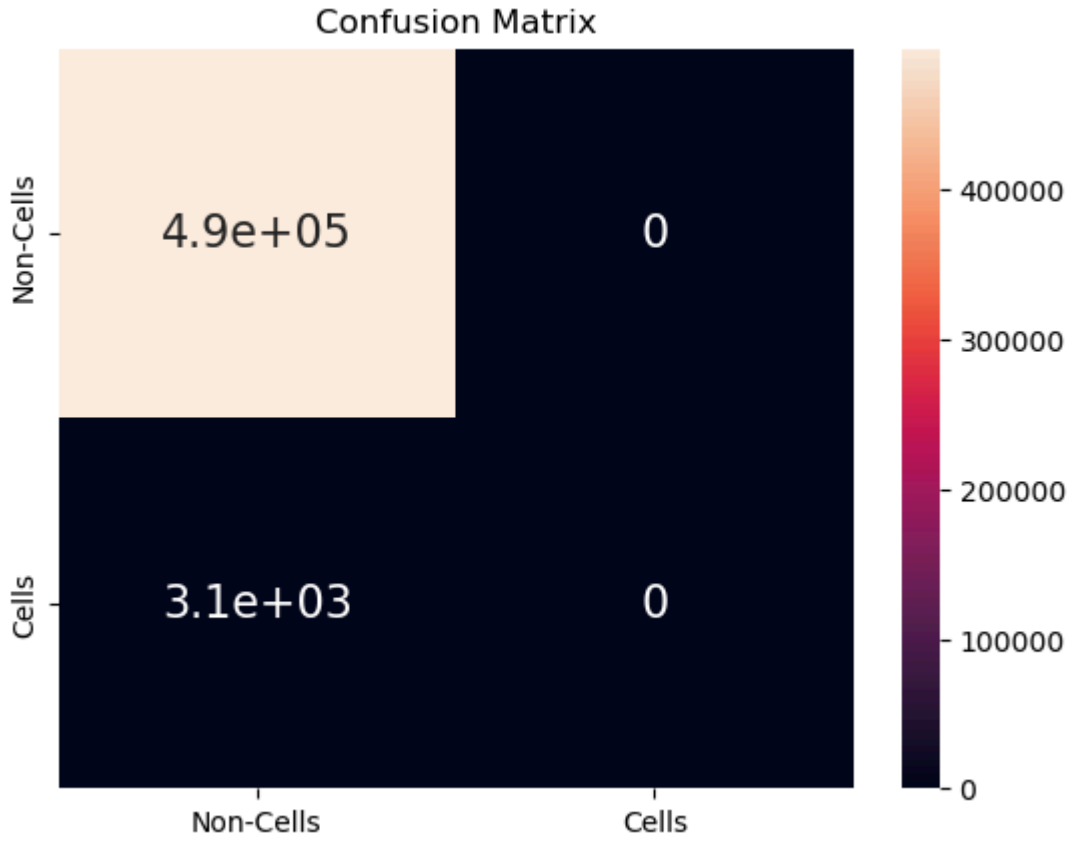
In [244…

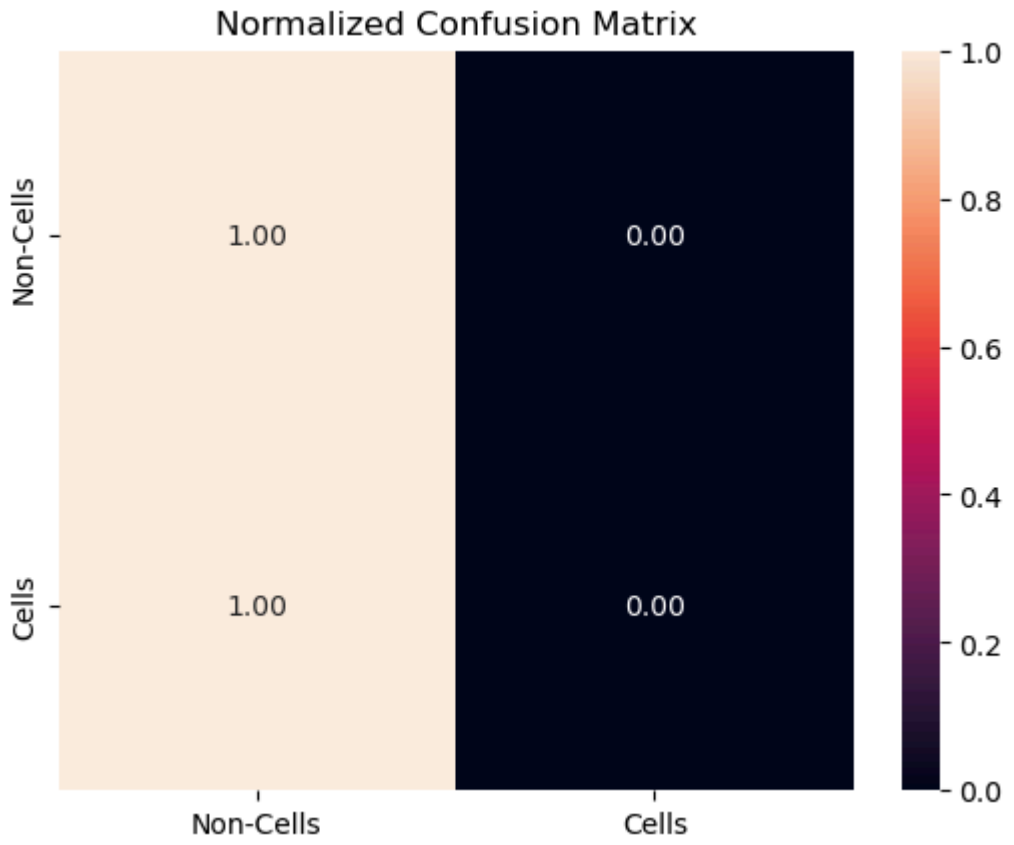
```
def computeMetrics(pred, groundTruthImages):
    actualList = []
    predList = []
    for p, groundTruth in zip(pred, groundTruthImages):
        resized = groundTruth.resize((finalMaskSize, finalMaskSize), Image.BILIN
        resizedThreshold = np.array(resized, dtype=np.uint8) > 122  # 0-255
        predThreshold = (p > 0.5).astype(np.uint8)  # 0-1
        actualList.append(np.reshape(resizedThreshold, -1))
        predList.append(np.reshape(predThreshold, -1))
    actual = np.concatenate((actualList), axis=0)
    predictions = np.concatenate((predList), axis=0)
    return (predictions, actual)

# Use U-Net predictions
pred1Du, actual1Du = computeMetrics(predictionsunet2, mask_test)

# Classification report and confusion matrix
labels = ["Non-Cells", "Cells"]
print(classification_report(actual1Du, pred1Du, target_names=labels))

confusionMatrixu = confusion_matrix(actual1Du, pred1Du)
df_cm = pd.DataFrame(confusionMatrixu, index=labels, columns=labels)
sn.heatmap(df_cm, annot=True, annot_kws={"size": 16}).set_title("Confusion Matri
plt.show()

confMatrixNormalizedu = confusionMatrixu.astype('float') / confusionMatrixu.sum(
df_cm_norm = pd.DataFrame(confMatrixNormalizedu, index=labels, columns=labels)
sn.heatmap(df_cm_norm, annot=True, fmt=".2f")
plt.title("Normalized Confusion Matrix")
plt.show()
```

```
C:\Users\gonca\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:153
1: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in label
s with no predicted samples. Use `zero_division` parameter to control this behavi
or.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
C:\Users\gonca\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:153
1: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in label
s with no predicted samples. Use `zero_division` parameter to control this behavi
or.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

	precision	recall	f1-score	support
Non-Cells	0.99	1.00	1.00	494406
Cells	0.00	0.00	0.00	3094
accuracy			0.99	497500
macro avg	0.50	0.50	0.50	497500
weighted avg	0.99	0.99	0.99	497500

```
C:\Users\gonca\anaconda3\Lib\site-packages\sklearn\metrics\_classification.py:153
1: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in label
s with no predicted samples. Use `zero_division` parameter to control this behavi
or.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```





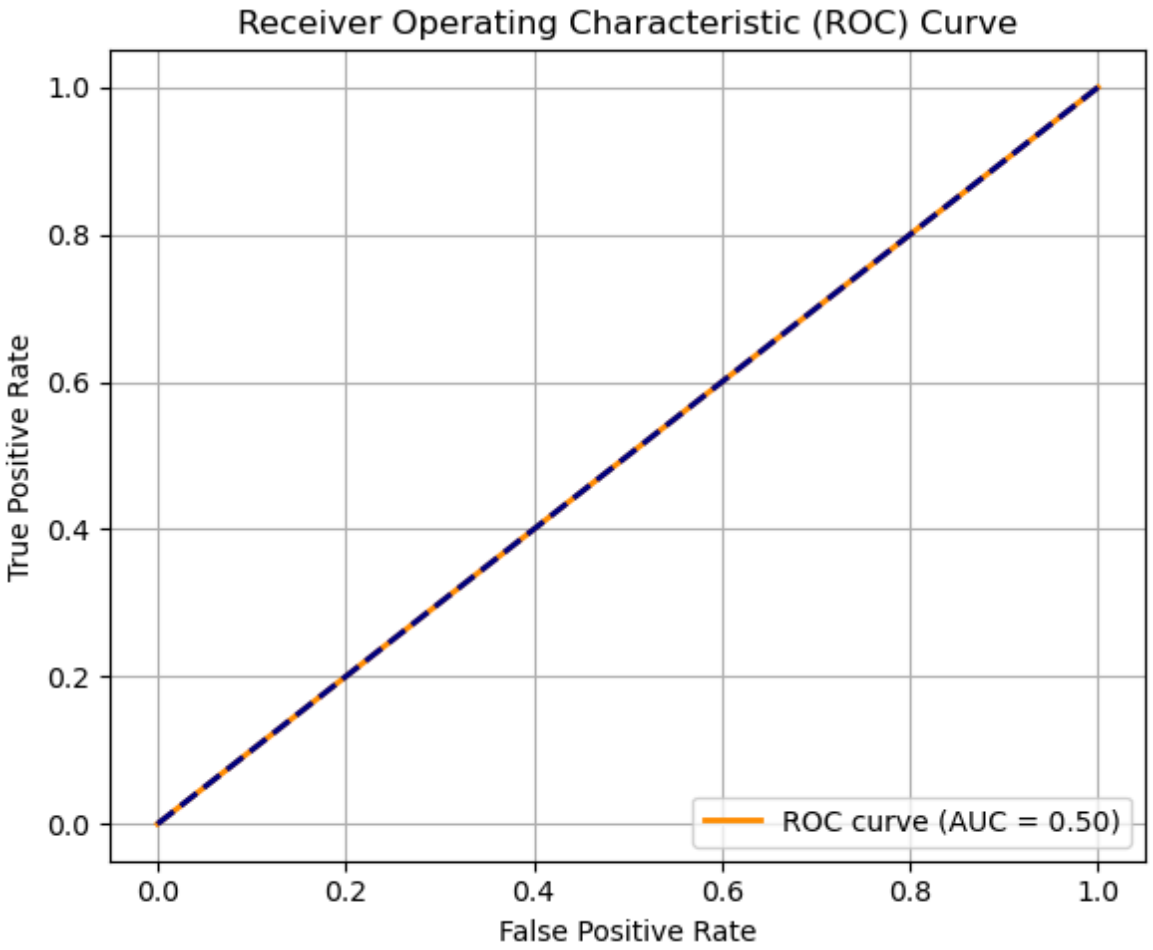
This evaluation confirms the U-Net model is completely failing to perform the image segmentation task of identifying cells. While it achieves a misleadingly high overall accuracy (0.99) and perfect performance on the "Non-Cells" (background) class, it has zero precision, recall, and F1-score for the "Cells" class. This means the model is effectively predicting every single pixel as "Non-Cells" (background). It's unable to differentiate cells from the background whatsoever, rendering it useless for the intended purpose.

This severe issue is a direct consequence of the extreme class imbalance introduced by using "full images" that often contain no cells. The model has learned the simplest way to minimize its overall pixel-wise loss and maximize accuracy: predict the dominant background class for everything.

In [236…

```
from sklearn.metrics import roc_curve, auc
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def flattenPredictions_unet(predictionsunet2, groundTruthImages):
    actualList = []
    predProbsList = []
    for pred, groundTruth in zip(predictionsunet2, groundTruthImages):
        # Resize ground truth to match prediction shape
        resized = groundTruth.resize((finalMaskSize, finalMaskSize), Image.BILIN
        actualMask = (np.array(resized, dtype=np.uint8) > 122).astype(np.uint8)

        actualList.append(actualMask.flatten())
        predProbsList.append(pred.flatten())  # Use raw probabilities

    actual = np.concatenate(actualList)
```

```
    predProbs = np.concatenate(predProbsList)
    return predProbs, actual

# Compute predictions vs ground truth
predProbs1Du, actual1Du = flattenPredictions_unet(predictionsunet2, mask_test)

# Compute ROC and AUC
fpr, tpr, _ = roc_curve(actual1Du, predProbs1Du)
roc_auc = auc(fpr, tpr)

# Plot ROC curve
plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()
```

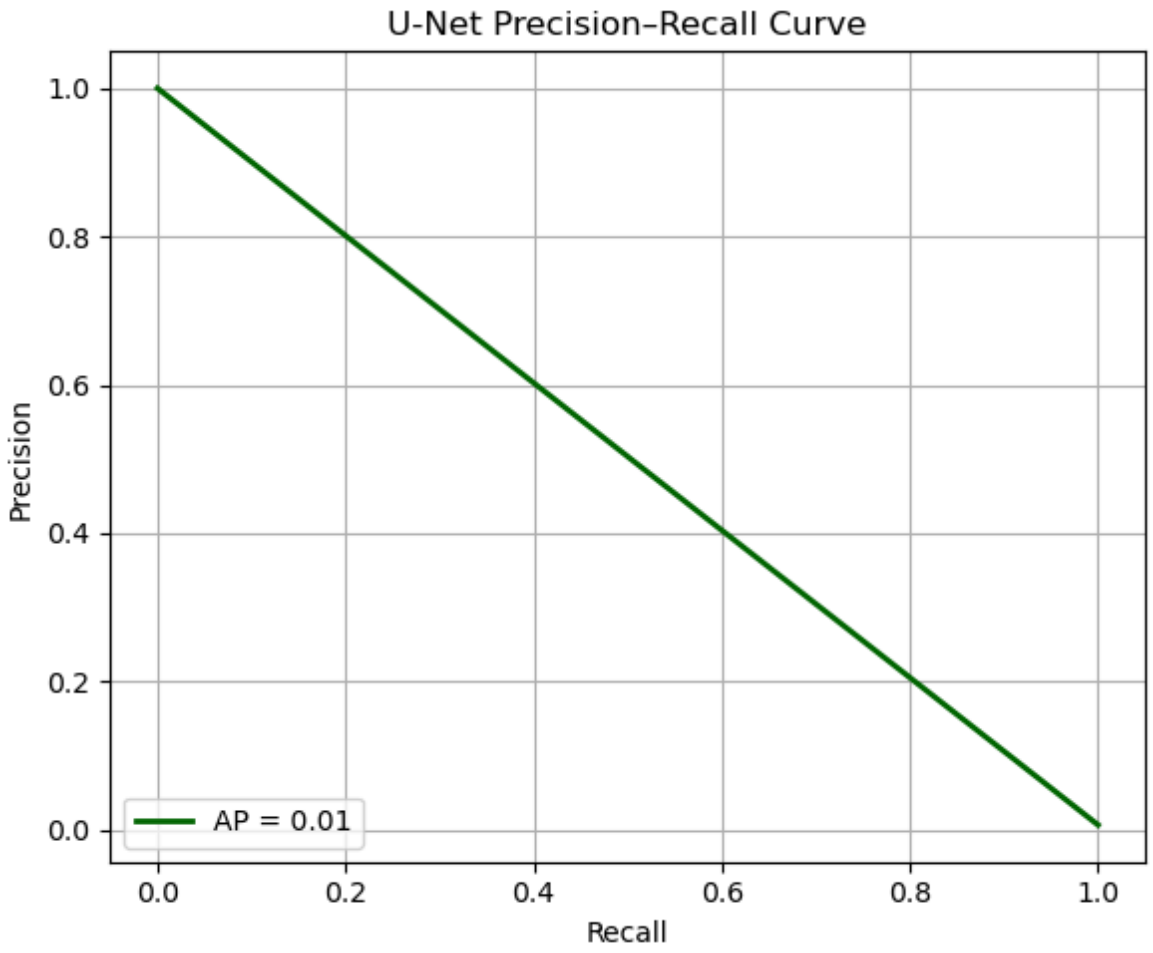


An AUC of 0.50 is the worst possible score for a binary classifier, indicating that the model performs no better than randomly guessing the class of a pixel. In the context of our U-Net model, this directly confirms its complete inability to distinguish between "Cells" and "Non-Cells" at a pixel level. >.>

In [248…

```
from sklearn.metrics import precision_recall_curve, average_precision_score
import matplotlib.pyplot as plt
```

```
# predProbs1Du and actual1Du were produced by:
# predProbs1Du, actual1Du = flattenPredictions_unet(predictionsunet2, mask_test)

precision, recall, _ = precision_recall_curve(actual1Du, predProbs1Du)
avg_precisionu = average_precision_score(actual1Du, predProbs1Du)

plt.figure(figsize=(6, 5))
plt.plot(recall, precision, color='darkgreen', lw=2,
         label=f'AP = {avg_precisionu:.2f}')
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('U-Net Precision-Recall Curve')
plt.legend(loc='lower left')
plt.grid(True)
plt.tight_layout()
plt.show()
```



An Average Precision (AP) of or 1% is indicative of a model that fails to find most positive instances and, when it does predict positives, is overwhelmingly incorrect.

In [299…

```
print("Positive pixel ratio:", np.mean(actual1Du))
```

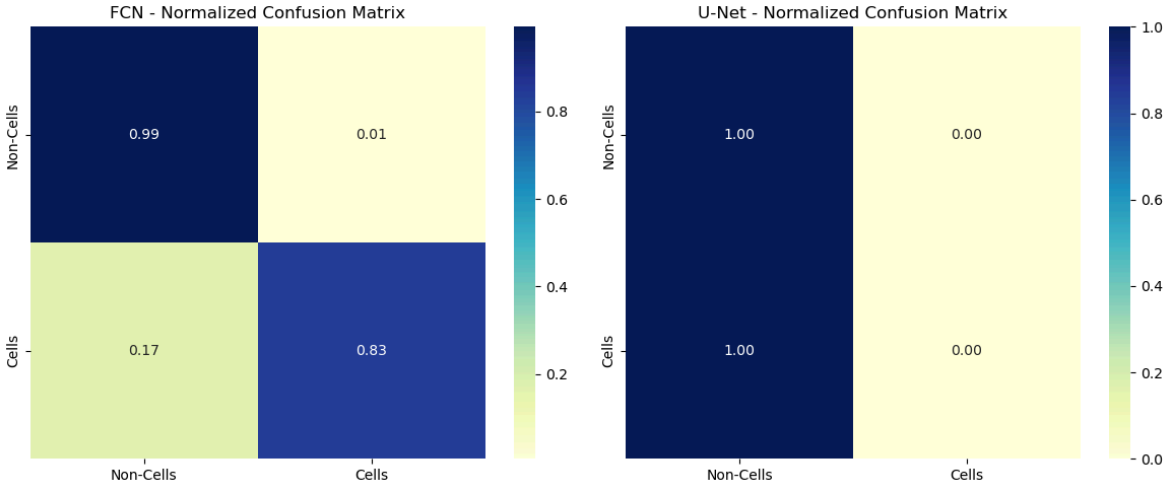
```
Positive pixel ratio: 0.006219095477386935
```

Comparison between models

comparison between the two models: dice coefficient, accuracy, loss, recall, matrix, roc, application to images

In [254…

```
# --- Plot Normalized Confusion Matrices Side-by-Side ---
fig, ax = plt.subplots(1, 2, figsize=(12, 5))

df_cm_norm_fcn = pd.DataFrame(confMatrixNormalized, index=labels, columns=labels
df_cm_norm_unet = pd.DataFrame(confMatrixNormalizedu, index=labels, columns=labe

sn.heatmap(df_cm_norm_fcn, annot=True, fmt=".2f", ax=ax[0], cmap="YlGnBu")
ax[0].set_title("FCN - Normalized Confusion Matrix")

sn.heatmap(df_cm_norm_unet, annot=True, fmt=".2f", ax=ax[1], cmap="YlGnBu")
ax[1].set_title("U-Net - Normalized Confusion Matrix")

plt.tight_layout()
plt.show()
```

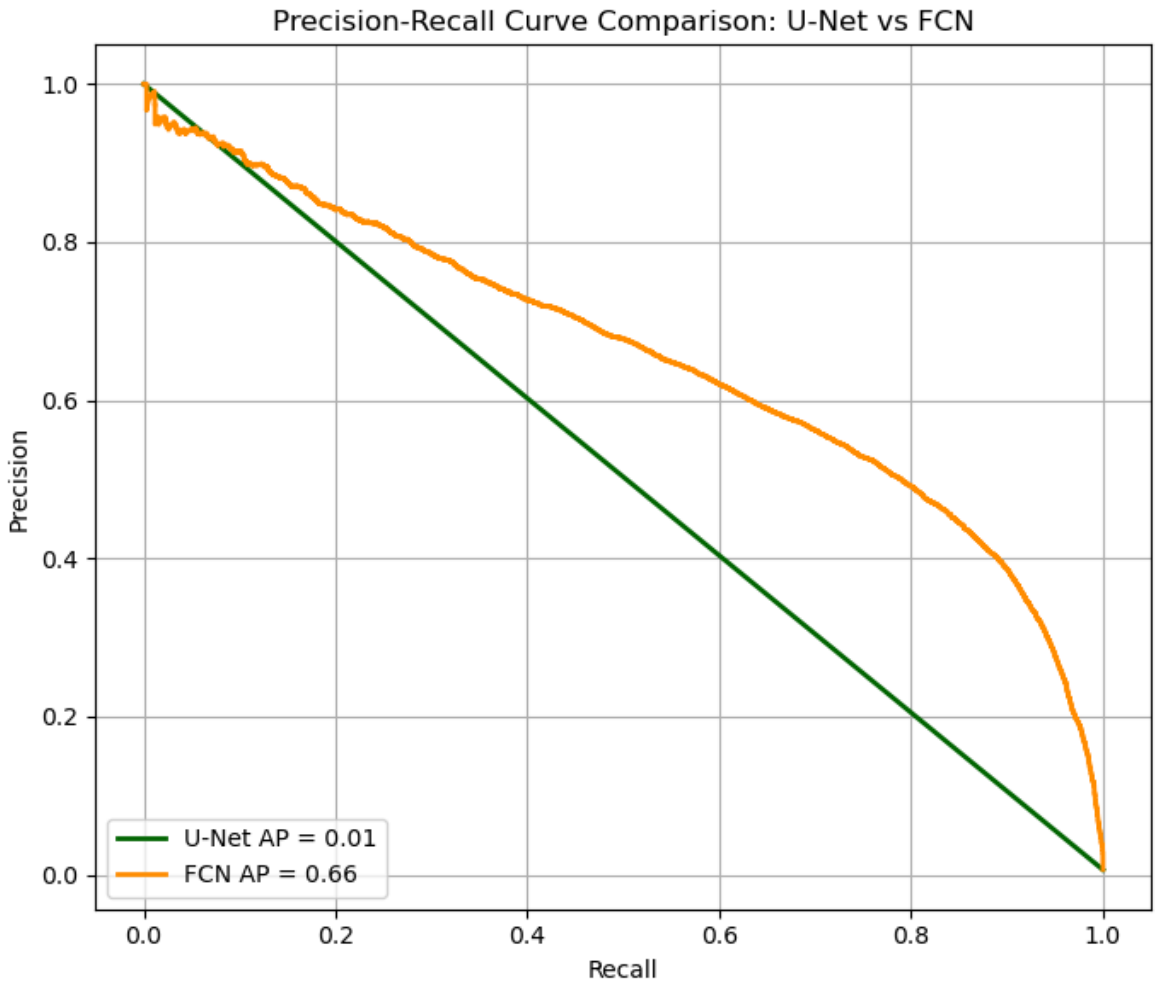


The fundamental difference lies in their current ability to handle the "Cells" class. The FCN is imperfect but functional, attempting to segment and finding a good portion of the cells. The U-Net is entirely non-functional for cell segmentation, having effectively "given up" on the minority class due to the overwhelming presence of background.

In [256…

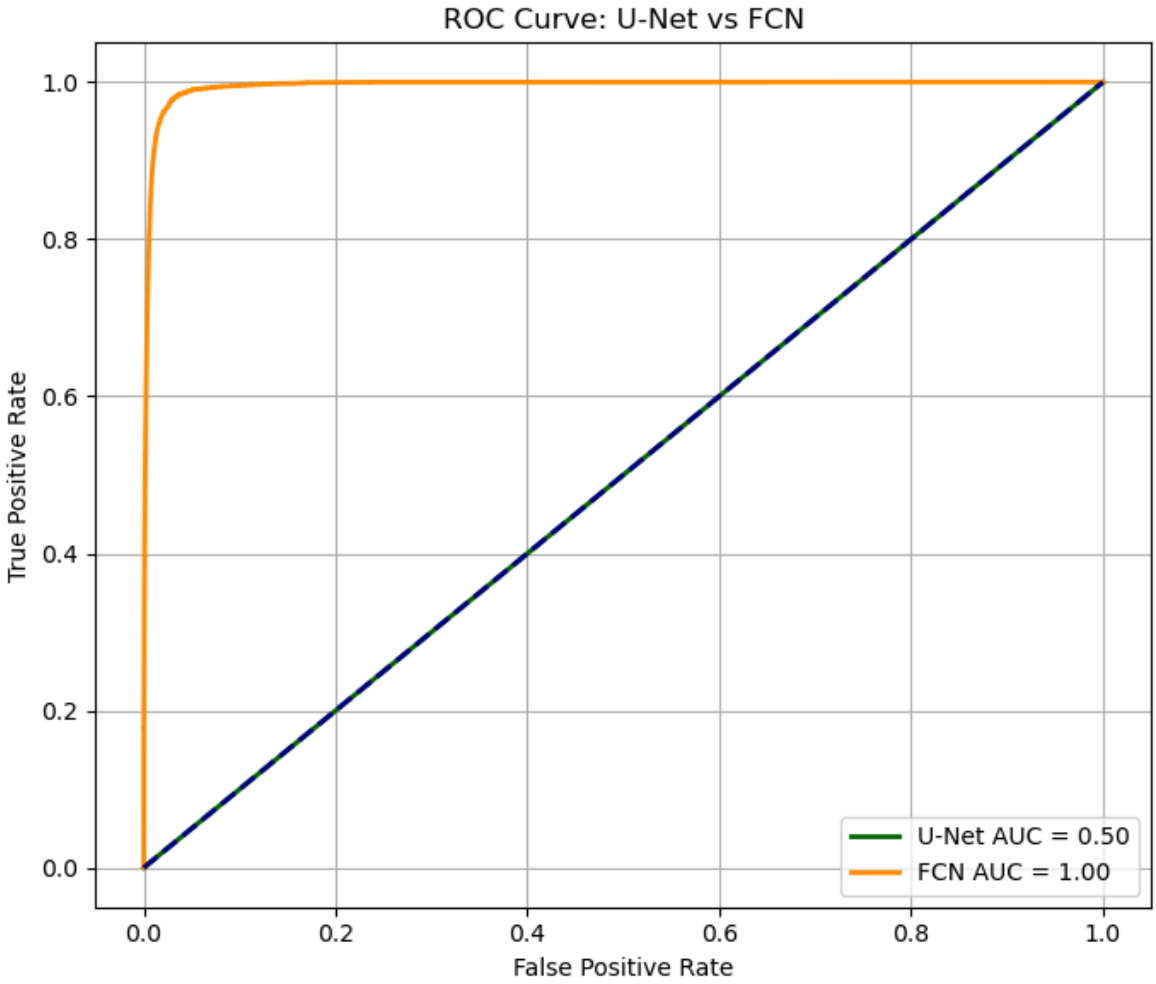
```
from sklearn.metrics import precision_recall_curve, average_precision_score
import matplotlib.pyplot as plt

precision_u, recall_u, _ = precision_recall_curve(actual1Du, predProbs1Du)
avg_precision_u = average_precision_score(actual1Du, predProbs1Du)

precision_f, recall_f, _ = precision_recall_curve(actual1D, predProbs1D)
avg_precision_f = average_precision_score(actual1D, predProbs1D)

plt.figure(figsize=(7, 6))
plt.plot(recall_u, precision_u, color='darkgreen', lw=2, label=f'U-Net AP = {avg
plt.plot(recall_f, precision_f, color='darkorange', lw=2, label=f'FCN AP = {avg_

plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve Comparison: U-Net vs FCN')
plt.legend(loc='lower left')
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```



In [258…

```
# Compute ROC and AUC for U-Net
fpr_u, tpr_u, _ = roc_curve(actual1Du, predProbs1Du)
roc_auc_u = auc(fpr_u, tpr_u)

# Compute ROC and AUC for FCN
fpr_f, tpr_f, _ = roc_curve(actual1D, predProbs1D)
roc_auc_f = auc(fpr_f, tpr_f)

# Plot both ROC curves
plt.figure(figsize=(7, 6))
plt.plot(fpr_u, tpr_u, color='darkgreen', lw=2, label=f'U-Net AUC = {roc_auc_u:.
plt.plot(fpr_f, tpr_f, color='darkorange', lw=2, label=f'FCN AUC = {roc_auc_f:.2
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve: U-Net vs FCN')
plt.legend(loc='lower right')
plt.grid(True)
plt.tight_layout()
plt.show()
```



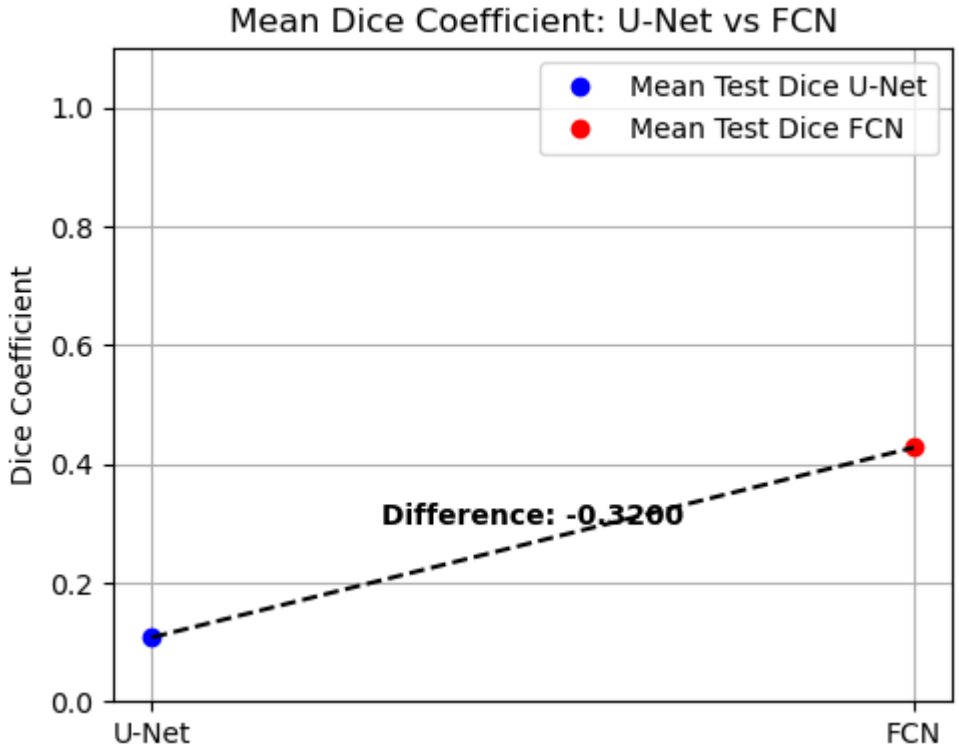
- The FCN model is demonstrating active learning and is capable of performing the cell segmentation task. It consistently finds a good portion of the cells (high recall) and shows a reasonable ability to distinguish them from the background. While it has some issues with precise boundary delineation and occasional over-segmentation (as seen in its lower precision and "blobby" visual predictions), it's a working model that provides meaningful output. Its Average Precision (AP) of 0.66 is a fair indicator of its performance on the foreground class.

- The U-Net, in its current state, is entirely failing at cell segmentation. It has learned to classify every single pixel as "Non-Cells" (background). It completely misses all actual cells, resulting in zero precision, zero recall, and zero F1-score for the "Cells" class. Visually, it produces only blank (background) masks even when cells are present. Its Average Precision (AP) of 0.01 and ROC AUC of 0.50 quantitatively confirm it's no better than random guessing for the foreground, or rather, it's consistently guessing "background.

In [313…

```
plt.figure(figsize=(5, 4))

# Plot points
plt.plot([0], [mean_dice_testu], 'bo', label='Mean Test Dice U-Net')
plt.plot([1], [mean_dice_test], 'ro', label='Mean Test Dice FCN')

# Draw line connecting the points
plt.plot([0, 1], [mean_dice_testu, mean_dice_test], 'k--')
```

```
# Calculate difference and plot as text
diff = mean_dice_testu - mean_dice_test
mid_x = 0.5
mid_y = (mean_dice_testu + mean_dice_test) / 2
plt.text(mid_x, mid_y + 0.03, f'Difference: {diff:.4f}', ha='center', fontsize=1

plt.xticks([0, 1], ['U-Net', 'FCN'])
plt.ylabel('Dice Coefficient')
plt.title('Mean Dice Coefficient: U-Net vs FCN')
plt.legend()
plt.ylim(0, 1.1)
plt.grid(True)
plt.tight_layout()
plt.show()
```



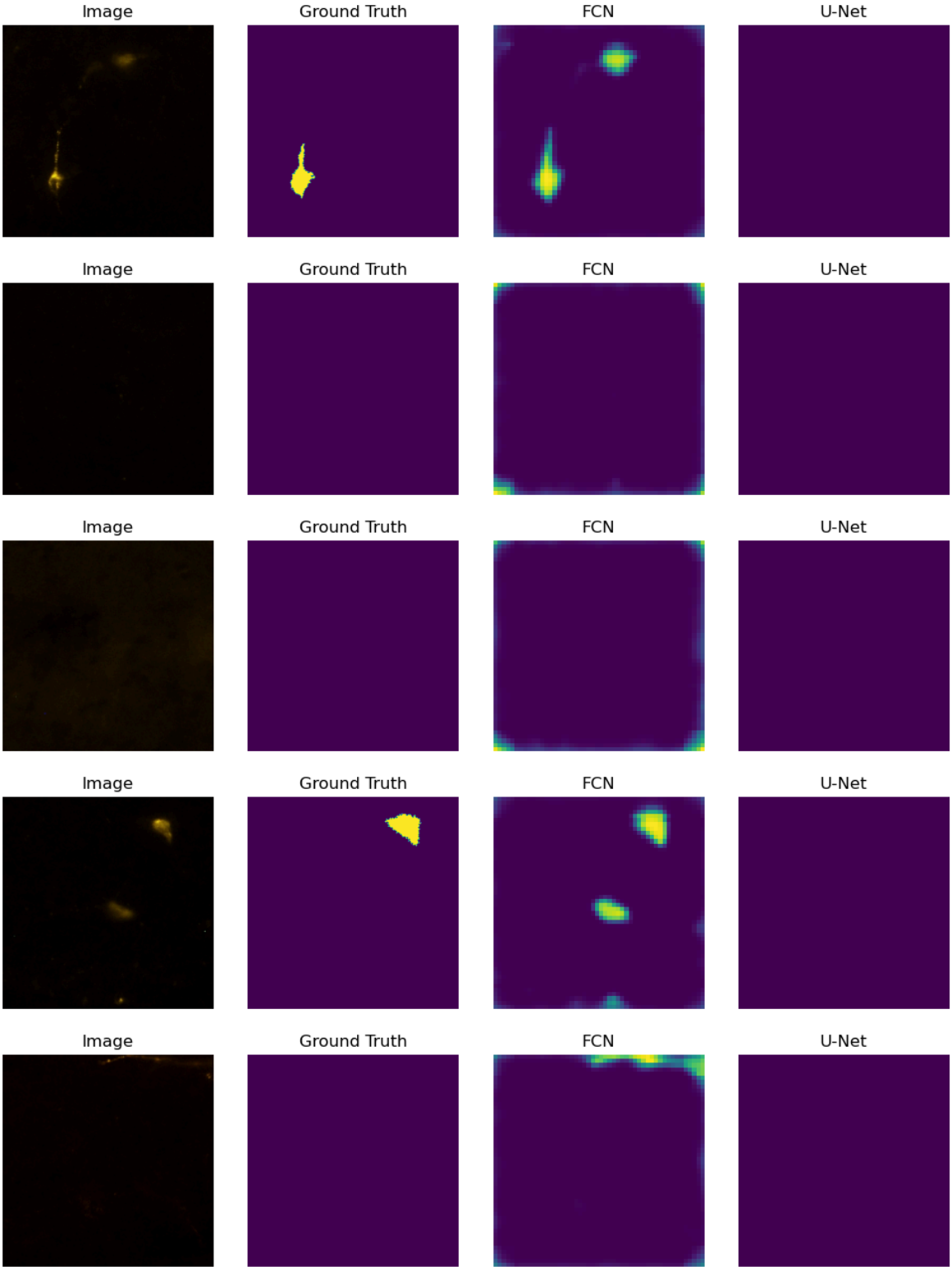
The difference of -0.3200 unequivocally demonstrates that the FCN model achieves a significantly higher Dice Coefficient on the test set compared to the U-Net model.

In [260…

```
# Get FCN and U-Net predictions
predictions_fcn = model.predict(np.array([np.array(img) for img in img_test]))
predictions_unet = unet_model.predict(np.array([np.array(img) for img in img_tes

# Display 5 samples: image, ground truth, FCN prediction, U-Net prediction
for img, mask, pred_fcn, pred_unet in itertools.islice(zip(img_test, mask_test,
    fig, ax = plt.subplots(1, 4, figsize=(10, 2.5))

    ax[0].imshow(img)
    ax[0].set_title('Image')
    ax[0].axis('off')

    ax[1].imshow(mask)
    ax[1].set_title('Ground Truth')
    ax[1].axis('off')

    ax[2].imshow(pred_fcn)
```

```
    ax[2].set_title('FCN')
    ax[2].axis('off')

    ax[3].imshow(pred_unet)
    ax[3].set_title('U-Net')
    ax[3].axis('off')

    plt.tight_layout()
    plt.show()
```

```
18/18 ━━━━━━━━━━━━━━━━━━━━ 44s 2s/step
18/18 ━━━━━━━━━━━━━━━━━━━━ 17s 921ms/step
```

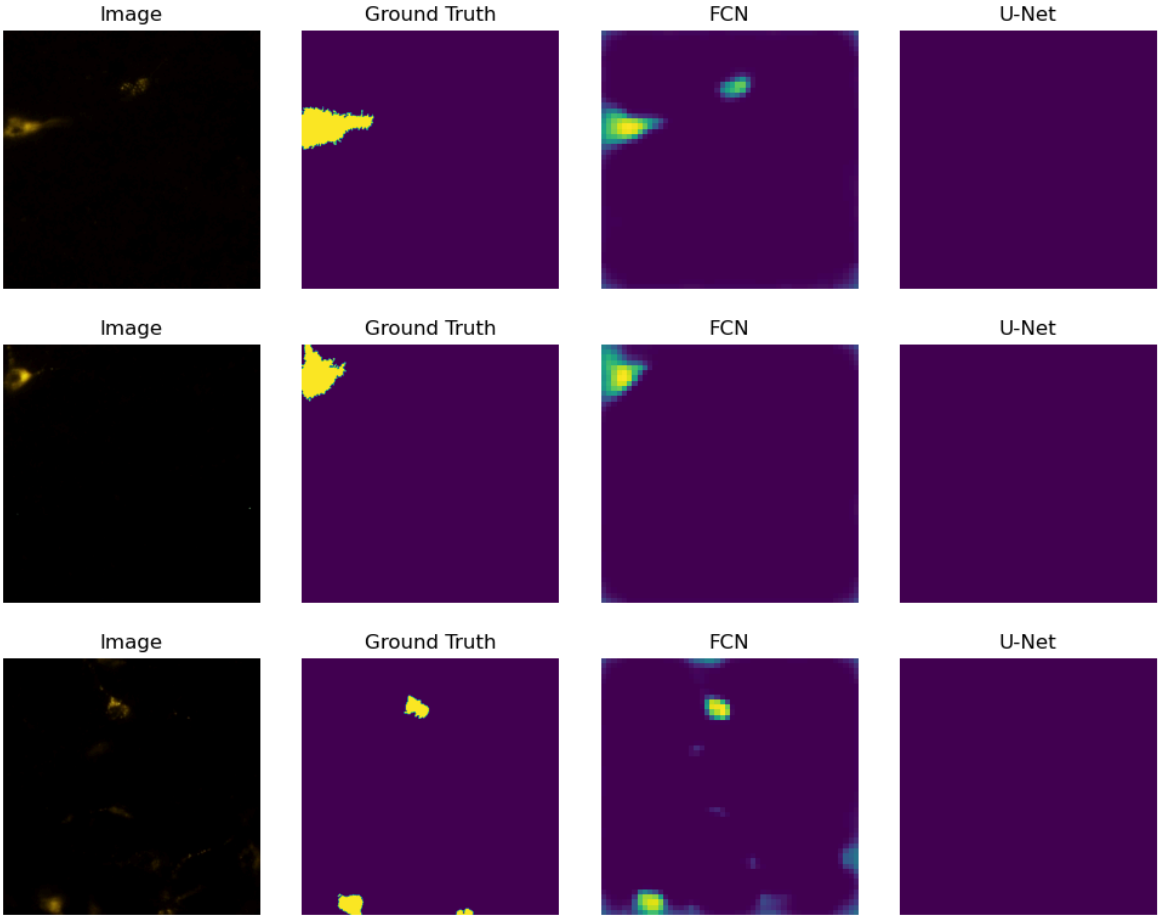


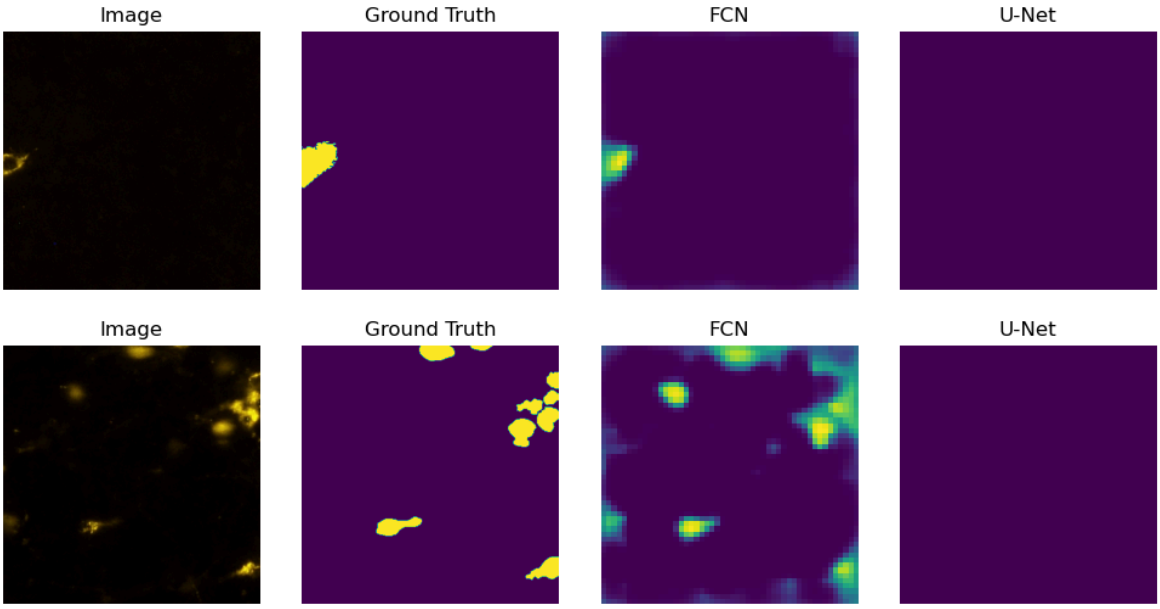
more visual failure of U-net model

In [263…

```
predictions_fcn2 = model.predict(np.array([np.array(img) for img in img_test_wit
predictions_unet2 = unet_model.predict(np.array([np.array(img) for img in img_te

# Display 5 samples: image, ground truth, FCN prediction, U-Net prediction
for img, mask, pred_fcn, pred_unet in itertools.islice(zip(img_test_with_cells,
    fig, ax = plt.subplots(1, 4, figsize=(10, 2.5))

    ax[0].imshow(img)
    ax[0].set_title('Image')
    ax[0].axis('off')

    ax[1].imshow(mask)
    ax[1].set_title('Ground Truth')
    ax[1].axis('off')

    ax[2].imshow(pred_fcn)
    ax[2].set_title('FCN')
    ax[2].axis('off')

    ax[3].imshow(pred_unet)
    ax[3].set_title('U-Net')
    ax[3].axis('off')

    plt.tight_layout()
    plt.show()
```

```
7/7 ━━━━━━━━━━━━━━━━━━━━ 15s 2s/step
7/7 ━━━━━━━━━━━━━━━━━━━━ 6s 779ms/step
```





Application of the trained models to an image

With the trained model, full-size images can now be processed. Each image is first split into 400x400 pixel sub-images, which are passed through the model. The predicted masks are then resized back to 400x400 and stitched together into one full mask.

In areas where the sub-images overlap, a "minimum" blending method is used, this keeps the lower pixel value in overlapping regions. This gives a more cautious prediction, which is helpful given the model's slight tendency for false positives.

An added benefit of this sub-image approach is that the model can be used on images of any size, as long as the cell sizes are similar to those seen during training.

In [266…

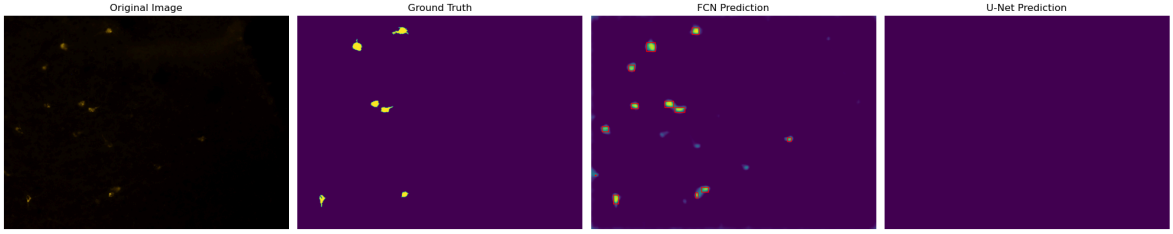
```
def applyModelToImage(pilImage, model, patch_size=400, overlap=100):
    width, height = pilImage.size
    partitions = EvenlySpacedPartitions(width, height, patch_size, overlap)

    subImages = []
    for partition in partitions:
        top, bot = partition.y, partition.y + partition.height
        left, right = partition.x, partition.x + partition.width
        croppedImg = pilImage.crop((left, top, right, bot))
        subImages.append(np.array(croppedImg))

    # Get model predictions
    predictions = model.predict(np.array(subImages))

    # Resize prediction masks
    predictionsUpscaled = []
    for pred in predictions:
        img = Image.fromarray(np.uint8(pred.reshape((50, 50)) * 255))
        upscaled = np.array(img.resize((400, 400)), dtype=np.uint8)
        predictionsUpscaled.append(upscaled)

    # Stitch back together
```

```
    output = np.full((height, width), 255, dtype=np.uint8)
    for pred, location in zip(predictionsUpscaled, partitions):
        top, bot = round(location.y), round(location.y + location.height)
        left, right = round(location.x), round(location.x + location.width)
        output[top:bot, left:right] = np.minimum(output[top:bot, left:right], pr

    return output
```

In [268…

```
# Choose image index
idx = 15

# Apply FCN and U-Net to same image
stitchedMask_fcn = applyModelToImage(imgListRaw[idx], model, overlap=100)
stitchedMask_unet = applyModelToImage(imgListRaw[idx], unet_model, overlap=100)

# Mark cell locations for visualization (if needed)
predictedCellLocations_fcn = markGroups(stitchedMask_fcn)

# Plot: Original | Ground Truth | FCN | U-Net
fig, ax = plt.subplots(1, 4, figsize=(20, 5))

ax[0].imshow(imgListRaw[idx])
ax[0].set_title('Original Image')
ax[0].axis('off')

ax[1].imshow(maskListRaw[idx])
ax[1].set_title('Ground Truth')
ax[1].axis('off')

ax[2].imshow(stitchedMask_fcn)
ax[2].set_title('FCN Prediction')
ax[2].axis('off')
# Optional: add rectangles
for cellLoc in predictedCellLocations_fcn:
    rect = patches.Rectangle((cellLoc.x, cellLoc.y), cellLoc.width, cellLoc.heig
                             linewidth=1, edgecolor='r', facecolor='none')
    ax[2].add_patch(rect)

ax[3].imshow(stitchedMask_unet)
ax[3].set_title('U-Net Prediction')
ax[3].axis('off')

plt.tight_layout()
plt.show()

# Optional: Print info
print("Number of cells detected by FCN (threshold=122):", len(predictedCellLocat
for i, loc in enumerate(predictedCellLocations_fcn, 1):
    print(f"{i}) {loc}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 2s 2s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 600ms/step
```



```
Number of cells detected by FCN (threshold=122): 12
1) CellLocation(nPixels=1076, x=566, y=65, width=39, height=38)
2) CellLocation(nPixels=1716, x=312, y=148, width=49, height=50)
3) CellLocation(nPixels=881, x=209, y=276, width=34, height=33)
4) CellLocation(nPixels=1264, x=415, y=478, width=48, height=35)
5) CellLocation(nPixels=776, x=225, y=492, width=38, height=29)
6) CellLocation(nPixels=1092, x=471, y=514, width=54, height=26)
7) CellLocation(nPixels=726, x=61, y=622, width=35, height=28)
8) CellLocation(nPixels=367, x=1100, y=684, width=25, height=21)
9) CellLocation(nPixels=25, x=20, y=893, width=8, height=4)
10) CellLocation(nPixels=795, x=619, y=962, width=40, height=27)
11) CellLocation(nPixels=308, x=587, y=993, width=13, height=29)
12) CellLocation(nPixels=1100, x=125, y=1006, width=30, height=52)
```

- The FCN model is actively performing segmentation. While it exhibits some minor inaccuracies in terms of precision and boundary definition, it successfully identifies the majority of the foreground objects. Its output is a meaningful attempt at the task.

- The U-Net model, in this instance, has completely failed to segment any cells. Its prediction of a solid background for an image containing clear foreground objects perfectly illustrates the severe issue of class imbalance that has plagued its training. It has learned to prioritize predicting the dominant background class, thereby ignoring the minority cell class entirely.

This visual evidence strongly aligns with the quantitative metrics previously discussed, where the FCN consistently shows moderate to good Dice coefficients and AP values, while the U-Net shows near-zero Dice and AP for the "Cells" class. The FCN is a working model that needs refinement, whereas the U-Net fundamentally requires addressing its inability to detect the foreground class.

Conclusions

This work explored image segmentation of fluorescent neuron images, a task characterized by a heavily imbalanced dataset where foreground (neuron cells) constituted only 1% to 2% of pixels in images even when cells were present. Two deep learning architectures, FCN and a light U-Net model, were evaluated through three training phases: two with images containing only cells and a final phase using the full dataset, including images without cells.

The FCN model consistently demonstrated superior performance compared to the U-Net. In its best evaluations, the FCN achieved a strong Mean Test Dice Coefficient of approximately 0.67 (or 0.43 in another test), and an Average Precision (AP) of 0.66 for the "Cells" class. Its classification report showed good recall (0.83) but lower precision (0.46) for cells, indicating it successfully finds most cells but struggles with precise boundary delineation and produces some over-segmentation. The FCN, despite its limitations, proved to be a functional and promising model for this challenging segmentation task.

In stark contrast, the U-Net model experienced a catastrophic failure when exposed to the full, highly imbalanced dataset in its final training phase. Initially, the U-Net showed promise with validation Dice coefficients around 0.60. However, the introduction of a large number of images without cells led to its complete collapse, resulting in a Mean Test Dice Coefficient of a mere 0.12, an AP of 0.01, and an ROC AUC of 0.50. Quantitatively, the U-Net achieved zero precision, recall, and F1-score for the "Cells" class, effectively learning to classify every single pixel as background. This severe issue highlights its vulnerability to extreme class imbalance, where it prioritized minimizing overall pixel-wise loss by ignoring the minority foreground class entirely.

Therefore, for segmenting fluorescent neurons in this context, the FCN model is the only currently viable solution, demonstrating a practical ability to detect cells despite the challenging dataset characteristics. The U-Net, while powerful in theory, requires fundamental adjustments to its training methodology to address the inherent class imbalance problem.

Future Perspectives

- Refined FCN Performance: The current FCN, with an AP of 0.66 and a Dice of up to 0.67, has a strong foundation. Future work can focus on pushing its performance closer to expert-level segmentation, particularly by improving its precision and boundary accuracy.
- Addressing U-Net's Limitations: Understanding and overcoming the U-Net's sensitivity to class imbalance is crucial. If successful, the U-Net's architectural strengths (e.g., skip connections for fine-grained detail) could potentially lead to even better segmentation quality.

The FCN model, despite its imperfections, is currently a valuable asset. We can:

- Automate Preliminary Segmentation: Use the FCN to generate initial segmentation masks for new fluorescent neuron images. This can significantly speed up the workflow compared to purely manual annotation.
- Semi-Automated Annotation: The FCN's predictions can serve as a strong starting point for human annotators, who can then quickly review and correct the masks. This greatly reduces the time and effort of manual labeling.
- Quantitative Analysis: Once refined, the FCN can be used to extract quantitative features from neuron images (e.g., number of cells, total cell area, cell morphology parameters).

In [ ]: